BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU ỨNG DỤNG AI TẠO SINH TRONG KIỂM THỬ XÂM NHẬP

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện:
Nguyễn Văn Khải - MSSV: AT190128

Người hướng dẫn:
TS. Vũ Thị Vân
Khoa An toàn thông tin - Học viện Kỹ thuật mật mã

Hà Nội, 2026

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU ỨNG DỤNG AI TẠO SINH TRONG KIỂM THỬ XÂM NHẬP

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện:
Nguyễn Văn Khải - MSSV: AT190128

Người hướng dẫn:
TS. Vũ Thị Vân
Khoa An toàn thông tin - Học viện Kỹ thuật mật mã

Hà Nội, 2026

MỤC LỤC

Danh mục kí hiệu và viết tắt ..... **iii**
Danh mục hình ảnh ..... **v**
Danh mục bảng ..... **vi**
Lời cảm ơn ..... **vii**
Lời nói đầu ..... **viii**
Chương 1. Tổng quan về kiểm thử xâm nhập và AI tạo sinh **1**
1.1. Tổng quan về kiểm thử xâm nhập 1
1.1.1. Khái niệm và phân loại kiểm thử xâm nhập *1*
1.1.2. Quy trình kiểm thử xâm nhập *2*
1.1.3. Các khung tiêu chuẩn và phương pháp kiểm thử xâm nhập *3*
1.2. Tổng quan về an toàn ứng dụng web và các lỗ hổng phổ biến 4
1.2.1. Tổng quan về OWASP Top 10 ..... *4*
1.2.2. Một số lỗ hổng bảo mật ứng dụng web phổ biến *5*
1.3. Tổng quan về AI tạo sinh và mô hình ngôn ngữ lớn 7
1.3.1. Khái niệm AI tạo sinh và đặc trưng nổi bật *7*
1.3.2. Kiến trúc Transformer và cơ chế hoạt động của LLM *8*
1.3.3. Prompt Engineering và Chain-of-Thought *10*
1.3.4. Cơ chế ReAct trong mô hình ngôn ngữ lớn *11*
1.3.5. AI Agent dựa trên LLM và cơ chế Tool Calling *12*
1.4. Ứng dụng AI tạo sinh trong kiểm thử xâm nhập 15
1.4.1. Hạn chế của kiểm thử truyền thống và vai trò của AI tạo sinh *15*
1.4.2. Các hướng ứng dụng AI tạo sinh trong kiểm thử xâm nhập *17*
1.4.3. Thách thức và xu hướng phát triển *19*
1.5. Kết luận chương 1 21
Chương 2. Phân tích và thiết kế hệ thống AI Pentest Agent **22**
2.1. Mô tả bài toán 22
2.2. Các yêu cầu chức năng và yêu cầu phi chức năng 22
2.2.1. Yêu cầu chức năng *22*
2.2.2. Yêu cầu phi chức năng *23*
2.3. Thiết kế kiến trúc hệ thống 24
2.3.1. Kiến trúc tổng quát hệ thống *24*
2.3.2. Kiến trúc các thành phần của hệ thống *27*
2.4. Thiết kế luồng xử lý và cơ chế hoạt động 28
2.4.1. Quy trình hoạt động của hệ thống *28*
2.4.2. Cơ chế quản lý ngữ cảnh và trạng thái phiên *32*
2.4.3. Cơ chế xử lý và thực thi công cụ *36*
2.5. Thiết kế các mô-đun chức năng hệ thống 38
2.5.1. Mô-đun điều phối và lập kế hoạch tác vụ *38*
2.5.2. Mô-đun tri thức và kỹ năng kiểm thử *43*
2.5.3. Mô-đun quản lý và thực thi công cụ *49*
2.5.4. Mô-đun quản lý phiên và bộ nhớ *52*
2.5.5. Mô-đun trạng thái và kết quả kiểm thử *56*

2.6. Kết luận chương 2 63
Chương 3. Xây dựng và đánh giá hệ thống AI Pentest Agent 64
3.1. Môi trường và công nghệ sử dụng 64
3.2. Hiện thực và triển khai hệ thống 65
3.2.1. Tổ chức mã nguồn và khởi tạo hệ thống 65
3.2.2. Hiện thực các chức năng chính 66
3.2.3. Giao diện và tương tác với hệ thống 74
3.2.4. Cấu hình và triển khai hệ thống 76
3.3. Thực nghiệm và đánh giá kết quả 77
3.3.1. Thực nghiệm kiểm thử xâm nhập ứng dụng web 77
3.3.2. Thực nghiệm phối hợp KAgent với Burp Suite 85
3.4. Hạn chế và hướng phát triển 89
3.4.1. Hạn chế của hệ thống 89
3.4.2. Định hướng phát triển hệ thống 91
3.5. Kết luận chương 3 93
Kết luận 94
Tài liệu tham khảo 96
Phụ lục 98

DANH MỤC KÍ HIỆU VÀ VIẾT TẮT

Từ viết tắt / Ký hiệu	Tiếng Anh	Tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge	Khung tri thức về chiến thuật, kỹ thuật và kiến thức phổ biến về tấn công
CAPEC	Common Attack Pattern Enumeration and Classification	Danh mục phân loại các mẫu tấn công phổ biến
CLI	Command Line Interface	Giao diện dòng lệnh
CoT	Chain-of-Thought	Chuỗi suy luận
CSRF	Cross-Site Request Forgery	Giả mạo yêu cầu liên trang
CVE	Common Vulnerabilities and Exposures	Danh mục định danh các lỗ hổng bảo mật được công bố
CWE	Common Weakness Enumeration	Danh mục phân loại các điểm yếu phổ biến
DOM	Document Object Model	Mô hình đối tượng tài liệu
GPT	Generative Pre-trained Transformer	Mô hình Transformer tạo sinh được tiền huấn luyện
HTML	Hypertext Markup Language	Ngôn ngữ đánh dấu siêu văn bản
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
IDOR	Insecure Direct Object References	Tham chiếu đối tượng trực tiếp không an toàn
IP	Internet Protocol	Giao thức Internet
JSON	JavaScript Object Notation	Định dạng biểu diễn dữ liệu dạng văn bản
JSONL	JSON Lines	Định dạng dữ liệu JSON theo từng dòng
LLM	Large Language Model	Mô hình ngôn ngữ lớn

Từ viết tắt / Ký hiệu	Tiếng Anh	Tiếng Việt
NIST	National Institute of Standards and Technology	Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ
NVD	National Vulnerability Database	Cơ sở dữ liệu lỗ hổng quốc gia
OWASP	Open Worldwide Application Security Project	Tổ chức phi lợi nhuận hoạt động vì an toàn phần mềm
PTES	Penetration Testing Execution Standard	Chuẩn thực hiện kiểm thử xâm nhập
RAG	Retrieval-Augmented Generation	Sinh tăng cường bằng truy xuất
ReAct	Reasoning and Acting	Cơ chế kết hợp suy luận và hành động
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
SQLi	SQL Injection	Tấn công chèn câu lệnh SQL
SSRF	Server-Side Request Forgery	Giả mạo yêu cầu phía máy chủ
TUI	Text User Interface	Giao diện người dùng dạng văn bản
WSTG	Web Security Testing Guide	Hướng dẫn kiểm thử an toàn ứng dụng web
XSS	Cross-Site Scripting	Tấn công chèn mã kịch bản liên trang
YAML	YAML Ain’t Markup Language	Ngôn ngữ tuần tự hóa dữ liệu

DANH MỤC HÌNH ẢNH

Hình 1.1. Kiến trúc tổng quát của Transformer [5] 8
Hình 1.2. So sánh các phương pháp Prompting và cơ chế ReAct [8]..... 12
Hình 1.3. Kiến trúc tổng quát của hệ thống LLM-based Agent [10]..... 13
Hình 2.1. Kiến trúc tổng quát hệ thống..... 25
Hình 2.2. Quy trình hoạt động tổng quát của KAgent..... 29
Hình 2.3. Quy trình xử lý một lượt tương tác của KAgent..... 31
Hình 2.4. Quy trình xây dựng ngữ cảnh cho một lượt tương tác KAgent..... 34
Hình 2.5. Cơ chế thu gọn ngữ cảnh và khôi phục phiên của KAgent..... 35
Hình 2.6. Cơ chế xử lý và thực thi một yêu cầu gọi công cụ của KAgent..... 37
Hình 2.7. Cơ chế điều phối vòng lặp Agent..... 39
Hình 2.8. Quy trình lập kế hoạch của Decision Planner..... 41
Hình 2.9. Cơ chế học và tích lũy tri thức..... 45
Hình 2.10. Quy trình lựa chọn và nạp kỹ năng..... 48
Hình 2.11. Cơ chế đồng bộ và truy vấn Coverage..... 60
Hình 2.12. Quy trình xử lý của Confirm Finding..... 62
Hình 3.1. Minh họa vòng lặp LLM và công cụ trong KAgent..... 67
Hình 3.2. Giao diện chọn nhà cung cấp và mô hình LLM..... 68
Hình 3.3. Nạp kỹ năng và thực thi công cụ trên KAgent..... 69
Hình 3.4. Liên kết dữ liệu xác minh và cập nhật kết quả trong KAgent..... 72
Hình 3.5. Giao diện thành phần mở rộng KAgent trong Burp Suite..... 73
Hình 3.6. Giao diện chính của KAgent..... 74
Hình 3.7. Hộp thoại xác nhận quyền thực thi công cụ..... 75
Hình 3.8. Quy trình kiểm thử của KAgent..... 78
Hình 3.9. KAgent kích hoạt kỹ năng theo từng giai đoạn kiểm thử..... 79
Hình 3.10. KAgent khám phá tài nguyên và ghi nhận đầu vào kiểm thử..... 80
Hình 3.11. KAgent yêu cầu xác định phương án xử lý các đầu vào B2B..... 80
Hình 3.12. Điều chỉnh phép thử SQLi theo phản hồi của ứng dụng..... 82
Hình 3.13. Lưu hai Finding SQL Injection sau xác minh..... 83
Hình 3.14. Quy trình phối hợp Burp Suite và KAgent..... 86
Hình 3.15. Chuyển yêu cầu tìm kiếm từ Burp Repeater tới KAgent..... 87
Hình 3.16. Nạp Skill XSS và triển khai xác minh tham số search..... 87
Hình 3.17. Kết quả xác minh và thông báo lưu Finding XSS..... 89

DANH MỤC BẢNG

Bảng 1.1. So sánh các phương pháp luận kiểm thử xâm nhập 3
Bảng 1.2. Các nhóm rủi ro trong OWASP Top 10:2021 5
Bảng 1.3. So sánh các hướng ứng dụng AI trong kiểm thử xâm nhập 18
Bảng 2.1. Các yêu cầu chức năng của hệ thống 23
Bảng 2.2. Các yêu cầu phi chức năng của hệ thống 24
Bảng 2.3. Cấu trúc trạng thái và các nguồn dữ liệu hỗ trợ của KAgent 33
Bảng 2.4. Các thành phần của DecisionPlan 43
Bảng 2.5. Các nhóm thông tin chính của IntelligenceScenario 44
Bảng 2.6. Các nhóm metadata chính của kỹ năng kiểm thử 46
Bảng 2.7. Các thành phần chính của công cụ 49
Bảng 2.8. Phân nhóm công cụ theo mục đích sử dụng 51
Bảng 2.9. Các thành phần chính của SessionFile 53
Bảng 2.10. Các nhóm thông tin chính của SessionMemory 54
Bảng 2.11. Các thuộc tính chính của cấu trúc MemoryFact 55
Bảng 2.12. Các thành phần dữ liệu chính của WorkflowState 57
Bảng 2.13. Các thành phần dữ liệu chính của CoverageEntry 58
Bảng 2.14. Các thành phần dữ liệu chính của Finding 61
Bảng 3.1. Các công nghệ và thư viện chính sử dụng trong KAgent 64
Bảng 3.2. Các nhóm mã nguồn chính của KAgent 65
Bảng 3.3. Các nhóm thiết lập phục vụ khởi chạy và vận hành KAgent 76
Bảng 3.4. Kết quả xác minh SQL Injection tại ba Candidate 82
Bảng 3.5. Sử dụng kỹ năng và công cụ trong thực nghiệm 83
Bảng 3.6. Tổng hợp kết quả qua các giai đoạn thực nghiệm 84
Bảng 3.7. Các phép thử và kết quả phân tích tại tham số search 88

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án tốt nghiệp, tôi đã nhận được sự hướng dẫn tận tình của TS. Vũ Thị Vân - Giảng viên Khoa An toàn thông tin, Học viện Kỹ thuật Mật mã. Tôi xin được bày tỏ lòng biết ơn sâu sắc tới cô vì những chỉ dẫn quý báu và sự hỗ trợ nhiệt tình trong suốt thời gian nghiên cứu và hoàn thiện đề tài!

SINH VIÊN THỰC HIỆN ĐỀ TÀI

Nguyễn Văn Khải

LỜI NÓI ĐẦU

Các hệ thống thông tin, ứng dụng web và dịch vụ trực tuyến ngày càng được triển khai rộng rãi, đặt ra yêu cầu bảo đảm an toàn trước khi đưa vào vận hành. Các lỗ hổng bảo mật trong ứng dụng có thể bị khai thác để truy cập trái phép, thay đổi hoặc đánh cắp dữ liệu, từ đó gây ảnh hưởng đến tính an toàn và hoạt động của hệ thống. Kiểm thử xâm nhập (penetration testing) là một phương pháp quan trọng nhằm phát hiện và đánh giá các điểm yếu bảo mật thông qua việc mô phỏng các kỹ thuật tấn công trong phạm vi được cho phép. Tuy nhiên, quá trình kiểm thử vẫn phụ thuộc đáng kể vào kiến thức và kinh nghiệm của kiểm thử viên. Kiểm thử viên thường phải thu thập thông tin, phân tích kết quả, lập kế hoạch, lựa chọn công cụ và xác định bước kiểm thử tiếp theo qua nhiều vòng xử lý, làm tăng thời gian và nguồn lực cần thiết.

Cùng với sự phát triển của trí tuệ nhân tạo tạo sinh (Generative AI), đặc biệt là các mô hình ngôn ngữ lớn (Large Language Model - LLM), nhiều nghiên cứu đã bắt đầu khai thác khả năng của AI trong việc hỗ trợ các hoạt động kiểm thử xâm nhập. LLM có khả năng xử lý và tổng hợp thông tin, lập kế hoạch, sinh lệnh hoặc hành động và tương tác với các công cụ bên ngoài. Trên cơ sở đó, nhiều hướng nghiên cứu đã được đề xuất nhằm xây dựng các tác nhân AI có khả năng hỗ trợ một phần hoặc một số giai đoạn của quy trình kiểm thử xâm nhập. Tuy nhiên, để LLM có thể tham gia hiệu quả vào một quy trình kiểm thử gồm nhiều bước liên tiếp, vẫn cần giải quyết các vấn đề như duy trì ngữ cảnh, quản lý trạng thái phiên, lựa chọn và điều phối công cụ, kiểm soát hành động, xử lý kết quả và xác định bước thực hiện tiếp theo. Những vấn đề này ảnh hưởng trực tiếp đến khả năng hoạt động ổn định và mức độ tự động hóa của AI Pentest Agent.

Từ thực tiễn kiểm thử xâm nhập và xu hướng nghiên cứu ứng dụng AI tạo sinh nêu trên, việc nghiên cứu cơ chế kết hợp mô hình ngôn ngữ với các công cụ kiểm thử là cần thiết. Đề tài "Nghiên cứu ứng dụng AI tạo sinh trong kiểm thử xâm nhập" được thực hiện với mục tiêu nghiên cứu và xây dựng một nguyên mẫu AI Pentest Agent theo kiến trúc tác nhân đơn (Single-Agent), sử dụng LLM để hỗ trợ một phần quy trình kiểm thử xâm nhập ứng dụng web. Đồ án tập trung giải quyết các vấn đề chính gồm cơ chế tương tác giữa Agent và LLM, quản lý ngữ cảnh và trạng thái phiên, cơ chế suy luận và thực hiện hành động, lựa chọn và điều phối công cụ, xử lý kết quả trả về và duy trì chu trình suy luận - hành động - quan sát. Hệ thống được xây dựng và thực nghiệm trong môi trường kiểm thử được kiểm soát, tập trung

đánh giá khả năng hỗ trợ kiểm thử đối với một số lỗ hổng web tiêu biểu như SQL Injection (SQLi) và Cross-Site Scripting (XSS).

Đồ án được tổ chức thành 03 chương:

Chương 1: Tổng quan về kiểm thử xâm nhập và AI tạo sinh. Chương này trình bày các kiến thức cơ bản về kiểm thử xâm nhập, AI tạo sinh và các hướng ứng dụng AI trong kiểm thử xâm nhập.

Chương 2: Phân tích và thiết kế hệ thống AI Pentest Agent. Chương này trình bày bài toán, yêu cầu, kiến trúc và cơ chế hoạt động của hệ thống.

Chương 3: Xây dựng, thực nghiệm và đánh giá hệ thống AI Pentest Agent. Chương này trình bày quá trình xây dựng, thực nghiệm và đánh giá hệ thống, đồng thời đề xuất hướng phát triển.

SINH VIÊN THỰC HIỆN ĐỀ TÀI

Nguyễn Văn Khải

CHƯƠNG 1. TỔNG QUAN VỀ KIỂM THỬ XÂM NHẬP VÀ AI TẠO SINH

1.1. Tổng quan về kiểm thử xâm nhập

1.1.1. Khái niệm và phân loại kiểm thử xâm nhập

Kiểm thử xâm nhập (Penetration Testing) là một hoạt động đánh giá an toàn thông tin mang tính chủ động, trong đó các chuyên gia bảo mật được ủy quyền mô phỏng các cuộc tấn công trong môi trường được kiểm soát nhằm xác định, xác minh khả năng khai thác và đánh giá mức độ ảnh hưởng của các lỗ hổng bảo mật tồn tại trong hạ tầng công nghệ thông tin của tổ chức. Hoạt động này có thể được thực hiện trên nhiều đối tượng khác nhau như hệ thống, mạng, ứng dụng và thiết bị để đánh giá mức độ an toàn cũng như khả năng chống chịu trước các cuộc tấn công.

Thông qua việc mô phỏng các chiến thuật, kỹ thuật và quy trình tấn công (Tactics, Techniques, and Procedures - TTPs) mà các tác nhân độc hại thường sử dụng, kiểm thử xâm nhập giúp tổ chức đánh giá hiệu quả của các biện pháp bảo vệ hiện có, xác định và ưu tiên khắc phục các lỗ hổng bảo mật, đồng thời hỗ trợ đáp ứng các yêu cầu về tiêu chuẩn, quy định an toàn thông tin và nâng cao mức độ bảo mật tổng thể của hệ thống.

Dựa trên mức độ thông tin được cung cấp về hệ thống mục tiêu, kiểm thử xâm nhập được chia thành ba hình thức chính: kiểm thử hộp đen (Black-box Testing), kiểm thử hộp xám (Gray-box Testing) và kiểm thử hộp trắng (White-box Testing).

a) Kiểm thử hộp đen

Người kiểm thử không hoặc chỉ được cung cấp ít thông tin về cấu trúc và cơ chế nội bộ của hệ thống mục tiêu. Mục tiêu, phạm vi và quy tắc thực hiện vẫn được xác định trước. Hình thức này có thể được áp dụng khi mô phỏng góc nhìn của tác nhân bên ngoài hoặc trong các điều kiện kiểm thử khác [1].

b) Kiểm thử hộp xám

Chuyên gia kiểm thử được cung cấp một phần thông tin về hệ thống, chẳng hạn như tài khoản người dùng, tài liệu thiết kế ở mức cơ bản hoặc sơ đồ kiến trúc. Việc có sẵn một số thông tin giúp quá trình kiểm thử hiệu quả hơn nhưng vẫn phản ánh tương đối sát các kịch bản tấn công trong thực tế.

c) Kiểm thử hộp trắng

Chuyên gia kiểm thử được cung cấp đầy đủ thông tin về hệ thống, bao gồm mã nguồn, kiến trúc, tài liệu thiết kế và cấu hình. Điều này cho phép quá trình kiểm

thử được thực hiện toàn diện hơn, giúp phát hiện cả những lỗ hổng khó nhận biết bằng các phương pháp kiểm thử từ bên ngoài.

1.1.2. Quy trình kiểm thử xâm nhập

Quy trình kiểm thử xâm nhập giúp tổ chức các hoạt động đánh giá theo trình tự thống nhất, từ xác định phạm vi đến tổng hợp kết quả. Trong đó, Penetration Testing Execution Standard (PTES) cung cấp phương pháp luận bao quát vòng đời của một cuộc kiểm thử, làm cơ sở để lựa chọn và phối hợp các hoạt động phù hợp với mục tiêu đánh giá. [2]

Theo PTES, quy trình kiểm thử xâm nhập được chia thành bảy giai đoạn chính, bao gồm: Trao đổi và chuẩn bị trước kiểm thử (Pre-engagement Interactions), Thu thập thông tin (Intelligence Gathering), Mô hình hóa mối đe dọa (Threat Modeling), Phân tích lỗ hổng (Vulnerability Analysis), Khai thác lỗ hổng (Exploitation), Hậu khai thác (Post Exploitation) và Báo cáo kết quả (Reporting).

a) Trao đổi và chuẩn bị trước kiểm thử

Nhóm kiểm thử và khách hàng thống nhất mục tiêu, phạm vi, phương pháp, thời gian và các quy tắc thực hiện (Rules of Engagement). Giai đoạn này cũng xác định quyền hạn, trách nhiệm và cách xử lý tình huống phát sinh, tạo cơ sở để các hoạt động tiếp theo diễn ra trong phạm vi được cấp phép và hạn chế tác động ngoài mong muốn.

b) Thu thập thông tin

Thông tin về mục tiêu được thu thập bằng các phương pháp thụ động hoặc chủ động, gồm tên miền, địa chỉ IP, dịch vụ, công nghệ và các thành phần liên quan. Kết quả thu thập giúp người kiểm thử hình dung cấu trúc hệ thống, nhận diện các điểm cần xem xét và chuẩn bị dữ liệu cho quá trình phân tích tiếp theo.

c) Mô hình hóa mối đe dọa

Dựa trên thông tin đã thu thập, người kiểm thử xác định các tài sản quan trọng, mối đe dọa tiềm ẩn và bề mặt tấn công. Các kịch bản tấn công được xây dựng và ưu tiên theo mức độ rủi ro, qua đó định hướng việc phân tích lỗ hổng và lựa chọn đối tượng cần kiểm tra sâu hơn.

d) Phân tích lỗ hổng

Giai đoạn này sử dụng công cụ hỗ trợ kết hợp với kiểm tra thủ công để nhận diện các điểm yếu có thể tồn tại trong hệ thống. Người kiểm thử tiếp tục phân tích khả năng khai thác và mức độ ảnh hưởng của những dấu hiệu đã phát hiện, làm cơ sở lựa chọn các giả thuyết cần kiểm chứng ở bước khai thác.

e) Khai thác lỗ hổng

Người kiểm thử thực hiện các thao tác khai thác có kiểm soát để kiểm chứng lỗ hổng và đánh giá tác động thực tế. Hoạt động này tuân theo phạm vi đã thỏa thuận, đồng thời thu thập bằng chứng về kết quả đạt được để phục vụ đánh giá rủi ro và lập báo cáo.

f) Hậu khai thác

Sau khi khai thác thành công, người kiểm thử đánh giá mức độ truy cập và kiểm soát có thể đạt được, chẳng hạn khả năng leo thang đặc quyền, tiếp cận dữ liệu quan trọng hoặc di chuyển giữa các hệ thống. Các hoạt động được lựa chọn theo mục tiêu và phạm vi cho phép nhằm làm rõ hậu quả của việc lỗ hổng bị khai thác.

g) Báo cáo kết quả

Kết quả kiểm thử được tổng hợp thành báo cáo gồm mô tả lỗ hổng, bằng chứng, mức độ rủi ro, tác động và khuyến nghị khắc phục. Báo cáo là căn cứ để tổ chức ưu tiên xử lý điểm yếu và kiểm tra lại hiệu quả của biện pháp khắc phục.

1.1.3. Các khung tiêu chuẩn và phương pháp kiểm thử xâm nhập

Bên cạnh PTES, nhiều khung tiêu chuẩn và phương pháp luận khác cũng được sử dụng trong lĩnh vực kiểm thử xâm nhập nhằm hướng dẫn và chuẩn hóa quá trình đánh giá an toàn thông tin. Mỗi khung có mục tiêu, phạm vi áp dụng và đặc điểm riêng. Bảng 1.1 trình bày một số khung tiêu chuẩn và phương pháp luận kiểm thử xâm nhập được sử dụng phổ biến hiện nay. [1], [2], [3]

Bảng 1.1. So sánh các phương pháp luận kiểm thử xâm nhập

Khung / Phương pháp luận	**Đơn vị phát triển**	**Mục tiêu chính**	**Ưu điểm**	**Hạn chế**
NIST SP 800-115	NIST	Hướng dẫn hoạt động đánh giá an toàn thông tin.	Có tính hệ thống, được sử dụng rộng rãi làm tài liệu tham chiếu.	Cung cấp tổng quan kỹ thuật và quy trình đánh giá, không thay thế hướng dẫn kiểm thử chi tiết cho từng loại lỗ hổng hoặc ứng dụng.

Khung / Phương pháp luận	Đơn vị phát triển	Mục tiêu chính	Ưu điểm	Hạn chế
PTES	PTES Team	Chuẩn hóa quy trình thực hiện kiểm thử xâm nhập.	Quy trình rõ ràng, bao quát toàn bộ vòng đời của một cuộc kiểm thử xâm nhập.	Các giai đoạn cần được cụ thể hóa theo mục tiêu, phạm vi và môi trường kiểm thử.
OWASP WSTG	OWASP Foundation	Hướng dẫn kiểm thử bảo mật ứng dụng web.	Nội dung chi tiết, được cập nhật thường xuyên và sử dụng rộng rãi trong kiểm thử web.	Tập trung vào kiểm thử bảo mật ứng dụng web và dịch vụ web, không bao quát các loại hệ thống khác.

1.2. Tổng quan về an toàn ứng dụng web và các lỗ hổng phổ biến

Ứng dụng web thường tiếp nhận dữ liệu từ người dùng và kết nối với cơ sở dữ liệu, API hoặc dịch vụ bên ngoài. Các lỗi trong xử lý đầu vào, kiểm soát truy cập, thiết kế hoặc cấu hình có thể tạo ra điểm yếu cho phép truy cập trái phép, làm lộ dữ liệu hay ảnh hưởng đến hoạt động của ứng dụng.

An toàn ứng dụng web hướng tới bảo đảm tính bí mật, toàn vẹn và sẵn sàng của dữ liệu và dịch vụ. Trong kiểm thử xâm nhập, việc nắm được các nhóm rủi ro và lỗ hổng phổ biến giúp định hướng khảo sát bề mặt tấn công, lựa chọn đầu vào cần kiểm tra và đánh giá kết quả.

1.2.1. Tổng quan về OWASP Top 10

OWASP Foundation là tổ chức phi lợi nhuận hoạt động trong lĩnh vực an toàn phần mềm, cung cấp các tài liệu, công cụ và hướng dẫn nhằm hỗ trợ phát triển ứng dụng an toàn. OWASP Top 10 là tài liệu tổng hợp mười nhóm rủi ro bảo mật quan trọng đối với ứng dụng web, được sử dụng để nâng cao nhận thức và hỗ trợ nhận diện các điểm yếu trong quá trình phát triển, vận hành và kiểm thử ứng dụng.

Các nhóm rủi ro thuộc phiên bản OWASP Top 10:2021 và nội dung mô tả tương ứng được trình bày trong Bảng 1.2. [4]

Bảng 1.2. Các nhóm rủi ro trong OWASP Top 10:2021

Mã	Nhóm rủi ro	Mô tả
A01	Broken Access Control	Kiểm soát truy cập không đầy đủ, cho phép người dùng thực hiện các hành động vượt quá quyền hạn được cấp.
A02	Cryptographic Failures	Các vấn đề liên quan đến việc bảo vệ dữ liệu bằng cơ chế mã hóa không phù hợp hoặc không đầy đủ, dẫn đến nguy cơ lộ thông tin nhạy cảm.
A03	Injection	Lỗ hổng xảy ra khi dữ liệu đầu vào không được kiểm soát cho phép kẻ tấn công chèn các câu lệnh hoặc mã độc vào hệ thống.
A04	Insecure Design	Các điểm yếu phát sinh từ quá trình thiết kế ứng dụng chưa tích hợp đầy đủ các nguyên tắc bảo mật.
A05	Security Misconfiguration	Cấu hình bảo mật không phù hợp trên ứng dụng, máy chủ hoặc các thành phần liên quan, làm gia tăng nguy cơ bị khai thác.
A06	Vulnerable and Outdated Components	Việc sử dụng các thư viện, framework hoặc thành phần phần mềm chứa lỗ hổng đã biết hoặc không còn được hỗ trợ cập nhật.
A07	Identification and Authentication Failures	Các vấn đề liên quan đến xác thực danh tính, quản lý tài khoản và quản lý phiên làm việc của người dùng.
A08	Software and Data Integrity Failures	Không đảm bảo tính toàn vẹn của phần mềm và dữ liệu trong quá trình phát triển, cập nhật hoặc triển khai hệ thống.
A09	Security Logging and Monitoring Failures	Thiếu cơ chế ghi nhận, giám sát và phân tích nhật ký bảo mật, gây khó khăn trong việc phát hiện và xử lý sự cố.
A10	Server-Side Request Forgery (SSRF)	Dữ liệu do kẻ tấn công kiểm soát khiến ứng dụng gửi yêu cầu từ phía máy chủ đến đích ngoài dự kiến.

1.2.2. Một số lỗ hổng bảo mật ứng dụng web phổ biến

Trong các nhóm rủi ro được đề cập trong OWASP Top 10, nhiều lỗ hổng bảo mật ứng dụng web xuất hiện thường xuyên trong thực tế và có mức độ ảnh hưởng lớn đến tính bảo mật của hệ thống. Các lỗ hổng này có thể cho phép kẻ tấn công truy cập trái phép vào dữ liệu, chiếm quyền người dùng, thực thi mã độc hoặc khai thác các tài nguyên nội bộ của hệ thống. Việc tìm hiểu các lỗ hổng phổ biến giúp chuyên

gia kiểm thử xây dựng các kịch bản đánh giá phù hợp, đồng thời là cơ sở cho việc phát triển các hệ thống hỗ trợ kiểm thử xâm nhập ứng dụng web.

a) SQL Injection (SQLi)

SQL Injection là một lỗ hổng thuộc nhóm Injection (A03) trong OWASP Top 10:2021, xảy ra khi dữ liệu đầu vào của người dùng không được kiểm tra hoặc xử lý đúng cách trước khi được đưa vào các câu truy vấn cơ sở dữ liệu [3]. Kẻ tấn công có thể lợi dụng lỗ hổng này để chèn các câu lệnh SQL độc hại nhằm truy xuất, sửa đổi hoặc xóa dữ liệu trái phép. Trong những trường hợp nghiêm trọng, SQL Injection có thể dẫn đến việc chiếm quyền kiểm soát hệ quản trị cơ sở dữ liệu hoặc ảnh hưởng đến toàn bộ ứng dụng.

b) Cross-Site Scripting (XSS)

Cross-Site Scripting cũng là một lỗ hổng thuộc nhóm Injection (A03) trong OWASP Top 10:2021, cho phép kẻ tấn công chèn các đoạn mã script độc hại, thường là JavaScript, vào nội dung của trang web [3]. Khi người dùng truy cập vào trang chứa mã độc, trình duyệt sẽ thực thi đoạn mã trong ngữ cảnh của ứng dụng. XSS thường được chia thành ba dạng chính gồm Stored XSS**,** Reflected XSS và DOM-based XSS. Lỗ hổng này có thể bị khai thác để đánh cắp cookie phiên, giả mạo người dùng hoặc thu thập các thông tin nhạy cảm.

c) Insecure Direct Object References (IDOR)

Insecure Direct Object References là một dạng lỗ hổng thuộc nhóm Broken Access Control (A01) trong OWASP Top 10:2021, phát sinh khi ứng dụng cho phép truy cập trực tiếp đến các đối tượng dữ liệu thông qua các tham số do người dùng kiểm soát nhưng không thực hiện kiểm tra quyền truy cập tương ứng [3]. Kẻ tấn công có thể thay đổi các tham số như mã định danh người dùng, số tài liệu hoặc tên tệp để truy cập trái phép vào dữ liệu thuộc về người dùng khác, dẫn đến nguy cơ rò rỉ thông tin hoặc truy cập trái phép vào các tài nguyên quan trọng.

d) Server-Side Request Forgery (SSRF)

Server-Side Request Forgery là một lỗ hổng thuộc nhóm A10 trong OWASP Top 10:2021. Lỗ hổng xảy ra khi dữ liệu do kẻ tấn công kiểm soát khiến ứng dụng gửi yêu cầu từ phía máy chủ đến đích ngoài dự kiến [3]. Tùy theo quyền truy cập và môi trường của máy chủ, kẻ tấn công có thể tiếp cận dịch vụ nội bộ, thu thập thông tin hệ thống hoặc tác động đến các tài nguyên khác.

e) Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery là kỹ thuật tấn công lợi dụng phiên đăng nhập hợp lệ của người dùng để khiến họ vô tình thực hiện các yêu cầu ngoài ý muốn đến ứng

dụng web [3]. Mặc dù không còn được xếp thành một nhóm rủi ro riêng trong OWASP Top 10:2021, CSRF vẫn là một lỗ hổng phổ biến liên quan đến quản lý phiên và xác thực yêu cầu, thường xảy ra khi ứng dụng không triển khai các cơ chế xác minh nguồn gốc của yêu cầu, chẳng hạn như CSRF Token. Nếu khai thác thành công, kẻ tấn công có thể thực hiện các thao tác thay mặt người dùng như thay đổi thông tin tài khoản, cập nhật mật khẩu hoặc thực hiện các giao dịch trái phép.

Các lỗ hổng trên là những vấn đề bảo mật phổ biến trong ứng dụng web và thường được lựa chọn làm đối tượng kiểm thử trong quá trình đánh giá an toàn thông tin. Đồng thời, đây cũng là các lỗ hổng thường được sử dụng trong các nghiên cứu về ứng dụng AI tạo sinh nhằm đánh giá khả năng của mô hình ngôn ngữ lớn trong việc hỗ trợ phân tích, phát hiện và kiểm thử các lỗ hổng bảo mật ứng dụng web.

1.3. Tổng quan về AI tạo sinh và mô hình ngôn ngữ lớn

1.3.1. Khái niệm AI tạo sinh và đặc trưng nổi bật

AI tạo sinh (Generative Artificial Intelligence - Generative AI) là nhánh của trí tuệ nhân tạo có khả năng học đặc điểm của dữ liệu và tạo ra nội dung như văn bản, hình ảnh, âm thanh hoặc mã nguồn. Đối với đề tài này, khả năng xử lý ngôn ngữ, tổng hợp thông tin và sinh nội dung là cơ sở để AI hỗ trợ phân tích dữ liệu kiểm thử, đề xuất hành động và trình bày kết quả.

Một nền tảng quan trọng của AI tạo sinh là các mô hình nền tảng (Foundation Models), được huấn luyện trước trên dữ liệu quy mô lớn và có thể thích ứng với nhiều nhiệm vụ. Mô hình ngôn ngữ lớn (Large Language Models - LLM) là nhóm tiêu biểu, xử lý và sinh ngôn ngữ dựa trên thông tin ngữ cảnh. Các dòng mô hình như GPT, Llama, Claude và Gemini cho phép người dùng định hướng tác vụ bằng chỉ dẫn ngôn ngữ tự nhiên.

LLM có thể thực hiện nhiều nhiệm vụ như trả lời câu hỏi, tóm tắt, phân tích thông tin và sinh mã nguồn thông qua việc thay đổi yêu cầu đầu vào. Khả năng khai thác ngữ cảnh giúp mô hình kết hợp yêu cầu của người dùng với dữ liệu được cung cấp, nhưng tính liên tục của một nhiệm vụ dài còn phụ thuộc vào cách hệ thống quản lý và bổ sung thông tin. Một số mô hình cũng hỗ trợ dữ liệu đa phương thức, mở rộng cách tiếp nhận thông tin ngoài văn bản.

AI tạo sinh vẫn có thể tạo nội dung sai lệch hoặc thiếu căn cứ (hallucination), chịu ảnh hưởng của chất lượng dữ liệu và gặp khó khăn với nhiệm vụ chuyên sâu. Việc sử dụng mô hình còn đặt ra yêu cầu về tài nguyên và chi phí. Vì vậy, các kỹ thuật Prompt Engineering, truy xuất tri thức, Tool Calling và kiến trúc Agent được

kết hợp để cung cấp ngữ cảnh phù hợp, hỗ trợ thực hiện nhiệm vụ và kiểm tra kết quả.

1.3.2. Kiến trúc Transformer và cơ chế hoạt động của LLM

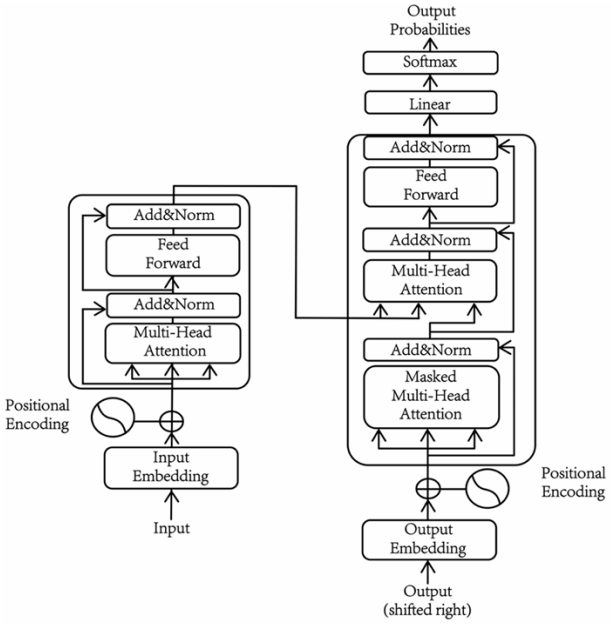
Transformer là kiến trúc mạng nơ-ron nền tảng của nhiều mô hình ngôn ngữ lớn. Trước đó, mạng nơ-ron hồi tiếp (RNN) và mạng nhớ dài - ngắn hạn (LSTM) thường xử lý chuỗi theo trình tự, gặp khó khăn khi học quan hệ giữa các token ở khoảng cách xa và hạn chế khả năng tính toán song song trong huấn luyện.

Transformer sử dụng cơ chế Attention để khai thác quan hệ giữa các phần tử trong chuỗi. Cách tổ chức này hỗ trợ mô hình hóa ngữ cảnh và xử lý song song trong huấn luyện, tạo điều kiện phát triển các mô hình ngôn ngữ trên dữ liệu quy mô lớn. [5]

a) Kiến trúc Transformer

Transformer ban đầu được giới thiệu với kiến trúc Encoder-Decoder. Từ kiến trúc này, nhiều biến thể đã được phát triển, trong đó kiến trúc decoder-only được sử dụng phổ biến trong các mô hình ngôn ngữ sinh văn bản, tiêu biểu như dòng GPT và nhiều mô hình cùng loại.

Transformer bổ sung thông tin về thứ tự token thông qua cơ chế biểu diễn vị trí, chẳng hạn Positional Encoding. Thông tin này giúp mô hình phân biệt vị trí và khai thác quan hệ theo thứ tự trong chuỗi, bên cạnh quan hệ được học bằng Attention.



Hình 1.1. Kiến trúc tổng quát của Transformer [5]

Mỗi tầng Transformer kết hợp Multi-Head Attention và Feed Forward Network để khai thác quan hệ giữa các token và biến đổi biểu diễn dữ liệu. Residual

Connection và Layer Normalization hỗ trợ duy trì quá trình huấn luyện ổn định khi mô hình gồm nhiều tầng.

b) Cơ chế Self-Attention

Self-Attention giúp Transformer xác định mức độ liên quan giữa một token với các token khác trong cùng chuỗi dữ liệu. Qua cơ chế này, mô hình khai thác thông tin từ các vị trí khác nhau để biểu diễn ngữ cảnh của token đang xét, kể cả khi các token có liên quan nằm cách xa nhau trong chuỗi.

Để nâng cao khả năng biểu diễn ngữ cảnh, Transformer sử dụng Multi-Head Attention với nhiều đầu Attention hoạt động song song. Mỗi đầu có thể học các mối quan hệ khác nhau trong dữ liệu, chẳng hạn như quan hệ ngữ pháp, ngữ nghĩa hoặc sự liên kết giữa các thành phần trong văn bản. Các kết quả sau đó được kết hợp để tạo thành biểu diễn giàu thông tin hơn cho chuỗi đầu vào.

Các cơ chế Attention giúp Transformer khai thác quan hệ trong chuỗi mà không phải truyền thông tin lần lượt qua từng bước như RNN. Khả năng xử lý song song này chủ yếu được tận dụng trong huấn luyện, còn mô hình sinh văn bản tự hồi quy vẫn tạo các token đầu ra theo trình tự.

c) Cơ chế hoạt động của mô hình ngôn ngữ lớn

LLM được huấn luyện trước trên khối lượng lớn dữ liệu văn bản và mã nguồn để học các quy luật ngôn ngữ. Các mô hình sinh văn bản tự hồi quy được huấn luyện để dự đoán token tiếp theo dựa trên các token đứng trước, qua đó học cách tạo nội dung theo ngữ cảnh.

Khi người dùng cung cấp một Prompt, dữ liệu đầu vào trước tiên được chia thành các đơn vị nhỏ gọi là token thông qua quá trình Tokenization. Các token sau đó được biểu diễn dưới dạng các vector số và đưa vào mô hình Transformer để phân tích ngữ cảnh thông qua các tầng xử lý và cơ chế Self-Attention.

Đối với các mô hình ngôn ngữ tự hồi quy (autoregressive), trong quá trình suy luận, mô hình sử dụng các tham số đã được học cùng với ngữ cảnh đầu vào để dự đoán token tiếp theo. Token mới được tạo ra tiếp tục được đưa vào ngữ cảnh để mô hình dự đoán token tiếp theo, và quá trình này được lặp lại cho đến khi đạt điều kiện kết thúc. Chuỗi token được sinh ra sau đó được chuyển đổi thành văn bản đầu ra để cung cấp cho người dùng.

Cùng một mô hình có thể hỗ trợ trả lời câu hỏi, tóm tắt, sinh mã nguồn hoặc phân tích thông tin thông qua những yêu cầu khác nhau. Chất lượng đầu ra phụ thuộc vào ngữ cảnh, cách xây dựng Prompt và năng lực của mô hình, nên việc tổ chức đầu vào là yêu cầu quan trọng khi ứng dụng LLM vào kiểm thử xâm nhập.

1.3.3. Prompt Engineering và Chain-of-Thought

Chất lượng đầu ra của LLM phụ thuộc không chỉ vào mô hình mà còn vào cách tổ chức yêu cầu và dữ liệu đầu vào. Prompt Engineering giúp xác định nhiệm vụ, phạm vi, ràng buộc và định dạng kết quả, trong khi Chain-of-Thought (CoT) hỗ trợ xử lý những nhiệm vụ cần suy luận qua nhiều bước.

a) Prompt Engineering

Prompt gồm các chỉ dẫn, thông tin ngữ cảnh và dữ liệu được cung cấp cho LLM để định hướng quá trình xử lý. Prompt Engineering là việc thiết kế và điều chỉnh các nội dung này nhằm tạo đầu ra phù hợp với tác vụ mà không thay đổi trọng số mô hình như khi tinh chỉnh.

Một Prompt rõ ràng giúp mô hình nhận biết mục tiêu, giới hạn và yêu cầu đầu ra, đồng thời giảm nguy cơ hiểu sai yêu cầu. Trong hệ thống Agent, Prompt còn cần phân biệt chỉ dẫn thực hiện nhiệm vụ với dữ liệu quan sát để kết quả công cụ được sử dụng đúng vai trò.

Thông thường, một Prompt có thể bao gồm các thành phần chính:

- Instruction: Mô tả nhiệm vụ mà mô hình cần thực hiện, yêu cầu về nội dung hoặc định dạng kết quả đầu ra.
- Examples: Cung cấp các cặp đầu vào - đầu ra mẫu nhằm minh họa cho mô hình cách thực hiện nhiệm vụ.
- Input: Dữ liệu hoặc câu hỏi thực tế mà mô hình cần xử lý.

Thông tin ngữ cảnh có thể được bổ sung để làm rõ mục tiêu, dữ liệu nền và các ràng buộc. Chẳng hạn, một yêu cầu phân tích phản hồi HTTP cần nêu phương thức, điểm đầu vào đang xem xét và yêu cầu chỉ đưa ra nhận định dựa trên bằng chứng được cung cấp.

Theo số lượng ví dụ được cung cấp, hai cách xây dựng Prompt thường gặp là Zero-shot và Few-shot Prompting.

Zero-shot Prompting cung cấp yêu cầu và dữ liệu trực tiếp, không kèm ví dụ mẫu. Cách này phù hợp khi nhiệm vụ và định dạng đầu ra được mô tả rõ. Ví dụ, người dùng có thể yêu cầu mô hình phân tích phản hồi HTTP, nêu dấu hiệu nghi vấn SQL Injection và đề xuất bước kiểm tra tiếp theo mà chưa kết luận có lỗ hổng.

Few-shot Prompting bổ sung một số cặp đầu vào - đầu ra mẫu trước yêu cầu chính để minh họa cách xử lý và trình bày kết quả. Với cùng tác vụ phân tích HTTP, các ví dụ có thể thể hiện cách phân biệt dấu hiệu lỗi thông thường với dấu hiệu cần kiểm chứng. Mô hình vận dụng các mẫu này trong ngữ cảnh, nhưng kết quả vẫn cần được đánh giá trên dữ liệu thực tế.

Role Prompting định hướng mô hình theo một vai trò, chẳng hạn trợ lý kiểm thử bảo mật, để phù hợp với phạm vi phân tích và cách diễn đạt. Structured Prompting tổ chức yêu cầu và đầu ra theo cấu trúc rõ ràng như Markdown hoặc JSON, hỗ trợ hệ thống tiếp nhận và sử dụng kết quả trong các bước tiếp theo.

Trong kiểm thử xâm nhập, Prompt Engineering có thể hỗ trợ phân tích kết quả công cụ, giải thích dấu hiệu lỗ hổng, đề xuất bước kiểm tra và tổng hợp báo cáo. Yêu cầu nên nêu rõ phạm vi mục tiêu, dữ liệu hiện có và tiêu chí kết luận. Cách tổ chức này giúp hạn chế phản hồi không liên quan, nhưng không bảo đảm loại bỏ hoàn toàn hallucination hoặc thay thế việc kiểm chứng bằng chứng.

b) Chain-of-Thought (CoT)

CoT khuyến khích mô hình xử lý nhiệm vụ thông qua các bước suy luận trung gian trước khi đưa ra kết quả. Kỹ thuật này có thể hỗ trợ phân tích thông tin, lập kế hoạch và giải quyết các bài toán có nhiều quan hệ phụ thuộc, thay vì yêu cầu mô hình trả lời ngay từ dữ liệu ban đầu. [6]

Zero-shot CoT sử dụng chỉ dẫn yêu cầu giải quyết vấn đề theo từng bước mà không kèm ví dụ mẫu [7]. Few-shot CoT cung cấp các ví dụ có phần suy luận trung gian và kết quả để định hướng cách xử lý những nhiệm vụ tương tự [6]. Hiệu quả của hai cách tiếp cận phụ thuộc vào mô hình, chất lượng chỉ dẫn và đặc điểm tác vụ.

CoT có thể cải thiện cách tổ chức phân tích, nhưng các bước suy luận và kết luận vẫn có thể sai. Việc tạo thêm nội dung còn làm tăng số token, thời gian phản hồi và chi phí xử lý. Do đó, trong kiểm thử xâm nhập, cần đối chiếu nhận định với dữ liệu thực nghiệm thay vì coi lời giải thích của mô hình là bằng chứng.

CoT chủ yếu sử dụng thông tin đã có trong ngữ cảnh và không tự cung cấp khả năng thực thi công cụ. Để thu thập thông tin mới và điều chỉnh hướng kiểm tra dựa trên phản hồi từ môi trường, hệ thống cần kết hợp suy luận với cơ chế hành động, như cách tiếp cận ReAct được trình bày tiếp theo.

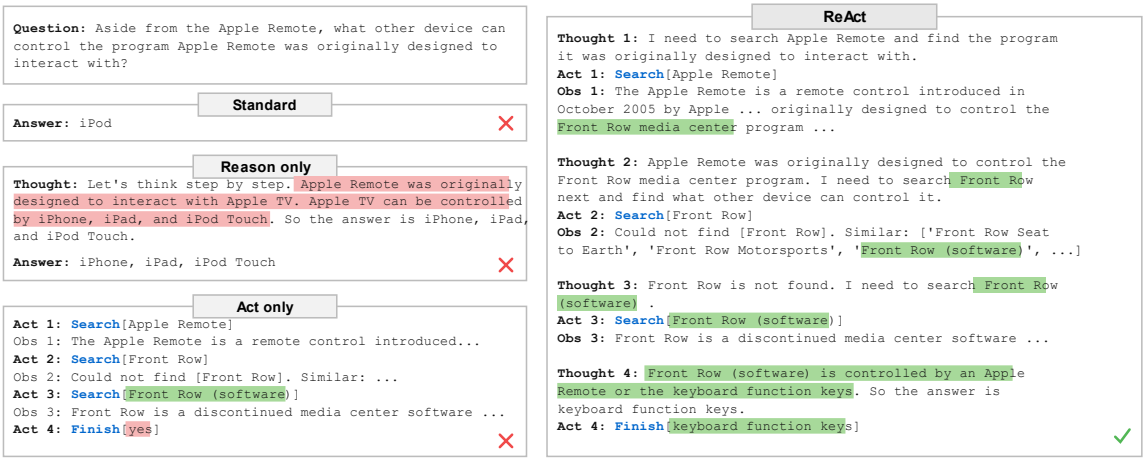
1.3.4. Cơ chế ReAct trong mô hình ngôn ngữ lớn

ReAct (Reasoning and Acting) kết hợp suy luận của mô hình ngôn ngữ với hành động trong một chu trình xử lý. Khi nhiệm vụ cần thông tin ngoài ngữ cảnh hoặc thao tác với môi trường, mô hình có thể đề xuất sử dụng công cụ, tiếp nhận kết quả và dựa vào đó xác định bước tiếp theo. [8]

Chu trình ReAct gồm Thought, Action và Observation. Thought biểu diễn việc phân tích thông tin hiện có và lựa chọn hướng xử lý. Ở bước Action, mô hình đề xuất hành động như gọi API, tìm kiếm hoặc sử dụng công cụ, còn hệ thống tích

hợp đảm nhiệm việc thực thi. Observation là kết quả trả về, được bổ sung vào ngữ cảnh để phục vụ lần suy luận tiếp theo.

Chu trình được lặp lại cho đến khi nhiệm vụ có thể kết thúc hoặc đạt điều kiện dừng. Phản hồi từ môi trường giúp mô hình điều chỉnh hành động thay vì chỉ dựa vào kế hoạch ban đầu. Hình 1.2 minh họa sự khác nhau giữa các phương pháp Prompting và cơ chế ReAct.



Hình 1.2. So sánh các phương pháp Prompting và cơ chế ReAct [8]

So với CoT chủ yếu phân tích ngữ cảnh sẵn có, ReAct bổ sung khả năng thu thập thông tin qua hành động. Trong kiểm thử web, hệ thống có thể gửi một yêu cầu HTTP, phân tích phản hồi, lựa chọn đầu vào cần kiểm tra rồi tiếp tục thu thập bằng chứng. Mỗi kết quả quan sát tạo cơ sở cho bước tiếp theo, nhưng không tự chứng minh kết luận của mô hình là đúng.

Hiệu quả của ReAct phụ thuộc vào năng lực lựa chọn hành động, độ tin cậy của công cụ và cách duy trì ngữ cảnh. Hành động không phù hợp hoặc phản hồi bị diễn giải sai có thể dẫn tới kiểm tra lặp hay kết luận thiếu căn cứ. Nhiều vòng tương tác cũng làm tăng chi phí và thời gian, nên hệ thống cần giới hạn thực thi và kiểm soát phạm vi.

Nguyên lý suy luận - hành động - quan sát của ReAct là cơ sở để xây dựng Agent thực hiện nhiệm vụ kiểm thử nhiều bước. Khi kết hợp với quản lý trạng thái và cơ chế gọi công cụ, hệ thống có thể duy trì tiến trình, điều chỉnh hướng kiểm tra và tổng hợp kết quả dựa trên các dữ liệu đã thu được.

1.3.5. AI Agent dựa trên LLM và cơ chế Tool Calling

a) Khái niệm AI Agent dựa trên mô hình ngôn ngữ lớn

LLM có thể phân tích và sinh nội dung, nhưng việc sử dụng mô hình như một giao diện hội thoại chưa đủ để thực hiện nhiệm vụ kéo dài có nhiều bước và tương

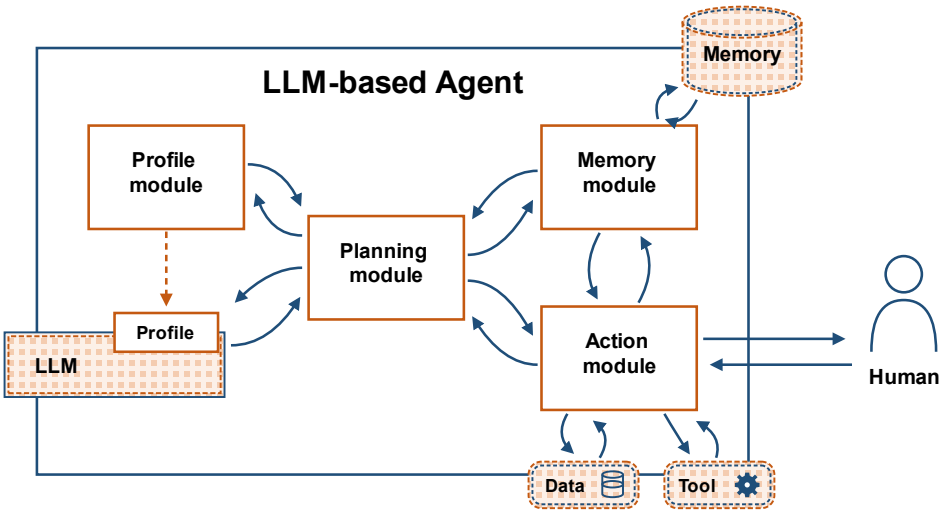
tác với môi trường. Hệ thống cần bổ sung cơ chế duy trì trạng thái, lập kế hoạch và thực thi hành động.

AI Agent dựa trên mô hình ngôn ngữ lớn (LLM-based Agent) kết hợp LLM với các thành phần hỗ trợ để tiếp nhận mục tiêu, lựa chọn bước xử lý và sử dụng công cụ. LLM cung cấp khả năng phân tích và đề xuất, còn hệ thống điều phối tổ chức vòng xử lý, quản lý dữ liệu và kiểm soát thực thi.

Thay vì chỉ sử dụng LLM để tạo phản hồi, AI Agent có thể tổ chức quá trình thực hiện nhiệm vụ thành chu trình nhiều bước. Agent có thể tiếp nhận mục tiêu ban đầu, xây dựng kế hoạch thực hiện, sử dụng các công cụ bên ngoài để thu thập thông tin, đánh giá kết quả nhận được và điều chỉnh hành động tiếp theo dựa trên trạng thái hiện tại.

b) Kiến trúc tổng quát của LLM-based Agent

Kiến trúc tổng quát của LLM-based Agent có thể được mô tả thông qua bốn nhóm chức năng gồm Profile, Memory, Planning và Action như Hình 1.3 [9]. Các thành phần này phối hợp để xác định vai trò của Agent, duy trì thông tin, xây dựng kế hoạch và thực hiện hành động trong quá trình xử lý nhiệm vụ.



Hình 1.3. Kiến trúc tổng quát của hệ thống LLM-based Agent [10]

Profile: Thành phần này xác định vai trò, mục tiêu và phạm vi nhiệm vụ của Agent, qua đó định hướng quá trình phân tích và lựa chọn hành động. Trong kiểm thử xâm nhập, thành phần này có thể mô tả vai trò trợ lý bảo mật, đối tượng cần đánh giá và yêu cầu sử dụng bằng chứng khi đưa ra kết luận. Các giới hạn thực thi được kiểm soát độc lập bởi hệ thống.

Memory: Thành phần này lưu trữ và cung cấp thông tin để Agent sử dụng kết quả đã thu thập, đồng thời duy trì tính liên tục giữa các bước xử lý. Bộ nhớ ngắn hạn phục vụ nhiệm vụ đang thực hiện, còn bộ nhớ dài hạn lưu những thông tin có thể tái sử dụng trong các nhiệm vụ sau. Đối với kiểm thử xâm nhập, thông tin được lưu có thể gồm kết quả khảo sát, các đầu vào đã phát hiện và tiến độ kiểm tra. Khi đưa vào ngữ cảnh của mô hình, dữ liệu cần được lựa chọn phù hợp để hạn chế vượt giới hạn ngữ cảnh hoặc nhầm lẫn thông tin giữa các mục tiêu.

Planning: Thành phần này hỗ trợ phân tích mục tiêu, phân rã nhiệm vụ và lựa chọn trình tự xử lý. Trong kiểm thử ứng dụng web, kế hoạch có thể bao gồm khảo sát mục tiêu, phân tích đầu vào và xác minh các dấu hiệu lỗ hổng. Kế hoạch được điều chỉnh theo phản hồi từ ứng dụng và tiến độ công việc để phù hợp với thông tin thu được trong quá trình thực hiện.

Action: Thành phần này cho phép Agent thực hiện các thao tác như truy vấn dữ liệu, gửi yêu cầu HTTP hoặc gọi công cụ chuyên dụng. Các yêu cầu hành động được chuyển đến thành phần thực thi và kiểm tra theo phạm vi, quyền cùng các giới hạn của hệ thống. Kết quả thực hiện được đưa trở lại Agent để bổ sung thông tin và xác định bước xử lý tiếp theo.

Sự phối hợp giữa bốn thành phần tạo thành quá trình xử lý nhiệm vụ nhiều bước. Mức độ tự chủ của Agent phụ thuộc vào cách tổ chức các thành phần, khả năng sử dụng công cụ và yêu cầu giám sát trong từng hệ thống.

c) Cơ chế Tool Calling trong mô hình ngôn ngữ lớn

Tool Calling, còn được gọi là Function Calling trong một số nền tảng, cho phép mô hình ngôn ngữ lớn yêu cầu sử dụng công cụ hoặc dịch vụ bên ngoài. Thông qua cơ chế này, Agent có thể thu thập dữ liệu và thực hiện các thao tác phục vụ nhiệm vụ, sau đó sử dụng kết quả để tiếp tục phân tích.

Quá trình bắt đầu khi LLM xem xét yêu cầu và thông tin hiện có để xác định có cần sử dụng công cụ hay không. Nếu cần, mô hình lựa chọn công cụ và tạo yêu cầu gọi gồm tên công cụ cùng các tham số đầu vào. LLM chỉ tạo yêu cầu, còn việc thực thi do thành phần trung gian của hệ thống đảm nhiệm.

Thành phần trung gian tiếp nhận yêu cầu, kiểm tra các điều kiện thực thi và gọi công cụ tương ứng. Kết quả được đưa trở lại mô hình như thông tin bổ sung để đánh giá tiến độ và xác định bước tiếp theo. Tùy theo kết quả thu được, mô hình có thể yêu cầu sử dụng thêm công cụ hoặc tổng hợp câu trả lời. Quá trình này có thể lặp lại nhiều lần trong cùng một nhiệm vụ.

Chẳng hạn, trong hệ thống hỗ trợ kiểm thử xâm nhập, Agent có thể yêu cầu sử dụng Nmap để khảo sát các cổng và dịch vụ trên mục tiêu. Thành phần thực thi của hệ thống tiến hành thao tác trong phạm vi được phép rồi chuyển kết quả về cho mô hình. Dựa trên các dịch vụ được ghi nhận, Agent tiếp tục lựa chọn nội dung cần kiểm tra.

d) Mối quan hệ giữa ReAct, Tool Calling và ứng dụng

ReAct mô tả cách tổ chức chu trình suy luận, hành động và quan sát, còn Tool Calling cung cấp cơ chế để mô hình yêu cầu sử dụng công cụ và tiếp nhận kết quả. Hai cơ chế có thể được kết hợp khi các hành động trong chu trình ReAct được thực hiện thông qua Tool Calling. Kết quả từ công cụ trở thành dữ liệu quan sát để mô hình đánh giá và điều chỉnh hướng xử lý. Tuy nhiên, yêu cầu gọi công cụ do mô hình tạo ra vẫn phải được hệ thống kiểm tra trước khi thực thi.

Trong kiểm thử xâm nhập, sự kết hợp này giúp Agent lựa chọn phép thử, thu thập phản hồi và điều chỉnh các bước kiểm tra theo thông tin thực tế. Các thành phần quản lý trạng thái duy trì tiến độ và kết quả đã thu được, trong khi cơ chế kiểm soát thực thi giới hạn hành động theo phạm vi và quyền được cấp. Nhờ đó, hệ thống có thể hỗ trợ tự động hóa một phần quy trình kiểm thử, đồng thời cho phép người kiểm thử theo dõi và can thiệp khi cần.

1.4. Ứng dụng AI tạo sinh trong kiểm thử xâm nhập

1.4.1. Hạn chế của kiểm thử truyền thống và vai trò của AI tạo sinh

a) Hạn chế của kiểm thử xâm nhập truyền thống

Dù đã có nhiều công cụ tự động, kiểm thử xâm nhập vẫn cần chuyên gia phân tích thông tin, lựa chọn phép thử và đánh giá kết quả. Một số hạn chế tiêu biểu của cách tiếp cận truyền thống gồm:

Phụ thuộc vào năng lực người kiểm thử: Việc nhận diện điểm yếu và lựa chọn phép thử đòi hỏi hiểu biết về cấu trúc, chức năng và cơ chế bảo mật của mục tiêu. Khả năng kết hợp thông tin để hình thành giả thuyết ảnh hưởng đến phạm vi và chất lượng kiểm thử của từng chuyên gia.

Tốn thời gian và nguồn lực: Quá trình khảo sát, phân tích và xác minh gồm nhiều bước. Người kiểm thử vẫn phải lựa chọn phương pháp và xử lý tình huống phát sinh. Với hệ thống có nhiều thành phần và điểm đầu vào, việc đánh giá đầy đủ gặp khó khăn khi nguồn lực bị giới hạn.

Khó đối chiếu kết quả từ nhiều công cụ: Các công cụ chuyên biệt cung cấp dữ liệu theo những định dạng khác nhau. Người kiểm thử cần liên kết các kết quả và

thực hiện phép kiểm tra bổ sung phù hợp với ngữ cảnh. Công việc này phức tạp hơn khi đánh giá lỗi logic nghiệp vụ hoặc xác minh qua nhiều bước phụ thuộc.

Khó theo kịp thay đổi của hệ thống: Việc cập nhật chức năng, giao diện và thành phần ứng dụng có thể tạo ra điểm yếu mới. Ở những hệ thống lớn hoặc cập nhật liên tục, hoạt động đánh giá khó theo kịp vì kiểm thử chuyên sâu cần thời gian phân tích và xác minh.

Tổng hợp và lập báo cáo thủ công: Phát hiện, bằng chứng và lịch sử thao tác thường nằm ở nhiều nguồn. Người kiểm thử phải tổng hợp và đối chiếu các nguồn này để mô tả lỗ hổng, đánh giá tác động và đề xuất biện pháp khắc phục. Công việc này mất thời gian và đòi hỏi sự nhất quán giữa các nội dung trong báo cáo.

b) Vai trò của AI tạo sinh trong kiểm thử xâm nhập

AI tạo sinh có thể hỗ trợ nhiều công đoạn trong kiểm thử xâm nhập, tập trung vào bốn nhóm hoạt động tiêu biểu sau:

Xử lý và phân tích thông tin: LLM có thể liên kết dữ liệu từ tài liệu kỹ thuật, kết quả quét, cấu hình dịch vụ, thông báo lỗi và phản hồi HTTP để nhận diện những điểm cần kiểm tra. Chẳng hạn, việc đối chiếu mô tả API với phản hồi thực tế giúp người kiểm thử xem xét các tham số, chức năng hoặc điều kiện xử lý trong cùng ngữ cảnh.

Lập kế hoạch và định hướng kiểm thử: Dựa trên mục tiêu, phạm vi và dữ liệu hiện có, LLM có thể đề xuất giả thuyết, xác định vấn đề cần ưu tiên và gợi ý phép thử. Mô hình cũng có thể chỉ ra thông tin còn thiếu, điều kiện thực hiện và kết quả dự kiến của phép thử. Khi nhận phản hồi mới, mô hình có thể điều chỉnh kế hoạch hoặc đề xuất kiểm tra bổ sung cho nhiệm vụ nhiều bước.

Hỗ trợ thực hiện kiểm thử thông qua công cụ: Khi được tích hợp vào hệ thống có khả năng sử dụng công cụ, LLM có thể đề xuất lời gọi để thu thập thông tin, gửi yêu cầu kiểm tra và tiếp nhận phản hồi. Các thao tác được thực hiện bởi thành phần tích hợp theo phạm vi và quyền đã thiết lập. Kết quả thu được tiếp tục được sử dụng để phân tích và lựa chọn bước kiểm thử tiếp theo.

Đánh giá kết quả và hỗ trợ lập báo cáo: LLM có thể đối chiếu đầu vào, thao tác và phản hồi để phân tích khác biệt, xác định dấu hiệu cần kiểm chứng. Từ kết quả và bằng chứng đã ghi nhận, mô hình hỗ trợ tổng hợp phát hiện, diễn giải tác động và đề xuất khắc phục. Sự hỗ trợ này giúp giảm công việc tổng hợp thủ công.

1.4.2. Các hướng ứng dụng AI tạo sinh trong kiểm thử xâm nhập

Hai hướng ứng dụng tiêu biểu là AI hỗ trợ chuyên gia và hệ thống AI Agent. Chúng khác nhau chủ yếu ở vai trò của người kiểm thử, khả năng thực hiện hành động và cách tổ chức tiến trình.

a) Kiểm thử xâm nhập có AI hỗ trợ

Đặc điểm cách tiếp cận: LLM phân tích dữ liệu về mục tiêu, giải thích kết quả công cụ và đề xuất phương pháp qua giao diện hội thoại hoặc các công cụ tích hợp. Mô hình có thể gợi ý phép thử, xây dựng câu lệnh và xác định thông tin cần thu thập thêm. Một số hệ thống còn ghi nhận công việc đã thực hiện để tổ chức bước tiếp theo. Người kiểm thử cung cấp dữ liệu và phản hồi, đồng thời trực tiếp lựa chọn phương pháp, thực hiện thao tác và xác minh kết quả.

Hệ thống PentestGPT: Được giới thiệu tại USENIX Security 2024, hệ thống gồm ba mô-đun chính. Mô-đun lập luận quản lý tiến trình bằng cây tác vụ, lưu mục tiêu, trạng thái công việc và định hướng kiểm tra tiếp theo. Mô-đun sinh bước thực hiện chuyển các tác vụ thành hướng dẫn hoặc thao tác cụ thể. Mô-đun phân tích kết quả trích xuất thông tin quan trọng từ đầu ra công cụ và dữ liệu liên quan để cập nhật nhận định. Trong mô hình tương tác của nghiên cứu, người kiểm thử thực hiện thao tác được đề xuất rồi đưa kết quả trở lại hệ thống để tiếp tục phân tích [11].

Nhận xét: Hướng AI hỗ trợ dễ bổ sung vào quy trình hiện có vì không đòi hỏi hệ thống tự thực thi toàn bộ thao tác. Tuy nhiên, sự tham gia thường xuyên của người kiểm thử vẫn cần thiết. Khối lượng công việc thủ công và chất lượng xác minh phụ thuộc vào năng lực của họ, độ tin cậy của đề xuất và mức độ tích hợp của giải pháp.

b) Kiểm thử xâm nhập sử dụng AI Agent

Đặc điểm cách tiếp cận: LLM phân tích mục tiêu và ngữ cảnh để đề xuất hành động, còn thành phần điều phối tổ chức và chuyển lời gọi đến công cụ thực thi. Kết quả được đưa trở lại mô hình và cập nhật vào ngữ cảnh cùng thông tin mục tiêu, trạng thái công việc và lịch sử kiểm thử. Từ phản hồi này, Agent đánh giá tiến độ, điều chỉnh kế hoạch và chọn phép thử tiếp theo để xử lý nhiệm vụ có quan hệ phụ thuộc qua nhiều bước.

Kiến trúc Single-Agent: Một tác tử chính đảm nhiệm phân tích, lập kế hoạch và điều phối hành động, đồng thời sử dụng các công cụ, nguồn tri thức và thành phần hỗ trợ. Cách tổ chức tập trung thuận lợi cho việc theo dõi tiến trình. Khi nhiệm vụ phức tạp hoặc phạm vi mở rộng, tác tử phải xử lý nhiều loại thông tin và công việc hơn, làm tăng yêu cầu về quản lý ngữ cảnh và lựa chọn hành động.

Kiến trúc Multi-Agent: Các tác tử được phân công những vai trò như thu thập thông tin, phân tích lỗ hổng, lập kế hoạch hoặc thực hiện kiểm tra. Chúng phối hợp thông qua trao đổi kết quả, chuyển giao nhiệm vụ hoặc sử dụng trạng thái và bộ nhớ chia sẻ. Cách tổ chức này hỗ trợ chuyên môn hóa, nhưng cần thêm cơ chế điều phối và đồng bộ thông tin để duy trì tính nhất quán.

Hệ thống PentestAgent: Được công bố tại AsiaCCS 2025, hệ thống sử dụng kiến trúc Multi-Agent để phối hợp các giai đoạn thu thập thông tin, phân tích lỗ hổng và khai thác. Tác tử khảo sát thu thập thông tin về môi trường mục tiêu. Tác tử tìm kiếm truy xuất thông tin về bề mặt tấn công và phương pháp liên quan. Tác tử lập kế hoạch sử dụng kết quả khảo sát và tri thức truy xuất để lựa chọn phương án kiểm tra. Tác tử thực thi tiến hành kiểm tra, thử nghiệm phương án khai thác và xử lý phản hồi. Cơ chế Retrieval-Augmented Generation (RAG) bổ sung tri thức từ tài liệu công cụ và cơ sở tri thức phục vụ các bước khảo sát, lập kế hoạch. Kết quả của từng giai đoạn được lưu và sử dụng cho hoạt động tiếp theo [12].

Mức độ tự chủ của Agent còn phụ thuộc vào công cụ tích hợp, quản lý trạng thái, quyền thực thi và sự giám sát của con người.

c) So sánh và đánh giá các hướng tiếp cận

Bảng 1.3 đối chiếu hai hướng ứng dụng theo vai trò của AI và người kiểm thử, khả năng tự động hóa, cách tổ chức tiến trình và yêu cầu triển khai.

Bảng 1.3. So sánh các hướng ứng dụng AI trong kiểm thử xâm nhập

Tiêu chí	AI hỗ trợ chuyên gia	Hệ thống AI Agent
Vai trò của AI	Phân tích thông tin và đề xuất hoạt động kiểm thử	Tham gia lập kế hoạch, điều phối và lựa chọn hành động
Mức độ tự động hóa	Chủ yếu hỗ trợ quyết định, người kiểm thử trực tiếp thực hiện	Có thể tự động thực hiện các tác vụ thông qua công cụ tích hợp
Khả năng lập kế hoạch	Đề xuất hướng kiểm tra để người kiểm thử xem xét	Có thể duy trì và điều chỉnh kế hoạch theo kết quả thực hiện
Sử dụng công cụ	Người kiểm thử thực hiện thao tác hoặc cung cấp kết quả cho AI	Hệ thống tiếp nhận và thực thi lời gọi công cụ theo cơ chế kiểm soát được thiết lập
Quản lý ngữ cảnh và trạng thái	Thông qua hội thoại hoặc cơ chế hỗ trợ của hệ thống	Duy trì thông tin phục vụ điều phối và xử lý nhiệm vụ nhiều bước

Tiêu chí	AI hỗ trợ chuyên gia	Hệ thống AI Agent
Vai trò của con người	Trực tiếp lựa chọn, thực hiện và xác minh các bước kiểm thử	Thiết lập mục tiêu, phạm vi, giám sát và phê duyệt khi cần thiết
Yêu cầu triển khai	Phụ thuộc mức độ tích hợp vào quy trình kiểm thử	Cần cơ chế điều phối, tích hợp công cụ, quản lý trạng thái và kiểm soát thực thi

AI hỗ trợ chuyên gia phù hợp với quy trình cần sự tham gia thường xuyên của người kiểm thử, nhưng vẫn đòi hỏi nhiều thao tác thủ công. AI Agent có thể giảm công việc thực hiện trực tiếp nhờ tích hợp công cụ, đồng thời đặt ra yêu cầu quản lý ngữ cảnh và trạng thái, kiểm soát hành động và xác minh kết quả. Mức độ tự động hóa vì vậy cần được xem xét cùng trách nhiệm của con người và điều kiện vận hành.

Single-Agent thuận lợi cho điều phối tập trung, còn Multi-Agent hỗ trợ chuyên môn hóa nhưng cần thêm cơ chế trao đổi và đồng bộ thông tin. Lựa chọn kiến trúc phải căn cứ vào nhiệm vụ, phạm vi kiểm thử và nguồn lực triển khai, thay vì dựa vào số lượng tác tử. Những đánh đổi này là cơ sở để lựa chọn thiết kế KAgent ở Chương 2. Hiệu quả của hệ thống cần được đánh giá qua chất lượng phân tích, khả năng xác minh và mức kiểm soát trong điều kiện thực nghiệm cụ thể.

1.4.3. Thách thức và xu hướng phát triển

a) Thách thức

Việc triển khai AI Pentest Agent đặt ra những vấn đề ảnh hưởng trực tiếp đến độ tin cậy và điều kiện vận hành:

Độ tin cậy của kết quả: Hiện tượng hallucination có thể khiến LLM đề xuất phép thử sai, diễn giải nhầm phản hồi hoặc kết luận thiếu căn cứ. Bằng chứng thiếu hoặc sai ngữ cảnh còn gây cảnh báo sai và bỏ sót lỗ hổng. Do đó, các nhận định phải được đối chiếu với kết quả kiểm tra thực tế.

Suy luận và lập kế hoạch nhiều bước: Các phép thử có quan hệ phụ thuộc và phải được điều chỉnh theo phản hồi của môi trường. Khi kết quả khác dự kiến hoặc dữ liệu chưa đầy đủ, Agent khó quyết định nên bổ sung thông tin, chuyển hướng, tiếp tục kiểm tra hay kết thúc nhiệm vụ.

Quản lý ngữ cảnh và bộ nhớ: Giới hạn cửa sổ ngữ cảnh có thể khiến thông tin về mục tiêu, lịch sử thao tác, kết quả và trạng thái nhiệm vụ bị lược bỏ hoặc không được sử dụng đúng lúc. Điều này ảnh hưởng đến tính nhất quán của quyết

định. Với Multi-Agent, hệ thống còn phải trao đổi thông tin và đồng bộ trạng thái giữa các tác tử.

Tích hợp và điều phối công cụ: Các công cụ khác nhau về chức năng, tham số và định dạng đầu ra. Hệ thống phải kiểm tra lời gọi, xử lý lỗi và cung cấp phản hồi rõ ràng cho Agent. Lời gọi không phù hợp, kết quả thiếu hoặc sai lệch, hay việc diễn giải nhầm phản hồi có thể khiến Agent chọn bước tiếp theo không phù hợp với trạng thái thực tế.

Kiểm soát an toàn và phạm vi thực thi: Các hành động phải tuân thủ phạm vi và quyền được cấp để hạn chế ảnh hưởng ngoài dự kiến. Website, phản hồi HTTP và dữ liệu công cụ có thể chứa Prompt Injection. Vì vậy, hệ thống cần phân biệt dữ liệu bên ngoài với chỉ dẫn hợp lệ, kiểm soát quyền và hỗ trợ người kiểm thử giám sát, phê duyệt hành động.

Chi phí và tài nguyên vận hành: Nhiệm vụ nhiều bước làm tăng lượt gọi LLM, chi phí API hoặc yêu cầu phần cứng cho mô hình cục bộ. RAG, Memory và Multi-Agent còn cần tài nguyên lưu trữ, truy hồi và trao đổi dữ liệu. Thiết kế phải cân đối chất lượng xử lý, thời gian và nguồn lực.

Đánh giá hiệu quả hệ thống: Sự khác nhau giữa ứng dụng đào tạo, máy ảo và bài toán CTF gây khó khăn cho việc so sánh kết quả. Mô hình, công cụ, đầu vào và mức hỗ trợ của con người cũng ảnh hưởng đến thực nghiệm. Do đó, việc đánh giá cần xét khả năng hoàn thành nhiệm vụ, độ tin cậy, chất lượng bằng chứng, thời gian và chi phí.

b) Xu hướng phát triển

Các hướng nghiên cứu tập trung vào bổ sung tri thức, duy trì tiến trình và mở rộng khả năng tự chủ có kiểm soát:

Tích hợp tri thức bảo mật bằng RAG: Cơ chế truy xuất bổ sung thông tin cho LLM khi thực hiện nhiệm vụ [13]. Các nguồn như CVE, CWE, CAPEC, MITRE ATT&CK, NVD và ExploitDB cung cấp dữ liệu về điểm yếu, kỹ thuật tấn công và phương pháp kiểm tra, giúp giảm phụ thuộc vào kiến thức sẵn có của mô hình. Chất lượng truy xuất và độ tin cậy của tài liệu vẫn cần được kiểm soát.

Cải thiện quản lý bộ nhớ và ngữ cảnh: Việc tóm lược, lựa chọn và truy hồi dữ liệu giúp giữ lại những kết quả quan trọng và thông tin về tiến độ để phục vụ các bước tiếp theo trong nhiệm vụ kéo dài. Với Multi-Agent, bộ nhớ chia sẻ và cơ chế đồng bộ trạng thái cần bảo đảm thông tin nhất quán giữa các tác tử.

Nâng cao khả năng tự chủ và xác minh: Cải tiến lập kế hoạch, lựa chọn công cụ và điều chỉnh theo phản hồi cần đi cùng kiểm chứng nhận định, đối chiếu

kết quả và lưu bằng chứng. Việc mở rộng tự động hóa phải duy trì kiểm soát phạm vi, quyền thực thi và giám sát của người kiểm thử.

Tích hợp kiểm thử vào DevSecOps và CI/CD: Khi phần mềm được cập nhật thường xuyên, các tác vụ kiểm thử có thể được kích hoạt ở những giai đoạn phù hợp. AI có tiềm năng hỗ trợ phân tích kết quả, lựa chọn khu vực cần đánh giá và giảm công việc thủ công. Việc tích hợp cần bảo đảm độ tin cậy của kết quả và hạn chế ảnh hưởng đến quá trình phát triển, triển khai.

1.5. Kết luận chương 1

Chương 1 đã trình bày cơ sở lý thuyết và tổng quan nghiên cứu liên quan đến đề tài ứng dụng AI tạo sinh trong kiểm thử xâm nhập. Trước hết, chương đã hệ thống hóa các khái niệm, quy trình và phương pháp kiểm thử xâm nhập, đồng thời phân tích các khung tiêu chuẩn và những lỗ hổng bảo mật phổ biến trên ứng dụng web. Qua đó, các hạn chế của phương pháp kiểm thử truyền thống đã được chỉ ra, làm rõ nhu cầu ứng dụng AI nhằm nâng cao mức độ tự động hóa và hiệu quả kiểm thử.

Bên cạnh đó, chương đã giới thiệu nền tảng của AI tạo sinh và mô hình ngôn ngữ lớn, cùng các kỹ thuật quan trọng như Prompt Engineering, Chain-of-Thought, ReAct, Tool Calling và AI Agent. Đồng thời, các hướng nghiên cứu ứng dụng AI tạo sinh trong kiểm thử xâm nhập đã được khảo sát, so sánh và đánh giá, từ mô hình AI hỗ trợ chuyên gia đến các hệ thống AI Agent và Multi-Agent, qua đó làm rõ những ưu điểm, hạn chế, thách thức và xu hướng phát triển của lĩnh vực này.

Những nội dung đã trình bày trong chương là cơ sở khoa học và thực tiễn để xây dựng hệ thống AI Pentest Agent. Trên nền tảng đó, Chương 2 sẽ tập trung phân tích yêu cầu, đề xuất kiến trúc và thiết kế các thành phần của hệ thống AI Pentest Agent hỗ trợ kiểm thử xâm nhập ứng dụng web.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG AI PENTEST AGENT

2.1. Mô tả bài toán

Đồ án đặt ra bài toán xây dựng một AI Pentest Agent hỗ trợ một phần quy trình kiểm thử xâm nhập ứng dụng web. Hệ thống cần có khả năng tiếp nhận yêu cầu, phân tích thông tin về mục tiêu và trạng thái kiểm thử, lựa chọn công cụ phù hợp và sử dụng kết quả thu được để xác định bước kiểm tra tiếp theo.

Đầu vào gồm yêu cầu của người kiểm thử, thông tin mục tiêu và dữ liệu thu thập trong quá trình kiểm tra. LLM phối hợp với cơ chế quản lý trạng thái và công cụ để hỗ trợ ra quyết định. Đầu ra gồm thông tin khảo sát, kết quả kiểm tra và các phát hiện được ghi nhận cùng bằng chứng liên quan.

Hệ thống được định hướng hoạt động theo nguyên lý suy luận - hành động - quan sát của ReAct. Các bước xử lý được điều chỉnh dựa trên phản hồi từ công cụ và tiến độ đã ghi nhận, thay vì tuân theo một chuỗi cố định. Quá trình này được hỗ trợ bởi cơ chế lập kế hoạch, thực thi hành động và cập nhật trạng thái từ kết quả thu được.

Hệ thống hoạt động trong phạm vi được cấp quyền và tuân thủ các điều kiện kiểm soát thực thi. Người kiểm thử thiết lập mục tiêu, theo dõi tiến trình và xác nhận thao tác khi cần. Agent đảm nhiệm việc phân tích và điều phối, đồng thời duy trì ngữ cảnh và trạng thái trong quá trình kiểm thử.

2.2. Các yêu cầu chức năng và yêu cầu phi chức năng

Từ bài toán đã được xác định tại mục 2.1, hệ thống AI Pentest Agent cần đáp ứng các yêu cầu về chức năng và chất lượng nhằm hỗ trợ người kiểm thử trong quá trình thực hiện kiểm thử bảo mật. Các yêu cầu này là cơ sở để thiết kế kiến trúc và các thành phần của hệ thống.

2.2.1. Yêu cầu chức năng

Các yêu cầu chức năng của hệ thống AI Pentest Agent được trình bày trong Bảng 2.1.

Bảng 2.1. Các yêu cầu chức năng của hệ thống

Mã	Tên yêu cầu	Mô tả
FR01	Tiếp nhận và xử lý yêu cầu kiểm thử	Cho phép người kiểm thử nhập mục tiêu và yêu cầu kiểm thử. Hệ thống phân tích thông tin đầu vào để xác định nhiệm vụ cần thực hiện.
FR02	Quản lý phiên, ngữ cảnh và trạng thái	Lưu trữ và duy trì thông tin cần thiết trong phiên kiểm thử, bao gồm mục tiêu, lịch sử tương tác, hành động và kết quả đã thu thập.
FR03	Phân tích và lập kế hoạch	Phân tích trạng thái hiện tại và thông tin đã thu thập để xác định hành động tiếp theo. Kế hoạch có thể được điều chỉnh khi có thông tin mới.
FR04	Lựa chọn và thực thi công cụ	Lựa chọn công cụ phù hợp với nhiệm vụ và thực hiện công cụ với các tham số tương ứng.
FR05	Tiếp nhận và phân tích kết quả	Tiếp nhận kết quả từ công cụ và phân tích những thông tin cần thiết cho các bước xử lý tiếp theo.
FR06	Kiểm soát quyền và xác nhận thao tác	Kiểm tra quyền thực thi đối với các hành động và yêu cầu người kiểm thử xác nhận khi cần thiết. Người kiểm thử có thể can thiệp vào quá trình thực thi.
FR07	Quản lý kết quả và phát hiện bảo mật	Ghi nhận và quản lý các kết quả, thông tin và phát hiện bảo mật trong quá trình kiểm thử.
FR08	Tương tác và giám sát	Cho phép người kiểm thử trao đổi với Agent và theo dõi quá trình thực hiện cũng như kết quả trả về.
FR09	Xử lý lỗi trong quá trình thực thi	Xử lý các lỗi phát sinh khi thực hiện tác vụ hoặc gọi công cụ, đồng thời hỗ trợ quá trình tiếp tục hoặc điều chỉnh khi cần.

2.2.2. Yêu cầu phi chức năng

Các yêu cầu phi chức năng của hệ thống AI Pentest Agent được trình bày trong Bảng 2.2.

Bảng 2.2. Các yêu cầu phi chức năng của hệ thống

Mã	Tên yêu cầu	Mô tả
NFR01	An toàn và kiểm soát phạm vi	Đảm bảo Agent chỉ thực hiện các hành động trong phạm vi mục tiêu và quyền được cấp phép. Các thao tác cần xác nhận phải được kiểm soát trước khi thực hiện.
NFR02	Độ chính xác và tin cậy của kết quả	Kết quả phân tích cần dựa trên thông tin thực tế thu được từ công cụ và quá trình kiểm thử. Hệ thống cần hạn chế ảnh hưởng của hiện tượng hallucination trong quá trình ra quyết định.
NFR03	Độ tin cậy và khả năng chịu lỗi	Hệ thống cần duy trì trạng thái phiên khi xảy ra lỗi trong quá trình thực thi. Lỗi của một công cụ hoặc tác vụ không nên làm mất toàn bộ thông tin đã lưu khi có thể xử lý và phục hồi.
NFR04	Hiệu năng	Đảm bảo thời gian phản hồi phù hợp với quá trình tương tác và xử lý lượng ngữ cảnh phát sinh trong phiên kiểm thử một cách hợp lý.
NFR05	Khả năng mở rộng	Cho phép bổ sung hoặc thay thế công cụ và các thành phần hỗ trợ mà không cần thay đổi lớn trong cơ chế hoạt động của Agent.
NFR06	Khả năng bảo trì	Các thành phần của hệ thống được tổ chức rõ ràng, thuận tiện cho việc sửa đổi, kiểm thử, gỡ lỗi và phát triển.
NFR07	Khả năng quan sát và truy vết	Ghi nhận quá trình xử lý, hoạt động của công cụ và kết quả thực thi để người kiểm thử có thể theo dõi và xem lại khi cần.
NFR08	Khả năng sử dụng	Giao diện hỗ trợ người kiểm thử nhập yêu cầu, theo dõi tiến trình, xem kết quả và thực hiện các thao tác xác nhận hoặc can thiệp.

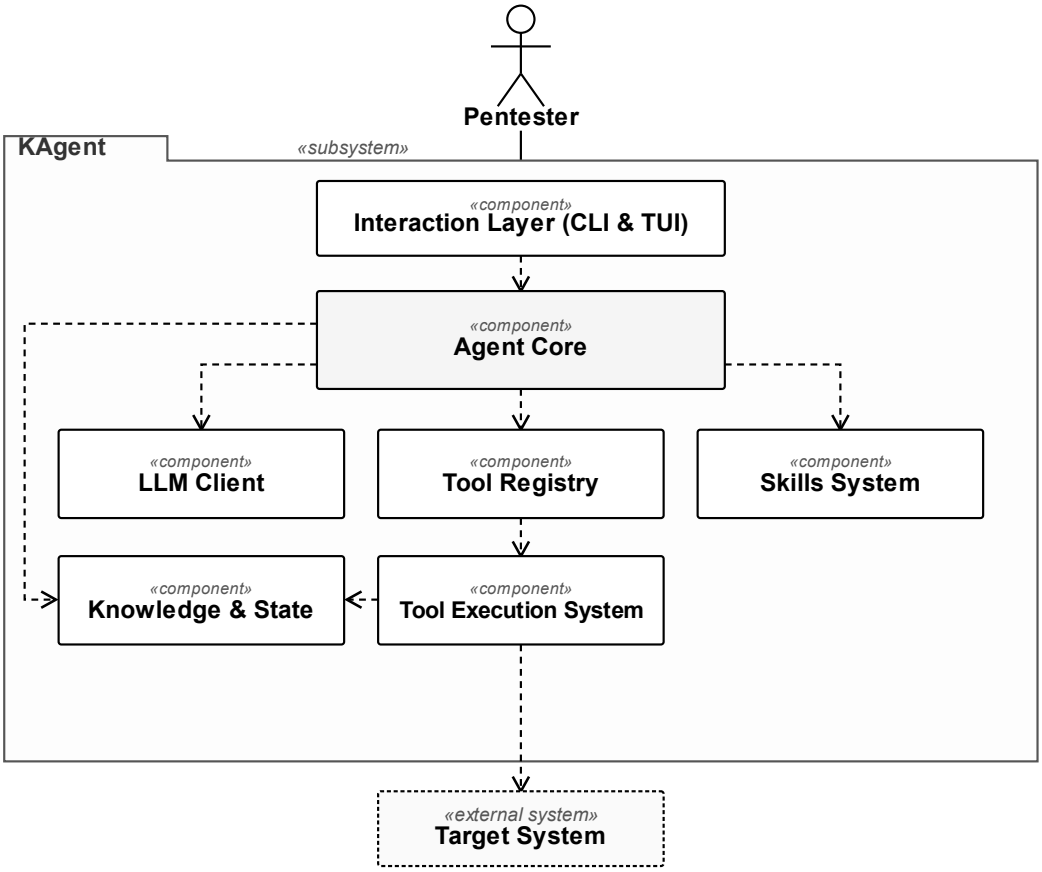
2.3. Thiết kế kiến trúc hệ thống

Trên cơ sở các yêu cầu thiết kế, kiến trúc Single-Agent được lựa chọn nhằm tập trung quá trình phân tích, lập kế hoạch và điều phối hoạt động kiểm thử vào một tác tử chính. Cách tổ chức này hạn chế sự phức tạp trong việc phối hợp và đồng bộ trạng thái giữa nhiều tác tử độc lập. Trong KAgent, các chức năng chuyên biệt được triển khai thông qua Skill, công cụ và các thành phần hỗ trợ, cho phép hệ thống thực hiện các nhiệm vụ kiểm thử nhiều bước trong một quy trình thống nhất.

2.3.1. Kiến trúc tổng quát hệ thống

Hệ thống KAgent được tổ chức thành các thành phần có chức năng và trách nhiệm khác nhau. Kiến trúc tổng quát của hệ thống được trình bày trong Hình 2.1,

thể hiện sự phân tách giữa giao tiếp với người kiểm thử, điều phối của tác nhân AI, các thành phần hỗ trợ và thực thi hoạt động kiểm thử.



Hình 2.1. Kiến trúc tổng quát hệ thống

KAgent được thiết kế theo kiến trúc Single-Agent, trong đó một thành phần điều phối (Agent Core) tổ chức hoạt động của các thành phần hỗ trợ và công cụ kiểm thử. Ở mức tổng quát, hệ thống được xem xét trong mối quan hệ giữa người kiểm thử (Pentester), KAgent và hệ thống mục tiêu (Target System). Pentester gửi yêu cầu và nhận kết quả thông qua lớp tương tác (Interaction Layer).

Interaction Layer: Lớp này cung cấp giao diện dòng lệnh (CLI) và giao diện văn bản tương tác (TUI) để Pentester nhập yêu cầu kiểm thử, theo dõi quá trình thực hiện và nhận kết quả. Lớp này chủ yếu chịu trách nhiệm tiếp nhận và hiển thị thông tin, không trực tiếp thực hiện các thao tác kiểm thử.

Agent Core: Thành phần này giữ vai trò điều phối trung tâm trong KAgent, tiếp nhận yêu cầu từ Interaction Layer, xây dựng ngữ cảnh xử lý và xác định các bước tiếp theo dựa trên trạng thái kiểm thử. Thành phần này kết nối khả năng suy luận của mô hình ngôn ngữ với các hoạt động thực thi, đồng thời truy xuất dữ liệu

từ thành phần quản lý tri thức và trạng thái (Knowledge & State) để bổ sung ngữ cảnh.

Các thành phần hỗ trợ chính của Agent Core gồm:

Lớp giao tiếp mô hình (LLM Client): Nhiệm vụ của thành phần này là kết nối KAgent với các mô hình ngôn ngữ lớn (LLM). Thành phần này gửi yêu cầu, nhận phản hồi và chuyển đổi kết quả về dạng phù hợp để Agent Core tiếp tục xử lý. Việc tách LLM Client thành một thành phần riêng giúp KAgent có thể hỗ trợ nhiều nhà cung cấp và mô hình khác nhau mà không phải thay đổi phần xử lý chính.

Hệ thống kỹ năng (Skills System): Hệ thống này quản lý các kỹ năng được sử dụng trong quá trình kiểm thử. Mỗi kỹ năng cung cấp hướng dẫn, quy trình xử lý và các yêu cầu cần thiết cho một nhóm nhiệm vụ cụ thể, giúp Agent Core bổ sung kiến thức, lựa chọn cách thức thực hiện phù hợp và định hướng các bước xử lý trong từng tình huống kiểm thử.

Bộ đăng ký công cụ (Tool Registry): Thành phần này quản lý danh mục và mô tả của các công cụ mà KAgent có thể sử dụng, cung cấp thông tin để LLM đề xuất lời gọi phù hợp với nhiệm vụ. Thành phần này tập trung việc đăng ký và tra cứu công cụ theo cơ chế thống nhất, giúp bổ sung công cụ mà không phải đưa logic thực hiện từng thao tác vào Agent Core.

Hệ thống thực thi công cụ (Tool Execution System): Hệ thống này tập hợp các công cụ thực hiện thao tác trên tài nguyên cục bộ, tương tác với Target System hoặc làm việc với dữ liệu kiểm thử của KAgent. Việc thực thi chịu các điều kiện kiểm soát tương ứng với từng loại công cụ. Kết quả hoặc thông tin lỗi được trả về Agent Core để tiếp tục phân tích và điều phối.

Quản lý tri thức và trạng thái (Knowledge & State): Nhóm này quản lý trạng thái và các thông tin được tích lũy trong quá trình hệ thống hoạt động, bao gồm trạng thái phiên làm việc, phạm vi kiểm thử, các đầu vào đã phát hiện, các điểm nghi ngờ cần kiểm tra, kết quả xác minh, bằng chứng, kết quả phát hiện và thông tin về mức độ bao phủ kiểm thử. Thành phần này cung cấp khả năng lưu trữ và truy xuất dữ liệu cần thiết cho quá trình xử lý, hỗ trợ Agent Core xác định trạng thái hiện tại và tiếp tục các bước kiểm thử tiếp theo.

Trong quá trình xử lý, Agent Core kết hợp ngữ cảnh với phản hồi từ LLM và kết quả công cụ để xác định bước tiếp theo. Việc cập nhật trạng thái của quy trình kiểm thử (workflow), đăng ký bằng chứng và ghi nhận phát hiện bảo mật (Finding) được thực hiện qua các công cụ quản lý tương ứng, theo những điều kiện đã xác định. Các dữ liệu được ghi nhận tiếp tục cung cấp thông tin cho hoạt động điều phối.

2.3.2. Kiến trúc các thành phần của hệ thống

Dựa trên kiến trúc tổng quát ở Mục 2.3.1, các thành phần bên trong KAgent được trình bày theo tám nhóm chức năng nhằm làm rõ trách nhiệm và quan hệ phối hợp. Ngoài các thành phần chính thể hiện trong Hình 2.1, hệ thống còn sử dụng hạ tầng dùng chung để hỗ trợ cấu hình, kiểm soát thực thi, ghi nhật ký và xử lý thông tin nhạy cảm.

Interaction Layer: Thành phần này gồm CLI và TUI, đóng vai trò kết nối người kiểm thử với hệ thống. CLI tiếp nhận tham số và cấu hình để khởi tạo các thành phần cần thiết. Trong phiên làm việc, TUI hiển thị diễn biến, tiếp nhận yêu cầu, lệnh điều khiển và các quyết định của người kiểm thử. Yêu cầu được chuyển tới Agent Core để xử lý, còn kết quả được đưa trở lại giao diện. Sự phân chia này tách trách nhiệm giao tiếp khỏi việc phân tích và thực thi kiểm thử.

Agent Core: Thành phần này duy trì vòng lặp Agent Loop, chuẩn bị ngữ cảnh xử lý và phối hợp với Decision Planner. Planner sử dụng mục tiêu và trạng thái quy trình để cung cấp định hướng khi có đủ cơ sở lập kế hoạch. LLM sử dụng ngữ cảnh để đề xuất hành động cụ thể, còn Agent Core xử lý phản hồi, điều phối lời gọi công cụ và theo dõi điều kiện dừng. Các sự kiện về quá trình xử lý, lời gọi công cụ và kết quả được phát tới giao diện để người kiểm thử theo dõi.

LLM Client: Thành phần này sử dụng các Provider Clients để chuyển đổi yêu cầu và phản hồi giữa Agent Core với dịch vụ LLM. Client Factory khởi tạo client theo cấu hình nhà cung cấp, mô hình và tham số sử dụng. Các kiểu dữ liệu chung cho thông điệp, lời gọi công cụ và phản hồi giúp Agent Core xử lý kết quả thống nhất. Việc truyền nhận và thử lại những lỗi có thể phục hồi được xử lý tại lớp kết nối.

Skills System: Hệ thống này quản lý danh mục kỹ năng thông qua Skill Registry, phát hiện kỹ năng trong các thư mục được cấu hình và nạp nội dung hướng dẫn khi được yêu cầu. Metadata của kỹ năng được sử dụng khi lập kế hoạch, còn nội dung playbook được bổ sung vào ngữ cảnh sau khi nạp thành công. Hệ thống kiểm tra trạng thái khả dụng và điều kiện nạp của từng kỹ năng. Với một số kỹ năng, hệ thống còn kiểm tra điều kiện hoàn tất để hỗ trợ Agent theo dõi nhiệm vụ. Nội dung kỹ năng định hướng hành động, còn quyền và giới hạn thực thi được kiểm soát bởi các cơ chế riêng của hệ thống.

Tool Registry: Thành phần này quản lý các công cụ qua tên định danh và cung cấp mô tả cùng lược đồ tham số cho LLM. Khi Agent Core chuyển một lời gọi phù hợp, Registry tra cứu công cụ, kiểm tra đối số theo cơ chế mà công cụ hỗ trợ và

phối hợp với hệ thống thực thi (runtime) của KAgent để áp dụng các kiểm soát tương ứng. Yêu cầu được chuyển tới phần hiện thực của công cụ sau khi đáp ứng các điều kiện tương ứng. Logic của từng thao tác được giữ tại công cụ, thay vì đưa vào Registry.

Tool Execution System: Hệ thống này tập hợp các công cụ được đăng ký theo giao diện chung Tool, phục vụ các nhóm chức năng như thao tác hệ thống và tệp tin, tương tác mạng và web, thu thập lưu lượng, quản lý kỹ năng và tài nguyên, quản lý trạng thái kiểm thử và kết quả. Mỗi công cụ thực hiện thao tác tương ứng, áp dụng các điều kiện kiểm soát cần thiết và trả kết quả hoặc thông tin lỗi về Agent Core để tiếp tục xử lý. Các nhóm công cụ theo mục đích sử dụng được trình bày tại Bảng 2.8.

Knowledge & State: Nhóm này duy trì dữ liệu cần thiết cho quá trình hoạt động của Agent. Nhóm Session & Memory quản lý lịch sử, trạng thái phiên và bộ nhớ phục vụ việc tiếp tục công việc. `WorkflowState` theo dõi tiến độ kiểm thử, đối tượng nghi vấn, kết quả xác minh và liên kết bằng chứng. Nhóm Coverage & Findings quản lý mức độ bao phủ kiểm thử và các phát hiện. Intelligence cung cấp tri thức hỗ trợ phân tích, còn Target lưu thông tin về mục tiêu kiểm thử. Dữ liệu mục tiêu và phạm vi được sử dụng trong các cơ chế kiểm soát thực thi. Các nguồn dữ liệu này hỗ trợ Agent lập kế hoạch, theo dõi công việc đã thực hiện và xác định các bước xử lý tiếp theo.

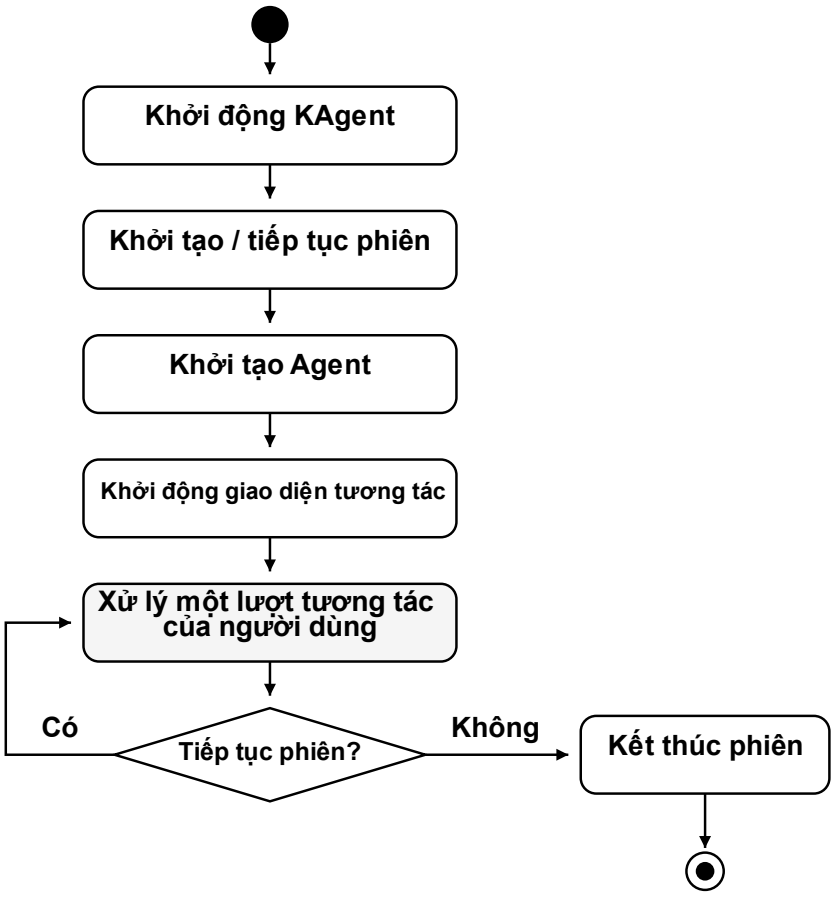
Hạ tầng dùng chung (Shared Infrastructure): Nhóm này cung cấp các dịch vụ cấu hình, kiểm soát quyền, ghi nhật ký và xử lý thông tin nhạy cảm cho toàn hệ thống. Configuration quản lý tham số khởi tạo và vận hành. Permission phối hợp với runtime để kiểm soát quyền, phạm vi và giới hạn thực thi theo từng loại công cụ. Logger ghi lại hoạt động để phục vụ theo dõi và truy vết. Redaction hỗ trợ che giấu thông tin nhạy cảm trong quá trình xử lý và lưu trữ dữ liệu.

2.4. Thiết kế luồng xử lý và cơ chế hoạt động

2.4.1. Quy trình hoạt động của hệ thống

a) Quy trình hoạt động tổng quát

Quy trình hoạt động tổng quát của KAgent mô tả các bước chính từ khi người dùng khởi động hệ thống, thiết lập phiên làm việc, khởi tạo Agent cho đến khi phiên kết thúc. Quy trình này cho thấy cách KAgent tổ chức một phiên làm việc và xử lý nhiều lượt tương tác của người dùng trong cùng một phiên. Sơ đồ tổng quan quy trình vận hành của hệ thống KAgent được trình bày trong Hình 2.2.



Hình 2.2. Quy trình hoạt động tổng quát của KAgent

1. **Khởi động KAgent:** Người dùng sử dụng giao diện dòng lệnh để khởi động hệ thống, từ đó hệ thống bắt đầu quá trình khởi tạo và chuẩn bị môi trường cho phiên làm việc.
2. **Khởi tạo / tiếp tục phiên:** Hệ thống tạo một phiên làm việc mới hoặc tiếp tục một phiên đã được lưu trước đó. Phiên làm việc lưu giữ trạng thái và các thông tin cần thiết trong quá trình người dùng tương tác với hệ thống, qua đó duy trì tính liên tục giữa các lượt xử lý.
3. **Khởi tạo Agent:** Hệ thống kết nối Agent với LLM Client, hệ thống kỹ năng, các công cụ đã đăng ký và các thành phần quản lý trạng thái. Sau bước này, Agent có thể tiếp nhận yêu cầu và sử dụng các thành phần đã được chuẩn bị để xử lý nhiệm vụ trong phiên.
4. **Khởi động giao diện tương tác:** Hệ thống mở giao diện trên terminal để người dùng nhập yêu cầu, theo dõi tiến trình và xem kết quả. Giao diện cũng tiếp nhận quyết định cho phép hoặc từ chối khi một hành động cần người dùng xác nhận.

5. **Xử lý một lượt tương tác:**

Khi người dùng gửi yêu cầu, KAgent tiếp nhận và chuyển yêu cầu cho Agent xử lý. Agent phân tích yêu cầu dựa trên ngữ cảnh, trạng thái phiên và các kết quả kiểm thử đã được duy trì, đồng thời có thể sử dụng các kỹ năng và công cụ cần thiết trước khi tạo kết quả trả về cho người dùng.

6. **Tiếp tục phiên:**

Sau khi hoàn thành một lượt tương tác, hệ thống kiểm tra xem phiên làm việc có tiếp tục hay không. Nếu tiếp tục, KAgent quay lại bước Xử lý một lượt tương tác để tiếp nhận yêu cầu tiếp theo. Trạng thái và thông tin của phiên được giữ lại để phục vụ các lượt xử lý sau. Nếu kết thúc, hệ thống chuyển sang bước Kết thúc phiên làm việc.

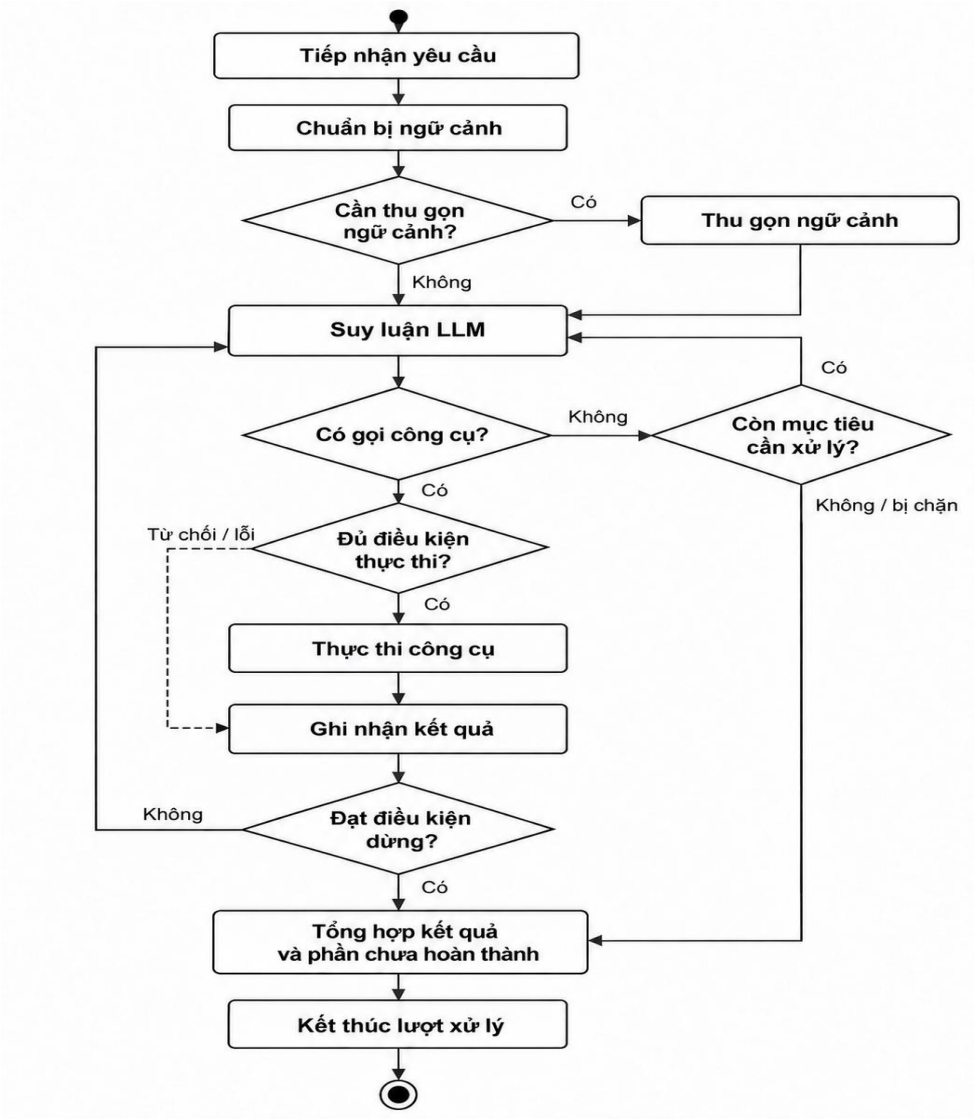
7. **Kết thúc phiên làm việc:**

Khi người dùng thoát khỏi KAgent, hệ thống dừng quá trình tương tác, kết thúc các hoạt động theo dõi đang chạy và đóng các kết nối tích hợp đã mở.

Qua quy trình trên, phiên làm việc giúp duy trì trạng thái tương tác giữa người dùng và KAgent, cho phép hệ thống xử lý nhiều lượt yêu cầu trong cùng một phiên mà không cần khởi tạo lại Agent cho từng yêu cầu. Mỗi lượt tương tác được xử lý như một chu trình riêng nhưng vẫn kế thừa trạng thái và các thông tin đã được duy trì trong phiên.

b) Quy trình xử lý một lượt tương tác

Trong một phiên làm việc, mỗi yêu cầu của người dùng được Agent xử lý như một lượt tương tác. Quá trình này bắt đầu khi yêu cầu được tiếp nhận và kết thúc khi Agent tạo phản hồi cuối cùng hoặc khi quá trình xử lý đạt một trong các điều kiện dừng của hệ thống. Quy trình xử lý một lượt tương tác của KAgent được trình bày trong Hình 2.3.



Hình 2.3. Quy trình xử lý một lượt tương tác của KAgent

Lượt tương tác bắt đầu bằng việc tiếp nhận yêu cầu và chuẩn bị ngữ cảnh dựa trên trạng thái hiện tại của phiên, bao gồm lịch sử tương tác, trạng thái kiểm thử và các khả năng hành động. KAgent duy trì các mục tiêu kiểm thử nhận diện từ yêu cầu trong workflow. Quá trình khảo sát và phân tích đầu vào hình thành các đối tượng nghi vấn (Candidate). Các lượt kiểm tra sau đó thu thập bằng chứng để Agent đánh giá và ghi nhận kết quả trước khi xem xét tạo Finding.

Ở đầu mỗi lượt tương tác, sau khi tiếp nhận yêu cầu của người dùng, KAgent ước lượng kích thước ngữ cảnh và kiểm tra các điều kiện thu gọn lịch sử. Khi đáp ứng các điều kiện cần thiết, hệ thống tạo bản tóm lược về mục tiêu, tiến độ và những công việc còn phải xử lý. Bản tóm lược chỉ được sử dụng để thay thế lịch sử cũ nếu mức giảm kích thước đạt yêu cầu. Sau bước chuẩn bị ngữ cảnh, Agent bắt đầu vòng suy luận để xử lý yêu cầu dựa trên lịch sử và trạng thái phiên hiện có.

Tại bước suy luận bằng LLM, Agent sử dụng ngữ cảnh hiện tại để tạo phản hồi hoặc yêu cầu thực hiện hành động thông qua các công cụ được cung cấp. Khi phản hồi không yêu cầu công cụ, hệ thống còn đối chiếu trạng thái workflow và các mục tiêu kiểm thử đã ghi nhận trước khi chấp nhận kết thúc. Nếu vẫn còn công việc có thể xử lý, vòng lặp được tiếp tục. Nếu bị chặn hoặc đạt giới hạn, hệ thống tổng hợp phần đã thực hiện và những nội dung còn chưa hoàn thành.

Đối với hành động yêu cầu sử dụng công cụ, KAgent kiểm tra tính hợp lệ và quyền thực hiện trước khi tiến hành thao tác. Khi đáp ứng các điều kiện cần thiết, công cụ được thực thi và kết quả được ghi nhận. Trường hợp hành động không hợp lệ, không được phép hoặc bị từ chối, hệ thống ghi nhận lý do tương ứng. Tùy theo kết quả và trạng thái hiện tại, quá trình xử lý có thể tiếp tục hoặc kết thúc lượt tương tác theo các điều kiện dừng đã thiết lập.

Sau mỗi yêu cầu gọi công cụ, KAgent ghi nhận kết quả thực thi hoặc lý do không thực hiện và bổ sung thông tin này vào ngữ cảnh. Các thay đổi về điểm đầu vào, Candidate hoặc kết quả kiểm tra được cập nhật vào trạng thái kiểm thử qua các công cụ quản lý tương ứng. Agent sử dụng dữ liệu vừa ghi nhận để lựa chọn bước tiếp theo. Vì vậy, một lượt tương tác có thể gồm nhiều lần suy luận và sử dụng công cụ trước khi tạo phản hồi cuối cùng.

Để hạn chế vòng xử lý kéo dài, KAgent kiểm soát số bước thực hiện. Khi đạt giới hạn số bước, hệ thống chuyển sang tổng hợp kết quả cùng những công việc còn chưa hoàn thành. Nếu gặp trở ngại không thể khắc phục trong phạm vi được phép, lượt xử lý được dừng và trạng thái tương ứng được ghi nhận. Trạng thái phiên đã lưu vẫn được giữ để phục vụ việc xem lại hoặc tiếp tục ở lượt sau.

Nhìn chung, quy trình xử lý một lượt tương tác của KAgent được tổ chức theo vòng lặp suy luận - thực hiện hành động - tiếp nhận kết quả. Kết quả của mỗi bước được ghi nhận và đưa trở lại ngữ cảnh làm việc để Agent tiếp tục suy luận và lựa chọn hành động tiếp theo. Vòng lặp kết thúc khi Agent tạo phản hồi cuối cùng hoặc khi một điều kiện dừng của hệ thống được kích hoạt, qua đó cho phép Agent thực hiện các nhiệm vụ nhiều bước trong cùng một lượt tương tác.

2.4.2. Cơ chế quản lý ngữ cảnh và trạng thái phiên

a) Cấu trúc trạng thái và các nguồn dữ liệu

Trong quá trình kiểm thử, KAgent cần duy trì thông tin qua nhiều bước xử lý và lượt tương tác để theo dõi tiến độ và sử dụng lại những kết quả đã ghi nhận. Để đáp ứng yêu cầu này, hệ thống quản lý trạng thái phiên kết hợp với các nguồn dữ

liệu hỗ trợ. Trạng thái phiên bao gồm lịch sử hội thoại, thông tin đích kiểm thử (Target), phạm vi kiểm thử (Scope), bộ nhớ phiên (Session Memory) và trạng thái quy trình (Workflow State). Trong đó, phạm vi kiểm thử được quản lý bởi `EngagementState`, đồng thời thành phần này lưu giữ trạng thái quyền liên quan trong phiên. Workflow State theo dõi mục tiêu nhiệm vụ, tiến độ, đầu vào, điểm nghi vấn, kết quả kiểm tra và tham chiếu bằng chứng. Bộ nhớ phiên và trạng thái quy trình được biểu diễn tương ứng bằng hai cấu trúc dữ liệu `SessionMemory` và `WorkflowState`. Các thành phần cùng vai trò tương ứng được trình bày trong Bảng 2.3.

Bảng 2.3. Cấu trúc trạng thái và các nguồn dữ liệu hỗ trợ của KAgent

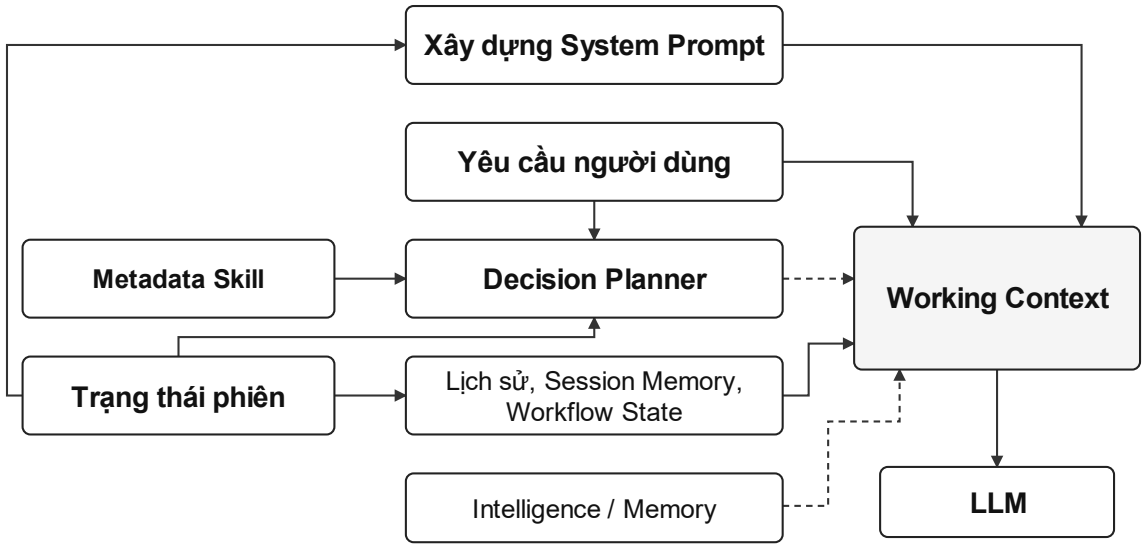
Nhóm	Thành phần	Vai trò
Trạng thái phiên	Session History	Duy trì nội dung tương tác trong phiên
	Session Memory	Lưu các thông tin cần thiết để tiếp tục xử lý
	Target / Scope	Lưu thông tin đích và phạm vi kiểm thử của phiên
	Workflow State	Theo dõi mục tiêu, tiến độ, kết quả kiểm tra và tham chiếu bằng chứng
Nguồn dữ liệu hỗ trợ	Durable Memory	Cung cấp thông tin được truy hồi khi phù hợp
	Intelligence	Cung cấp thông tin hỗ trợ quá trình phân tích
	Engagement	Cung cấp ghi chú hỗ trợ xây dựng chỉ dẫn hệ thống
	Coverage	Theo dõi mức độ bao phủ kiểm thử
	Findings	Lưu các phát hiện đã được xác nhận

Các nguồn dữ liệu hỗ trợ bổ sung thông tin cho Agent, nhưng không thay thế trạng thái quy trình của nhiệm vụ đang thực hiện. Khi chuẩn bị một lượt tương tác, KAgent kết hợp thông tin phù hợp từ các nguồn này với trạng thái phiên hiện tại để xây dựng ngữ cảnh cho LLM.

b) Xây dựng ngữ cảnh cho một lượt tương tác

Trước mỗi lượt tương tác, KAgent xây dựng ngữ cảnh từ trạng thái và các dữ liệu phù hợp với nhiệm vụ. Chỉ dẫn hệ thống (system prompt) được xây dựng từ các chỉ dẫn nền và thông tin vận hành phù hợp với trạng thái phiên, như mục tiêu và phạm vi kiểm thử, nhằm xác định vai trò cùng các ràng buộc của Agent. Trong khi

đó, trạng thái quy trình, bộ nhớ phiên và thông tin truy hồi được cung cấp dưới dạng dữ liệu tham khảo. Việc phân biệt chỉ dẫn hệ thống với dữ liệu tham khảo giúp hạn chế nguy cơ sử dụng thông tin đã lưu như chỉ dẫn của người kiểm thử hoặc coi đó là căn cứ cấp quyền thực thi hay chấp nhận kết luận kiểm thử. Quy trình xây dựng ngữ cảnh cho một lượt tương tác của KAgent được minh họa trong Hình 2.4.



Hình 2.4. Quy trình xây dựng ngữ cảnh cho một lượt tương tác KAgent

Ngữ cảnh làm việc (Working Context) kết hợp chỉ dẫn hệ thống, lịch sử hội thoại, yêu cầu hiện tại và thông tin hỗ trợ. Decision Planner sử dụng yêu cầu người dùng, thông tin mục tiêu, trạng thái workflow và metadata của các kỹ năng khả dụng để đề xuất định hướng công việc. Hệ thống bổ sung định hướng này vào ngữ cảnh khi phù hợp. Tùy nhiệm vụ, hệ thống còn bổ sung tri thức Intelligence và dữ liệu Memory. Nội dung chi tiết của kỹ năng chỉ được đưa vào khi kỹ năng tương ứng được nạp hoặc kích hoạt, giúp ngữ cảnh tập trung vào công việc đang xử lý.

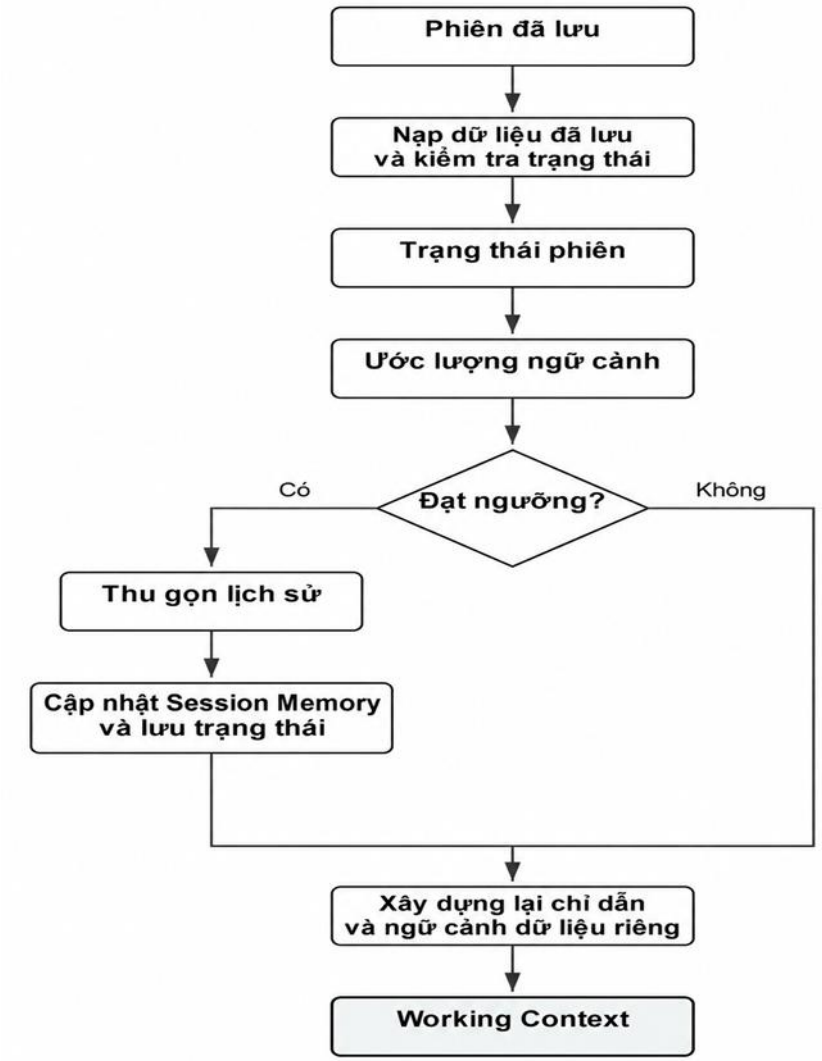
Ngữ cảnh này giúp LLM tiếp tục xử lý nhiệm vụ dựa trên tiến độ và kết quả đã ghi nhận, trong khi các bản ghi workflow vẫn được quản lý riêng trong phiên.

c) Thu gọn ngữ cảnh và khôi phục phiên

Để quản lý lượng thông tin trong ngữ cảnh và hỗ trợ tiếp tục kiểm thử sau khi phiên làm việc bị gián đoạn, KAgent sử dụng các cơ chế thu gọn ngữ cảnh và khôi phục phiên. Hai cơ chế này được minh họa trong Hình 2.5.

Trong quá trình kiểm thử, lịch sử hội thoại tích lũy thêm yêu cầu, phản hồi và kết quả công cụ, làm tăng lượng dữ liệu gửi tới LLM. Khi đáp ứng các điều kiện thu gọn, KAgent có thể tạo bản tóm lược về mục tiêu, tiến độ và công việc còn phải xử lý. Hệ thống ước lượng kích thước ngữ cảnh trước và sau khi thu gọn để kiểm tra mức giảm. Chỉ khi đạt mức giảm tối thiểu, bản tóm lược mới được đưa vào bộ nhớ

phiên và sử dụng cùng phần hội thoại gần nhất để thay thế lịch sử cũ. Nếu chưa đạt yêu cầu này, lịch sử cũ được giữ lại.



Hình 2.5. Cơ chế thu gọn ngữ cảnh và khôi phục phiên của KAgent

Để hạn chế lượng dữ liệu đưa vào ngữ cảnh, KAgent lưu các kết quả công cụ dài từ những nguồn được hỗ trợ vào tệp riêng, đồng thời giữ bản xem trước và tham chiếu trong lịch sử hội thoại. Khi kích thước ngữ cảnh tăng cao, hệ thống có thể tiếp tục rút gọn các kết quả công cụ cũ theo chính sách lưu giữ tương ứng, ưu tiên bảo toàn những thông tin quan trọng phục vụ quá trình kiểm thử. Khi cần phân tích thêm, Agent có thể truy xuất phần dữ liệu đã lưu sau khi kiểm tra quyền truy cập.

Cơ chế này cho phép KAgent tiếp tục sử dụng kết quả từ các công cụ mà không cần duy trì toàn bộ nội dung của chúng trong ngữ cảnh làm việc. Khác với cơ chế tạo bản tóm lược lịch sử hội thoại, việc rút gọn kết quả công cụ giúp giảm lượng thông tin đưa vào ngữ cảnh mà không làm thay đổi các bản ghi trạng thái và bằng chứng kiểm thử được quản lý riêng trong workflow.

Để tiếp tục kiểm thử sau khi phiên làm việc bị gián đoạn, KAgent lưu lịch sử hội thoại, thông tin mục tiêu, bộ nhớ phiên, phạm vi kiểm thử và trạng thái workflow. Khi mở lại phiên, hệ thống nạp các dữ liệu này và xây dựng lại chỉ dẫn cho LLM. Các lượt xác minh chưa kết thúc được ghi nhận là gián đoạn để phân biệt với những lượt đã hoàn tất. Quyền thực thi tạm thời không được khôi phục, vì vậy các thao tác tiếp theo phải được kiểm tra quyền lại trước khi thực hiện.

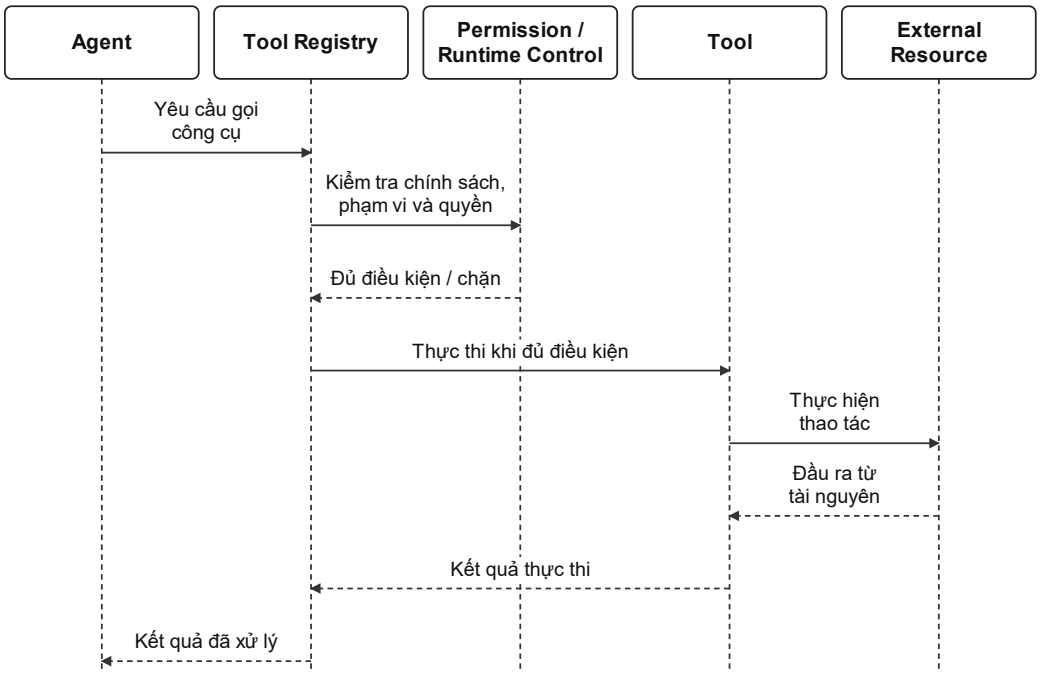
2.4.3. Cơ chế xử lý và thực thi công cụ

Để thực hiện hành động do LLM đề xuất, KAgent tiếp nhận yêu cầu gọi công cụ, kiểm tra tham số và quyền sử dụng, sau đó chuyển yêu cầu tới công cụ đã đăng ký. Kết quả được đưa trở lại Agent làm dữ liệu cho bước suy luận tiếp theo. Phần này trình bày cơ chế xử lý một yêu cầu gọi công cụ và cách điều phối khi phản hồi của LLM chứa nhiều yêu cầu.

a) Xử lý và thực thi một yêu cầu gọi công cụ

Đối với một yêu cầu gọi công cụ, Agent tiếp nhận yêu cầu từ phản hồi của LLM và kiểm tra các thông tin cần thiết trước khi thực hiện. Trong trường hợp lượt tương tác hoặc kỹ năng đang thực hiện áp dụng giới hạn đối với công cụ hay hành động được phép sử dụng, Agent kiểm tra yêu cầu có phù hợp với phạm vi hiện tại hay không trước khi tiếp tục.

Yêu cầu phù hợp được chuyển đến Tool Registry để tra cứu và điều phối tới công cụ tương ứng. Thành phần này tách việc quản lý công cụ khỏi cách thực hiện từng thao tác cụ thể. Nếu không tìm thấy công cụ, hệ thống trả thông tin lỗi về Agent. Cơ chế xử lý một yêu cầu được trình bày trong Hình 2.6.



Hình 2.6. Cơ chế xử lý và thực thi một yêu cầu gọi công cụ của KAgent

Để ngăn công cụ thực hiện thao tác ngoài quyền được cấp, hệ thống kiểm tra phạm vi tài nguyên hoặc mục tiêu, giới hạn thao tác và hiệu lực của quyền hiện hành tùy theo loại công cụ. Các trạng thái từ chối, thu hồi hoặc hết hạn được kiểm tra khi áp dụng. Với hành động cần phê duyệt, người kiểm thử xem thông tin thao tác và quyết định cho phép hoặc từ chối. Chế độ YOLO có thể tự xử lý bước phê duyệt theo chính sách, nhưng các kiểm tra về phạm vi và quyền vẫn được thực hiện. Đối với công cụ thực thi lệnh, hệ thống kiểm tra yêu cầu trước khi khởi chạy tiến trình và kiểm soát thời gian chạy, việc hủy tác vụ cùng lượng đầu ra thu nhận. Các lệnh được thực hiện dưới quyền của tài khoản vận hành. Các tương tác qua công cụ HTTP tích hợp được kiểm tra phạm vi mục tiêu và quyền hiện hành theo cơ chế riêng.

Sau khi các điều kiện cần thiết được thỏa mãn, Tool Registry chuyển yêu cầu đến công cụ tương ứng để thực thi. Một số công cụ có thể áp dụng thêm các bước kiểm tra an toàn hoặc kiểm tra tính hợp lệ của đầu vào trước khi thực hiện thao tác. Công cụ sau đó tương tác với tài nguyên hoặc mục tiêu phù hợp với chức năng của mình. Kết quả thực thi, bao gồm kết quả thành công hoặc thông tin lỗi, được trả về thông qua cơ chế điều phối và chuyển lại cho Agent.

Agent sử dụng kết quả công cụ để đánh giá thao tác vừa thực hiện và xác định cần kiểm tra thêm hay có thể kết thúc. Những điểm đầu vào mới, Candidate hoặc kết quả xác minh được ghi nhận qua các công cụ quản lý trạng thái. Việc ghi nhận này

giúp các bước tiếp theo sử dụng tiến độ đã lưu thay vì chỉ dựa vào phần mô tả trong hội thoại.

b) Xử lý nhiều yêu cầu gọi công cụ

Phản hồi của LLM có thể chứa nhiều yêu cầu gọi công cụ. Việc lựa chọn hình thức xử lý phụ thuộc vào tính chất của công cụ và quan hệ giữa các yêu cầu. Những thao tác cần bảo đảm thứ tự hoặc tính nhất quán của trạng thái được xử lý tuần tự, trong khi các thao tác độc lập có thể được thực hiện đồng thời trong giới hạn của hệ thống. Mỗi yêu cầu đều tuân theo cơ chế kiểm tra và thực thi đã trình bày ở phần a).

- **Xử lý tuần tự:**

Một yêu cầu được hoàn tất trước khi chuyển sang yêu cầu tiếp theo. Cách tổ chức này phù hợp khi thao tác sau phụ thuộc vào trạng thái hoặc kết quả của thao tác trước, giúp duy trì trình tự cần thiết của quá trình kiểm tra.

- **Xử lý đồng thời:**

Những yêu cầu phù hợp được thực hiện song song trong giới hạn đồng thời do hệ thống quy định nhằm giảm thời gian chờ và kiểm soát tài nguyên. Kết quả của từng yêu cầu được thu nhận sau khi hoàn tất và trả về Agent để tiếp tục phân tích. Việc thực hiện đồng thời không bỏ qua kiểm tra phạm vi, quyền hoặc điều kiện riêng của từng công cụ.

Sau khi các yêu cầu gọi công cụ được xử lý, KAgent tổng hợp kết quả hoặc thông tin lỗi tương ứng với từng yêu cầu và chuyển lại cho Agent. Agent sử dụng những thông tin này để đánh giá kết quả thực hiện, cập nhật trạng thái liên quan khi cần thiết và xác định các bước xử lý tiếp theo.

2.5. Thiết kế các mô-đun chức năng hệ thống

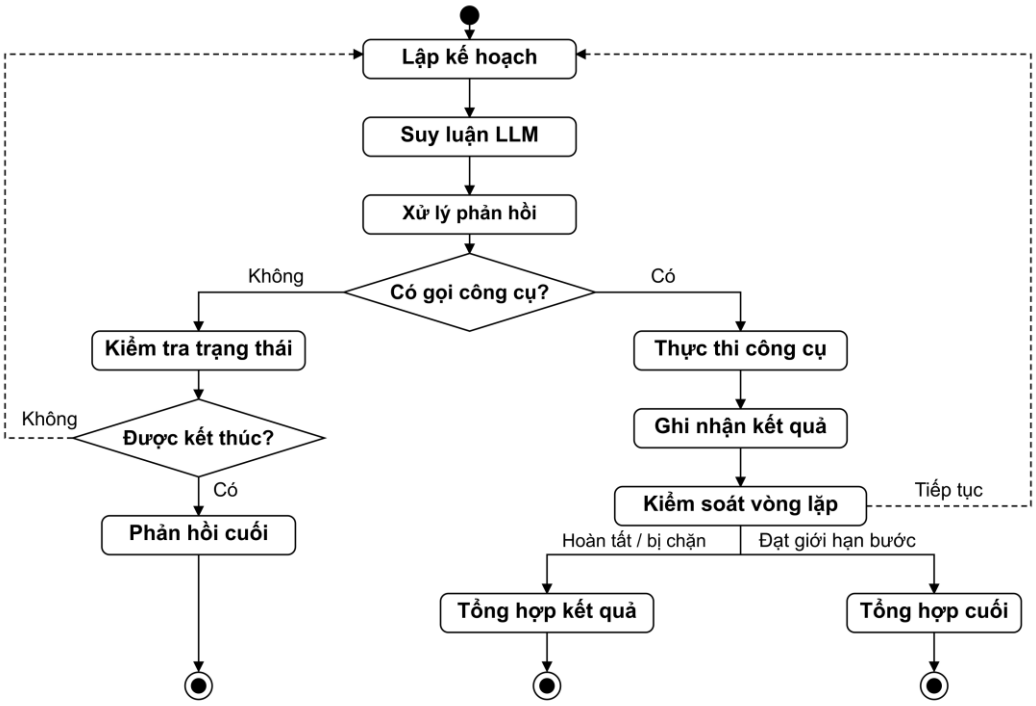
2.5.1. Mô-đun điều phối và lập kế hoạch tác vụ

a) Cơ chế điều phối vòng lặp Agent

Agent Core điều phối phiên làm việc thông qua vòng xử lý giữa mô hình ngôn ngữ và các công cụ của hệ thống. Từ yêu cầu người dùng, trạng thái mục tiêu và ngữ cảnh hiện có, Agent lựa chọn thao tác, tiếp nhận kết quả rồi cập nhật thông tin phục vụ bước tiếp theo. Vòng lặp này kết nối kế hoạch điều phối với việc thực hiện và theo dõi tiến trình của nhiệm vụ.

Trong quá trình xử lý, Decision Planner cung cấp định hướng dựa trên yêu cầu và trạng thái kiểm thử hiện tại. Định hướng này được cập nhật theo tiến trình để Agent lựa chọn bước tiếp theo, trong khi việc thực thi vẫn chịu các điều kiện về quyền, phạm vi và trạng thái quy trình. Hình 2.7 thể hiện cách Agent xử lý phản hồi

của LLM, thực thi công cụ khi cần và quyết định tiếp tục hoặc kết thúc lượt tương tác.



Hình 2.7. Cơ chế điều phối vòng lặp Agent

Trong mỗi bước xử lý, Agent gửi ngữ cảnh hiện tại tới LLM và xử lý phản hồi nhận được. Nếu phản hồi chứa yêu cầu gọi công cụ, Agent chuyển các yêu cầu này tới hệ thống thực thi. Kết quả được ghi nhận và bổ sung vào ngữ cảnh làm việc. Sau đó, Agent kiểm soát vòng lặp để xác định có tiếp tục xử lý hay chuyển sang nhánh kết thúc.

Một lượt tương tác có thể gồm nhiều bước trong Agent Loop. Trong cùng một phản hồi, LLM có thể đề xuất nhiều lời gọi công cụ và các lời gọi này vẫn thuộc cùng một bước xử lý. Tuy nhiên, không phải phản hồi nào của mô hình cũng chứa yêu cầu gọi công cụ.

Khi phản hồi của LLM không chứa yêu cầu gọi công cụ, Agent xem xét điều kiện kết thúc theo chế độ kiểm thử và trạng thái hiện tại. Với chế độ Whole-target hoặc nhiệm vụ Direct đã ghi nhận các mục tiêu kiểm thử, Agent đối chiếu trạng thái workflow trước khi chấp nhận phản hồi cuối. Việc hoàn tất một Candidate không đồng nghĩa với hoàn thành nhiệm vụ nếu vẫn còn mục tiêu có thể xử lý. Những mục tiêu bị chặn, tạm hoãn hoặc chưa có đủ ngữ cảnh được thể hiện là nội dung chưa hoàn thành trong phần tổng hợp.

Đối với nhánh có thực thi công cụ, sau khi tiếp nhận kết quả, Agent đối chiếu trạng thái quy trình để xác định bước xử lý tiếp theo. Agent tiếp tục xử lý khi còn công việc khả thi và chưa đạt điều kiện dừng. Khi quy trình hoàn tất, bị chặn do trở ngại thực tế hoặc đạt giới hạn số bước, Agent chuyển sang xử lý kết thúc phù hợp với trạng thái hiện tại. Vòng lặp không tự dừng chỉ vì nhiều bước liên tiếp chưa tạo thêm kết quả. Các trạng thái chi tiết của quy trình được quản lý bởi mô-đun trạng thái và kết quả kiểm thử, trong khi Agent Loop sử dụng các thông tin này để điều phối quá trình xử lý.

Bên cạnh việc theo dõi tiến độ, Agent duy trì giới hạn số bước cho mỗi lượt tương tác. Khi đạt giới hạn, Agent chuyển sang bước tổng hợp cuối. Ở bước này, mô hình tổng hợp các kết quả đã thu được cùng những nội dung còn chưa hoàn thành và không được cung cấp khả năng gọi thêm công cụ.

Ngoài trường hợp đạt giới hạn số bước, các nhánh kết thúc khác phụ thuộc vào trạng thái xử lý. Nếu được phép kết thúc sau phản hồi không gọi công cụ, Agent đưa ra phản hồi cuối. Khi kiểm soát vòng lặp xác định công việc đã hoàn tất, Agent tổng hợp kết quả. Nếu quy trình bị chặn, Agent nêu lý do chưa thể tiếp tục và những công việc chưa hoàn thành.

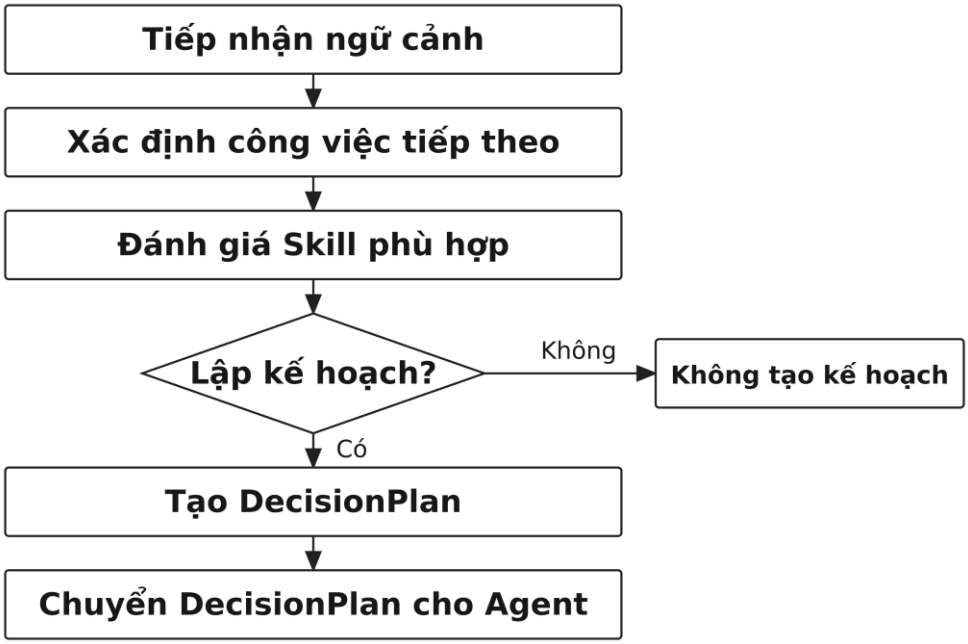
Như vậy, cơ chế điều phối giúp Agent duy trì tiến trình qua nhiều bước xử lý, cập nhật định hướng theo trạng thái hiện tại và phân biệt việc kết thúc một lượt tương tác với việc hoàn tất toàn bộ mục tiêu kiểm thử.

b) Cơ chế lập kế hoạch của Decision Planner

Decision Planner đề xuất công việc tiếp theo cho Agent dựa trên yêu cầu của người dùng và trạng thái kiểm thử hiện tại. Trước các bước suy luận trong vòng lặp, Planner cập nhật định hướng để phù hợp với tiến độ thực hiện. Khi cần đưa ra định hướng, kết quả được thể hiện dưới dạng kế hoạch có cấu trúc gọi là `DecisionPlan`. Agent sử dụng kế hoạch này để lựa chọn bước xử lý, còn các thao tác kiểm thử được thực hiện thông qua hệ thống công cụ.

Để lập kế hoạch, Planner sử dụng yêu cầu của người dùng, thông tin đích kiểm thử (Target), các kỹ năng khả dụng và ngữ cảnh lập kế hoạch (`PlannerContext`). `PlannerContext` tổng hợp thông tin về kỹ năng đang sử dụng hoặc đã hoàn tất, các giai đoạn đã thực hiện, đầu vào chờ phân tích, các điểm nghi vấn đã ghi nhận (Candidate) và kết quả liên quan. Ngữ cảnh này cũng cho biết chế độ kiểm thử, mục tiêu cần đạt, mức độ bao phủ và những trở ngại hiện có.

Từ các thông tin này, Planner xác định công việc cần thực hiện tiếp theo. Quy trình lập kế hoạch được minh họa trong Hình 2.8.



Hình 2.8. Quy trình lập kế hoạch của Decision Planner

Decision Planner sử dụng trạng thái có cấu trúc của quy trình để xác định công việc cần thực hiện tiếp theo. Trong chế độ Whole-target, Planner xem xét các pha đã hoàn thành, đầu vào còn chờ phân tích, thông tin bao phủ và Candidate. Ở giai đoạn đầu, Planner ưu tiên các hoạt động thu thập thông tin, liệt kê bề mặt tấn công và phân tích đầu vào. Planner lựa chọn công việc tiếp theo từ các mục tiêu và Candidate đã ghi nhận theo trạng thái hiện tại. Khi đã hoàn thành thu thập thông tin và liệt kê bề mặt tấn công, Planner có thể đề xuất xác minh Candidate có đầu vào liên kết tương thích đã được phân tích, có dữ liệu phân tích hợp lệ và đáp ứng các điều kiện khác mà hệ thống kiểm tra trước khi bắt đầu xác minh. Các đầu vào độc lập còn chờ xử lý vẫn được giữ để tiếp tục phân tích.

Trong chế độ Direct, Planner đối chiếu các mục tiêu kiểm thử đã ghi nhận cùng Candidate thuộc mục tiêu hiện tại. Nếu chưa có mục tiêu cụ thể, Planner phân tích yêu cầu và siêu dữ liệu của kỹ năng để xác định có đủ cơ sở lập kế hoạch hay không. Planner chỉ đề xuất xử lý trong phạm vi nhiệm vụ đã được yêu cầu, kể cả khi còn thiếu ngữ cảnh. Như vậy, kế hoạch được xây dựng theo chế độ kiểm thử và trạng thái hiện tại, thay vì sử dụng cùng một chuỗi kỹ năng cho mọi yêu cầu.

Ở nhánh phân tích nội dung yêu cầu, Decision Planner đối chiếu yêu cầu với siêu dữ liệu của các kỹ năng được đăng ký để lựa chọn kỹ năng phù hợp. Trước khi

đối chiếu, nội dung văn bản được chuẩn hóa nhằm giảm ảnh hưởng của khác biệt về cách viết, ký tự phân cách và khoảng trắng. Planner đánh giá mức độ phù hợp của kỹ năng dựa trên nhiều loại tín hiệu từ siêu dữ liệu (metadata), kết hợp với trạng thái hiện tại của quy trình. Các tín hiệu này gồm tín hiệu mạnh và yếu trong `triggers`, các lớp đối tượng mà kỹ năng có thể xử lý thông qua `candidate-classes`, pha kiểm thử được mô tả bởi `stage` và thông tin về các kỹ năng liên quan trong `requires`. Thông tin `requires` được dùng để bổ sung cơ sở xếp hạng khi các kỹ năng liên quan đang được sử dụng hoặc đã hoàn tất.

Khi nhiều kỹ năng cùng khớp với yêu cầu, Planner so sánh mức độ liên quan của các tín hiệu và ngữ cảnh hiện tại. Nếu chưa đủ cơ sở để phân biệt các lựa chọn, Planner không thiết lập kỹ năng đề xuất nhằm tránh đưa ra lựa chọn thiếu căn cứ. Tùy ngữ cảnh, Planner vẫn có thể tạo `DecisionPlan` để định hướng xử lý hoặc nêu trở ngại cần giải quyết mà không đề xuất Skill cụ thể.

Đối với Candidate cần kiểm tra, Planner ưu tiên Skill chuyên biệt có ánh xạ duy nhất và khả dụng. Nhánh kiểm tra dự phòng có giới hạn chỉ được xem xét khi không có ánh xạ Skill chuyên biệt và đề xuất phép thử khớp với ngữ cảnh thực hiện đã được kiểm chứng, đồng thời đáp ứng các điều kiện kiểm soát thực thi khác. Nhánh này không được dùng để thay thế Skill đã có hoặc bỏ qua lỗi nạp Skill. Nếu ánh xạ mơ hồ, Skill không khả dụng hoặc điều kiện dự phòng chưa đáp ứng, Planner ghi nhận trở ngại hoặc phần ngữ cảnh còn thiếu. Candidate chưa đủ điều kiện được giữ ở trạng thái chưa thể tiếp tục kiểm tra.

Khi đã có mục tiêu kiểm thử có cấu trúc, Planner ưu tiên xem xét trạng thái của mục tiêu đó. Ở nhánh phân tích nội dung yêu cầu, Planner bỏ qua việc tạo `DecisionPlan` đối với truy vấn được nhận diện là chỉ yêu cầu giải thích hoặc cung cấp thông tin.

Khi có đủ cơ sở lập kế hoạch, Planner tổng hợp các thông tin định hướng thành `DecisionPlan` (Bảng 2.4) và cung cấp cho Agent để tiếp tục xử lý.

Bảng 2.4. Các thành phần của DecisionPlan

Thành phần	Ý nghĩa
`recommended_skill`	Kỹ năng được đề xuất cho lượt xử lý hiện tại, nếu có
`reason`	Cơ sở dẫn đến đề xuất của Planner
`risk`	Mức rủi ro do Planner xác định để bổ sung hướng dẫn kiểm soát
`checklist`	Các nội dung cần lưu ý trong quá trình xử lý
`guidance`	Chỉ dẫn định hướng Agent
`candidate_id`	Định danh Candidate liên quan đến kế hoạch, nếu có

Trong `DecisionPlan`, trường `risk` thể hiện mức rủi ro của yêu cầu. Tùy ngữ cảnh, kế hoạch có thể bổ sung các chỉ dẫn kiểm soát phù hợp. Việc thực thi và kiểm soát quyền đối với các công cụ được xử lý độc lập bởi các thành phần chuyên trách, không thuộc trách nhiệm trực tiếp của Decision Planner.

Nếu kế hoạch liên quan đến một Candidate cụ thể, trường `candidate_id` xác định Candidate tương ứng. Trong `DecisionPlan`, trường `checklist` có thể chứa các nội dung cần lưu ý trong lượt xử lý. Trường `guidance` cung cấp định hướng cho Agent dựa trên ngữ cảnh và trạng thái quy trình.

`DecisionPlan` chỉ mang tính định hướng, không trực tiếp thực thi công cụ hoặc cấp quyền thực hiện hành động. Các bước kiểm thử chi tiết vẫn được cung cấp bởi kỹ năng tương ứng khi được nạp, trong khi Agent Core chịu trách nhiệm duy trì vòng lặp và điều phối quá trình thực thi.

2.5.2. Mô-đun tri thức và kỹ năng kiểm thử

a) Mô-đun quản lý tri thức

Mô-đun quản lý tri thức (Intelligence) cung cấp cho Agent những thông tin và kinh nghiệm đã được lưu trữ để hỗ trợ quá trình phân tích và kiểm thử. Tri thức được tổ chức dưới dạng các scenario có cấu trúc, mô tả những tình huống, dấu hiệu và kinh nghiệm có thể tái sử dụng khi gặp các nhiệm vụ tương tự. Intelligence chỉ cung cấp thông tin tham khảo, không trực tiếp thực hiện thao tác kiểm thử hoặc quyết định hành động thay cho Agent.

Thành phần Intelligence Store chịu trách nhiệm lưu trữ, tìm kiếm, xếp hạng và quản lý các scenario. Thành phần này cũng hỗ trợ chuẩn hóa, hạn chế trùng lặp và tích lũy tri thức mới. Những scenario phù hợp được lựa chọn để bổ sung vào ngữ cảnh làm việc của Agent, trong khi việc lựa chọn và thực hiện thao tác kiểm thử vẫn do Agent điều phối.

Đơn vị dữ liệu cơ bản của mô-đun là `IntelligenceScenario`, biểu diễn một tình huống hoặc kinh nghiệm có thể tái sử dụng. Thay vì chỉ lưu trữ dưới dạng văn bản tự do, mỗi scenario được tổ chức thành các nhóm thông tin để phục vụ việc quản lý và truy hồi. Các nhóm này bao gồm nội dung tri thức, định hướng kiểm thử, thông tin phục vụ truy hồi và xếp hạng, cùng các thông tin quản lý scenario. Chi tiết các nhóm thông tin được trình bày trong Bảng 2.5.

Bảng 2.5. Các nhóm thông tin chính của IntelligenceScenario

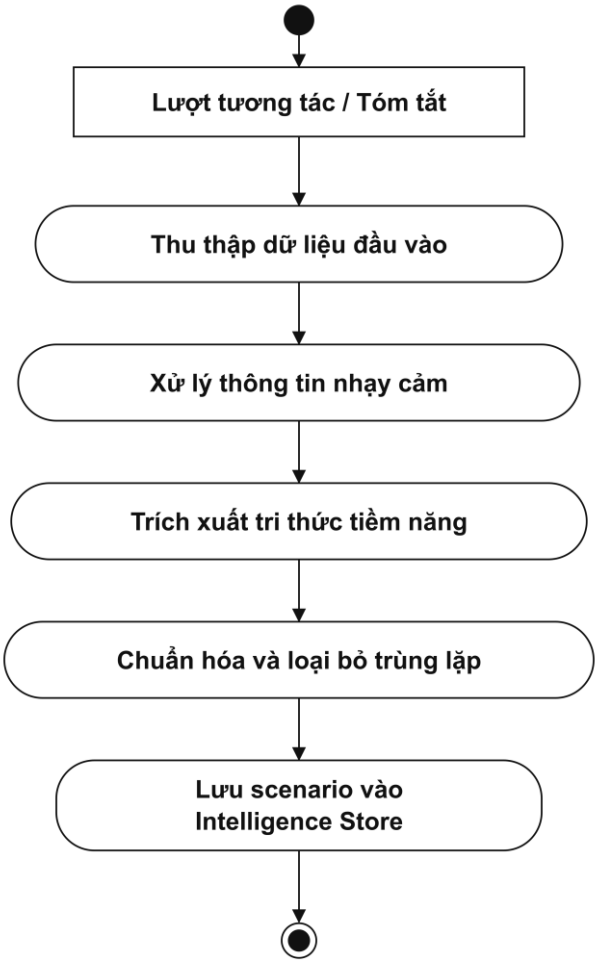
Nhóm thông tin	**Thành phần chính**	**Vai trò**
Nội dung tri thức	`title`, `category`, `lesson`	Mô tả nội dung và bản chất của scenario.
Định hướng kiểm thử	`recommended_checks`, `avoid_missing`	Gợi ý các bước kiểm tra và điểm cần lưu ý.
Truy hồi và xếp hạng	`triggers`, `technologies`, `confidence`	Hỗ trợ tìm kiếm và xếp hạng scenario phù hợp.
Quản lý	`id`, `scope`, `source`	Định danh, phạm vi lưu trữ và nguồn thông tin.

Khi Agent xử lý một nhiệm vụ, KAgent sử dụng thông tin từ lượt tương tác hiện tại để tìm kiếm những scenario có liên quan trong Intelligence Store. Việc truy hồi dựa trên mức độ phù hợp giữa nội dung truy vấn và các trường thông tin của scenario. Trong đó, những trường thể hiện dấu hiệu trực tiếp như `triggers`, `title` và `technologies` được ưu tiên hơn các trường mô tả hoặc định hướng kiểm thử tổng quát. Sau khi tìm kiếm, Intelligence Store xếp hạng các scenario dựa trên mức độ phù hợp, độ tin cậy và độ mới của thông tin.

Hệ thống chỉ lựa chọn một số scenario phù hợp để bổ sung vào Working Context, thay vì đưa toàn bộ kho tri thức vào ngữ cảnh của mô hình. Trước khi bổ sung vào ngữ cảnh, hệ thống còn kiểm tra nguồn gốc và phạm vi áp dụng của scenario. Vì vậy, không phải mọi thông tin đã lưu đều được tự động sử dụng trong các phiên tiếp theo. Cách làm này giúp Agent tiếp cận thông tin liên quan mà không làm tăng kích thước ngữ cảnh quá mức. Tri thức được truy hồi chỉ hỗ trợ quá trình phân tích, không thay thế kết quả quan sát hoặc bằng chứng thu được từ công cụ kiểm thử.

Bên cạnh việc sử dụng tri thức đã lưu trữ, KAgent còn có thể tích lũy các scenario mới từ dữ liệu hình thành trong quá trình hoạt động. Cơ chế này được kích

hoạt khi lượt xử lý có thực thi công cụ hoặc khi hệ thống thực hiện thu gọn lịch sử phiên, thay vì sau mọi lượt tương tác. Quá trình tích lũy tri thức được minh họa trong Hình 2.9.



Hình 2.9. Cơ chế học và tích lũy tri thức

Trước khi trích xuất tri thức, dữ liệu phiên được xử lý nhằm hạn chế lưu trữ thông tin nhạy cảm. Hệ thống sau đó sử dụng các quy tắc và mẫu nhận diện đã xác định để tìm ra những thông tin có khả năng tái sử dụng. Những nội dung liên quan đến quyền hạn hoặc phạm vi tạm thời của phiên được loại trừ để tránh lưu giữ các điều kiện chỉ có hiệu lực trong phiên hiện tại.

Các scenario được trích xuất tiếp tục được chuẩn hóa và kiểm tra trùng lặp trước khi lưu trữ. Việc kiểm tra dựa trên định danh và thông tin mô tả đã được chuẩn hóa nhằm hạn chế lưu nhiều scenario có cùng nội dung. Các scenario đã qua xử lý được lưu trữ và quản lý thông qua Intelligence Store để phục vụ truy hồi khi phù hợp. Tri thức tích lũy tự động chỉ mang tính tham khảo, không tự cấp phạm vi, quyền kiểm thử hoặc phê duyệt cho các phiên làm việc sau.

b) Mô-đun kỹ năng kiểm thử

Bên cạnh mô-đun quản lý tri thức, KAgent sử dụng Skills System để quản lý các quy trình kiểm thử chuyên biệt. Mỗi kỹ năng được tổ chức dưới dạng tài liệu hướng dẫn (playbook), mô tả phương pháp và các bước khuyến nghị cho một loại nhiệm vụ hoặc tình huống kiểm thử cụ thể. Kỹ năng giúp Agent lựa chọn cách tiếp cận và công cụ phù hợp, nhưng không trực tiếp thực hiện thao tác kiểm thử hoặc quyết định hành động thay cho Agent.

Skills System chịu trách nhiệm phát hiện, đăng ký, tra cứu và nạp kỹ năng. Thành phần này cung cấp danh mục kỹ năng cho Decision Planner để phục vụ việc lập kế hoạch, đồng thời đưa nội dung kỹ năng đã nạp vào Working Context để Agent tham khảo.

Decision Planner có thể đề xuất kỹ năng phù hợp thông qua `DecisionPlan`, nhưng việc đề xuất không có nghĩa kỹ năng đó đã được nạp. Skills System vẫn phải kiểm tra các điều kiện sử dụng trước khi nạp kỹ năng. Việc tách biệt hai hoạt động này giúp hệ thống có thể bổ sung hoặc cập nhật kỹ năng mà không cần thay đổi cơ chế điều phối của Agent Core.

Mỗi kỹ năng được tổ chức thành một thư mục riêng, trong đó `SKILL.md` là tệp mô tả chính. Tệp này gồm phần YAML frontmatter dùng để khai báo thông tin cấu hình và phần thân Markdown chứa nội dung hướng dẫn kiểm thử. Thư mục kỹ năng có thể chứa tài nguyên bổ trợ như danh sách payload hoặc script hỗ trợ.

Phần YAML frontmatter cung cấp các thuộc tính giúp hệ thống nhận diện, lựa chọn và quản lý kỹ năng. Những thuộc tính chính được chia thành bốn nhóm như trình bày trong Bảng 2.6.

Bảng 2.6. Các nhóm metadata chính của kỹ năng kiểm thử

Nhóm metadata	Các trường thuộc tính	Ý nghĩa thiết kế
Định danh và giai đoạn	`name`, `description`, `stage`	Định danh kỹ năng, mô tả mục đích sử dụng và xác định giai đoạn kiểm thử tương ứng
Điều phối và tín hiệu	`triggers`, `candidate-classes`, `requires`	Khai báo các tín hiệu nhận diện, lớp đối tượng kiểm thử và quan hệ phụ thuộc phục vụ quá trình đánh giá, đề xuất và điều phối kỹ năng
Kiểm soát thực thi	`allowed-tools`, `disable-model-invocation`	Xác định phạm vi công cụ được phép sử dụng và cơ chế nạp kỹ năng từ mô hình

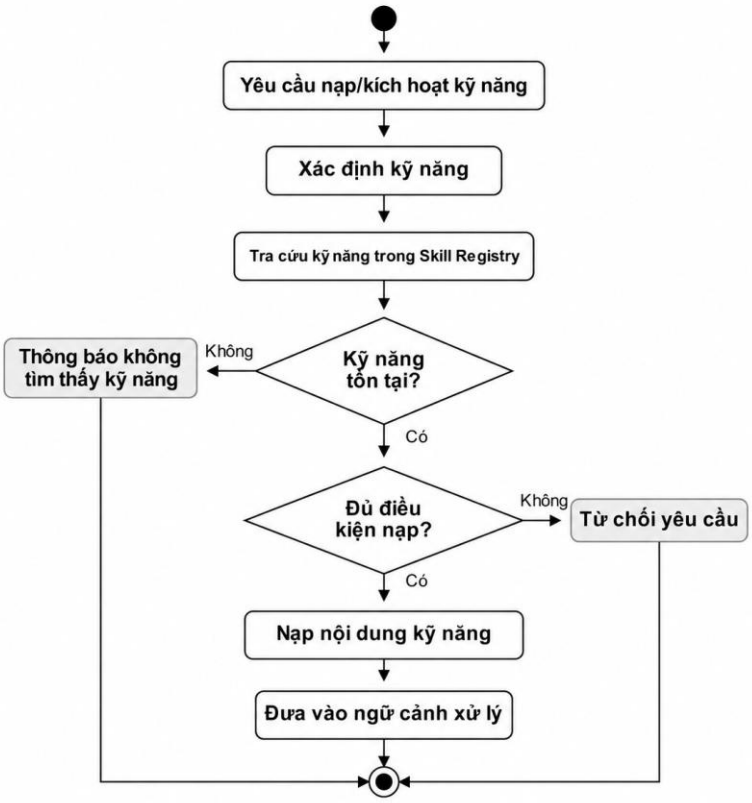
Nhóm metadata	Các trường thuộc tính	Ý nghĩa thiết kế
Hoàn tất kỹ năng	`completion-artifact`	Mẫu đường dẫn artifact bắt buộc phải có để xác nhận kỹ năng hoàn tất, áp dụng đối với các kỹ năng có yêu cầu artifact

Trong đó, `requires` mô tả quan hệ phụ thuộc để hỗ trợ lập kế hoạch và điều phối, nhưng không phải điều kiện bắt buộc để cho phép nạp kỹ năng. Thuộc tính `completion-artifact` xác định artifact cần có trước khi hệ thống ghi nhận một kỹ năng đã hoàn tất. Phần thân Markdown của `SKILL.md` trình bày mục tiêu, phạm vi, các bước thực hiện và cách xác nhận kết quả kiểm thử. Khi kỹ năng được nạp, nội dung hướng dẫn này được bổ sung vào ngữ cảnh để Agent sử dụng trong quá trình xử lý nhiệm vụ.

Khi khởi động, Skills System thực hiện phát hiện và kiểm tra các kỹ năng để xác định những kỹ năng có thể sử dụng trong hệ thống. Những kỹ năng đáp ứng yêu cầu về cấu trúc và thông tin khai báo được đăng ký vào Skill Registry, tạo thành danh mục quản lý tập trung. Thông qua danh mục này, hệ thống có thể tra cứu các kỹ năng khả dụng để phục vụ quá trình lập kế hoạch, lựa chọn và nạp kỹ năng khi cần thiết.

Để tránh đưa quá nhiều nội dung vào ngữ cảnh, KAgent sử dụng cơ chế nạp động (Dynamic loading). Theo mặc định, System Prompt chỉ cung cấp thông tin tóm tắt gồm tên và mô tả của các kỹ năng khả dụng. Nội dung hướng dẫn đầy đủ chỉ được bổ sung vào Working Context khi kỹ năng tương ứng được nạp hoặc kích hoạt.

Kỹ năng có thể được kích hoạt theo hai cách. Khi người dùng chủ động kích hoạt thông qua lệnh giao diện, kỹ năng được ghi nhận để áp dụng từ lượt tương tác tiếp theo. Khi mô hình gọi công cụ `load_skill`, kỹ năng được nạp và có thể sử dụng ngay trong lượt xử lý hiện tại. Quy trình lựa chọn và nạp kỹ năng được thể hiện trong Hình 2.10.



Hình 2.10. Quy trình lựa chọn và nạp kỹ năng

Trước khi nạp kỹ năng, hệ thống kiểm tra kỹ năng có tồn tại trong Skill Registry và có được phép sử dụng hay không. Kỹ năng bị vô hiệu hóa sẽ không được nạp. Nếu yêu cầu nạp xuất phát từ mô hình thông qua công cụ `load_skill`, hệ thống còn kiểm tra thuộc tính `disable-model-invocation`. Khi thuộc tính này được thiết lập, mô hình không được tự nạp kỹ năng mà chỉ người dùng mới có thể chủ động kích hoạt. Việc kiểm tra tại thời điểm nạp độc lập với đề xuất của Decision Planner.

Ngoài nội dung hướng dẫn, mỗi kỹ năng có thể khai báo giới hạn công cụ thông qua thuộc tính `allowed-tools`. Giới hạn này áp dụng với những thao tác cần cấp quyền. Việc xác định một thao tác có cần quyền hay không phụ thuộc vào loại công cụ, tham số và ngữ cảnh thực hiện. Khi điều kiện kiểm soát được áp dụng, hệ thống đối chiếu công cụ với các giới hạn do kỹ năng đang hoạt động khai báo. Những thao tác không yêu cầu cấp quyền không chịu ràng buộc `allowed-tools` này.

Khi nhiều kỹ năng cùng hoạt động, một công cụ đáp ứng giới hạn theo kỹ năng nếu có ít nhất một kỹ năng cho phép sử dụng hoặc có kỹ năng không khai báo giới hạn công cụ. Tuy nhiên, việc đáp ứng điều kiện `allowed-tools` không đồng nghĩa với việc công cụ được phép thực thi ngay. Các cơ chế kiểm tra quyền, phạm

vi và an toàn khác vẫn được thực hiện độc lập trước khi thao tác tác động lên mục tiêu.

Việc hoàn tất kỹ năng cũng phải đáp ứng các điều kiện tương ứng. Với kỹ năng yêu cầu artifact, hệ thống kiểm tra tệp tại đường dẫn được khai báo trong `completion-artifact` trước khi ghi nhận hoàn tất. Đối với các pha khảo sát trong chế độ Whole-target, hệ thống còn đối chiếu phạm vi, thông tin bao phủ và trạng thái đầu vào liên quan. Như vậy, việc hoàn tất pha không chỉ phụ thuộc vào sự tồn tại của tệp kết quả mà còn phải phù hợp với trạng thái và các điều kiện của quy trình kiểm thử. Đối với kỹ năng xác minh lỗ hổng, việc xác định hoàn tất còn dựa trên kết quả kiểm thử đã ghi nhận và các điều kiện về bằng chứng, thay vì chỉ căn cứ vào tệp artifact.

2.5.3. Mô-đun quản lý và thực thi công cụ

Mô-đun quản lý và thực thi công cụ kết nối các yêu cầu do mô hình ngôn ngữ đề xuất với những thao tác cụ thể trong môi trường kiểm thử. Agent Core điều phối việc sử dụng công cụ dựa trên nhiệm vụ và trạng thái hiện tại, Tool Registry quản lý các công cụ đã đăng ký, còn từng công cụ đảm nhiệm chức năng tương ứng. Cách tổ chức này tách biệt việc lựa chọn hành động, quản lý giao diện công cụ và thực hiện thao tác, đồng thời tạo điều kiện áp dụng các cơ chế kiểm tra tham số, kiểm soát quyền và giới hạn thực thi phù hợp.

a) Quản lý và mô tả công cụ

KAgent sử dụng giao diện chung Tool để thống nhất cách mô tả và tiếp nhận yêu cầu thực thi đối với các công cụ có chức năng khác nhau. Thông qua giao diện này, mỗi công cụ cung cấp tên định danh, mô tả chức năng, lược đồ tham số, yêu cầu phê duyệt mặc định và thành phần thực thi. Nhờ đó, hệ thống có thể quản lý công cụ theo một cấu trúc thống nhất mà không cần đưa logic xử lý riêng của từng thao tác vào Agent Core. Các thành phần chính của giao diện công cụ được trình bày trong Bảng 2.7.

Bảng 2.7. Các thành phần chính của công cụ

Thành phần	Ý nghĩa
Tên định danh (`name`)	Xác định tên dùng để gọi và tra cứu công cụ trong hệ thống.
Mô tả chức năng (`description`)	Cung cấp thông tin về chức năng và mục đích sử dụng của công cụ.

Thành phần	Ý nghĩa
Lược đồ tham số (`schema`)	Mô tả cấu trúc và các ràng buộc của tham số đầu vào.
Yêu cầu phê duyệt mặc định (`requires_permission`)	Khai báo yêu cầu phê duyệt mặc định trước khi công cụ thực hiện thao tác.
Thành phần thực thi (`run`)	Thực hiện chức năng của công cụ trên cơ sở dữ liệu đầu vào và các điều kiện thực thi tương ứng.

Bên cạnh giao diện chung, một số công cụ cung cấp thêm cơ chế kiểm tra tham số, xác định yêu cầu phê duyệt theo hành động và mô tả thao tác để người kiểm thử xem xét. Chẳng hạn, các hành động truy vấn và thay đổi dữ liệu trong cùng một công cụ có thể cần điều kiện kiểm soát khác nhau. Vì vậy, yêu cầu phê duyệt mặc định chỉ là một phần của cơ chế kiểm soát. Việc cho phép thực hiện một thao tác cụ thể còn phụ thuộc vào tham số, phạm vi tài nguyên và chính sách đang có hiệu lực.

Tool Registry lưu các công cụ đã đăng ký và sử dụng tên định danh để tra cứu công cụ tương ứng. Registry cung cấp chức năng đăng ký, tra cứu và điểm truy cập thống nhất cho việc thực thi. Việc duy trì danh mục tập trung giúp Agent Core sử dụng công cụ thông qua giao diện đã thống nhất, trong khi cách thực hiện từng chức năng vẫn được quản lý tại công cụ đó. Các giới hạn sử dụng theo kỹ năng và việc điều phối nhiều yêu cầu thuộc trách nhiệm của Agent Core.

Từ những công cụ đã đăng ký, Tool Registry tạo các cấu trúc mô tả `ToolSpec` để cung cấp cho mô hình ngôn ngữ. Mỗi `ToolSpec` chứa tên, mô tả chức năng và lược đồ tham số đầu vào, giúp mô hình nhận biết công cụ và hình thành yêu cầu gọi phù hợp. Phần mô tả này được tách khỏi logic thực thi và không mang ý nghĩa cấp quyền cho hành động do mô hình đề xuất. Khi kiểm tra công cụ, hệ thống quy đổi một số tên gọi tương đương về tên định danh thống nhất, giúp các điều kiện kiểm soát được áp dụng nhất quán.

Theo mục đích sử dụng, các công cụ được tổ chức thành những nhóm chức năng như trình bày trong Bảng 2.8. Cách phân nhóm này thể hiện phạm vi trách nhiệm của các công cụ trong kiến trúc, thay vì mô tả một trình tự kiểm thử cố định.

Bảng 2.8. Phân nhóm công cụ theo mục đích sử dụng

Nhóm công cụ	Chức năng chính
Thao tác hệ thống và tệp tin	Thực hiện lệnh cục bộ, đọc, ghi, chỉnh sửa và tìm kiếm dữ liệu phục vụ kiểm thử.
Tương tác mạng và web	Gửi yêu cầu tới ứng dụng mục tiêu, khảo sát dịch vụ, liệt kê tài nguyên và thu thập dữ liệu web hỗ trợ phân tích.
Trình duyệt và thu thập lưu lượng	Tiếp nhận, tra cứu và truy xuất lưu lượng từ trình duyệt hoặc công cụ trung gian trong phạm vi được kiểm soát. Nhóm này không mặc nhiên cung cấp khả năng điều khiển trình duyệt.
Quản lý kỹ năng và tài nguyên	Nạp kỹ năng kiểm thử và truy xuất các tài nguyên hỗ trợ thực hiện nhiệm vụ.
Quản lý trạng thái quy trình	Ghi nhận, cập nhật và truy vấn đầu vào, đối tượng nghi vấn, kết quả xác minh và thông tin bao phủ kiểm thử.
Xác nhận và ghi nhận phát hiện	Kiểm tra điều kiện của kết quả xác minh và bằng chứng liên quan trước khi ghi nhận phát hiện.
Tương tác người dùng và trạng thái quyền	Trao đổi với người kiểm thử khi cần bổ sung thông tin và cung cấp thông tin về trạng thái quyền thực thi.

Các nhóm trên phối hợp để hỗ trợ cả hoạt động tương tác với môi trường kiểm thử và quản lý thông tin trong hệ thống. Kiến trúc giao diện chung cho phép bổ sung công cụ mới thông qua cơ chế đăng ký và điều phối thống nhất, đồng thời duy trì sự phân tách giữa chức năng của từng công cụ và trách nhiệm quản lý của hệ thống.

b) Thực thi và kiểm soát công cụ

Việc kiểm soát công cụ được phân chia giữa Agent Core, Tool Registry, cơ chế quản lý quyền và từng công cụ. Đối với những thao tác chịu ràng buộc `allowed-tools` của kỹ năng, Agent Core đối chiếu yêu cầu với giới hạn đang áp dụng. Tool Registry tiếp nhận yêu cầu phù hợp, tra cứu công cụ và áp dụng bước kiểm tra tham số do công cụ cung cấp trước khi chuyển tới phần thực thi.

Lược đồ tham số và bước kiểm tra tại thời điểm thực thi có vai trò bổ sung cho nhau. Lược đồ giúp mô hình tạo yêu cầu theo cấu trúc đã công bố, còn Registry và công cụ kiểm tra dữ liệu thực tế nhận được để xác định yêu cầu có đáp ứng các điều kiện thực thi hay không. Tùy chức năng, công cụ có thể kiểm tra thêm đường dẫn tài nguyên, ngữ cảnh yêu cầu hoặc các điều kiện riêng của thao tác. Cách tổ chức này giúp duy trì giao diện chung nhưng vẫn cho phép áp dụng những kiểm tra phù hợp với từng loại công cụ.

Đối với hành động cần phê duyệt, cơ chế quản lý quyền cung cấp thông tin mô tả thao tác để người kiểm thử xem xét thông qua lớp giao diện. Quyết định cho phép được áp dụng theo phạm vi và điều kiện của yêu cầu đang được xét. Trước khi thực thi, hệ thống kiểm tra quyết định cho phép có còn hiệu lực và phù hợp với yêu cầu hay không. Khi bị từ chối hoặc không đáp ứng điều kiện, yêu cầu không được tiếp tục và thông tin tương ứng được trả về cho Agent. Chế độ YOLO có thể tự xử lý bước phê duyệt theo chính sách, nhưng không loại bỏ các kiểm tra về quyền hiện hành, phạm vi và giới hạn thực thi.

Các biện pháp kiểm soát cụ thể phụ thuộc vào đặc điểm của công cụ. Công cụ HTTP tích hợp kiểm tra yêu cầu theo phạm vi mục tiêu và quyền tương ứng trước khi tương tác với ứng dụng. Công cụ thao tác tệp áp dụng các điều kiện liên quan đến tài nguyên cục bộ và dữ liệu nhạy cảm. Đối với công cụ shell, lệnh được chạy trực tiếp dưới quyền của tài khoản vận hành, kèm theo kiểm soát thời gian chạy, hủy tác vụ và lượng đầu ra thu nhận. Những giới hạn này giúp quản lý quá trình thực thi nhưng không tạo thành cơ chế cô lập ở cấp hệ điều hành.

Kết quả công cụ cung cấp dữ liệu quan sát hoặc thông tin lỗi để Agent đánh giá và lựa chọn bước xử lý tiếp theo. Việc giới hạn dung lượng, xử lý thông tin nhạy cảm và lưu kết quả dài để đọc lại tuân theo cơ chế quản lý ngữ cảnh tại Mục 2.4.2.

Dữ liệu được lưu để phục vụ hội thoại không tự trở thành bằng chứng đã được chấp nhận. Khi sử dụng kết quả kiểm thử làm cơ sở ghi nhận phát hiện, hệ thống còn áp dụng cơ chế đăng ký bằng chứng và kiểm tra điều kiện của workflow. Mô-đun thực thi công cụ cung cấp dữ liệu quan sát, còn các thành phần quản lý trạng thái và kết quả kiểm thử đảm nhiệm việc đăng ký bằng chứng và kiểm tra điều kiện ghi nhận phát hiện.

2.5.4. Mô-đun quản lý phiên và bộ nhớ

Mô-đun quản lý phiên và bộ nhớ duy trì tính liên tục của quá trình kiểm thử qua các lượt tương tác và khi người kiểm thử tiếp tục một phiên đã lưu. Các dữ liệu cần thiết để theo dõi nhiệm vụ được quản lý trong trạng thái phiên, còn thông tin có giá trị tái sử dụng giữa nhiều phiên được lưu trong bộ nhớ bền vững. Sự phân chia này giúp hệ thống vừa bảo toàn tiến độ công việc, vừa lựa chọn được thông tin liên quan mà không phải đưa toàn bộ dữ liệu đã tích lũy vào ngữ cảnh của mô hình.

a) Mô-đun quản lý phiên

Session Store cung cấp cơ chế lưu và nạp dữ liệu phiên cho Agent Core. Thành phần này duy trì lịch sử tương tác, thông tin mục tiêu, khai báo phạm vi, bộ nhớ

phiên và trạng thái quy trình, nhưng không trực tiếp lập kế hoạch hoặc lựa chọn hành động kiểm thử. Agent Core sử dụng dữ liệu được nạp để xây dựng lại ngữ cảnh làm việc và đánh giá công việc cần tiếp tục theo các điều kiện hiện hành.

Dữ liệu phiên được lưu theo cấu trúc `SessionFile`. Trong cấu trúc này, lịch sử hội thoại, thông tin tóm lược và trạng thái workflow được tổ chức thành các thành phần riêng, vì mỗi loại dữ liệu có vai trò khác nhau đối với việc tiếp tục nhiệm vụ. Các thành phần chính của `SessionFile` được trình bày trong Bảng 2.9.

Bảng 2.9. Các thành phần chính của SessionFile

Thành phần	Vai trò
`id`	Định danh phiên làm việc.
`target`	Lưu thông tin về mục tiêu kiểm thử.
`engagement_state`	Lưu khai báo phạm vi kiểm thử của phiên, gồm các origin HTTP và phạm vi TCP liên quan. Không khôi phục quyền thực thi tạm thời.
`workflow`	Lưu trạng thái có cấu trúc của quy trình kiểm thử.
`memory`	Lưu `SessionMemory` để duy trì thông tin tóm lược giữa các lượt xử lý.
`messages`	Duy trì lịch sử tương tác. Những kết quả được lưu riêng có bản xem trước và tham chiếu, không tự nạp lại toàn bộ nội dung khi khôi phục.
`updated_at`	Ghi nhận thời điểm dữ liệu phiên được cập nhật.

Lịch sử hội thoại giúp duy trì mạch tương tác, trong khi workflow bảo toàn các bản ghi có cấu trúc về đầu vào, Candidate, lượt xác minh và kết quả liên quan. `EngagementState` lưu khai báo phạm vi của phiên, bao gồm các origin HTTP và phạm vi TCP khi có. Một origin HTTP được xác định bởi giao thức, tên máy chủ và cổng. Khai báo phạm vi này không biểu diễn toàn bộ chính sách tác nghiệp hoặc các quyền thực thi tạm thời. Các bản ghi workflow được lưu riêng nên việc thu gọn hội thoại không thay thế chúng bằng bản tóm lược.

Khi phiên kéo dài, `SessionMemory` lưu phần thông tin tóm lược có ích cho việc tiếp tục xử lý sau thu gọn. Cấu trúc này ghi nhận định hướng nhiệm vụ, tiến độ, công việc còn lại và những ghi chú kỹ thuật cần thiết, thay vì sao chép toàn bộ chuỗi hội thoại. Các nhóm thông tin chính của `SessionMemory` được trình bày trong Bảng 2.10.

Bảng 2.10. Các nhóm thông tin chính của SessionMemory

Nhóm thông tin	Các trường dữ liệu	Vai trò khi tiếp tục nhiệm vụ
Định hướng và công việc còn lại	`objectives`, `plan`, `todos`	Duy trì mục tiêu, kế hoạch và những công việc còn cần xử lý sau khi lịch sử được thu gọn.
Tiến độ và bề mặt đã kiểm tra	`completed`, `tested`	Tóm lược công việc đã hoàn thành và các đối tượng đã kiểm tra để hỗ trợ tiếp tục nhiệm vụ.
Ghi nhận kỹ thuật	`findings`, `files`, `commands`	Lưu ghi chú về kết quả đáng chú ý, tệp và lệnh trong phiên. Các ghi chú `findings` không thay thế Finding đã được chấp nhận.
Thông tin xác thực	`credentials`	Lưu ghi chú đã xử lý dữ liệu nhạy cảm để tham khảo, không thay thế ngữ cảnh xác thực hiện hành của công cụ.
Quản lý thu gọn	`compactions`, `last_compacted_at`, `last_summary`	Theo dõi số lần thu gọn, thời điểm và nội dung tóm lược gần nhất.

Trong đó, `todos` bổ sung danh sách công việc còn phải xử lý để Agent tiếp tục nhiệm vụ sau khi lịch sử cũ được thay bằng bản tóm lược. Các trường `completed`, `tested` và `findings` phục vụ ghi nhớ nội dung đã diễn ra, nhưng không thay thế trạng thái kiểm thử có cấu trúc. Đặc biệt, ghi chú `findings` trong `SessionMemory` không đồng nghĩa với Finding đã đáp ứng điều kiện xác nhận và lưu báo cáo. Thông tin xác thực trong bộ nhớ cũng chỉ có giá trị tham khảo sau khi xử lý dữ liệu nhạy cảm, không tự khôi phục trạng thái đăng nhập hoặc quyền của công cụ.

Quá trình thu gọn do Agent Core điều phối. Chỉ khi kết quả đạt yêu cầu giảm kích thước ngữ cảnh, phần tóm lược mới được chấp nhận để cập nhật `SessionMemory` và thay thế lịch sử cũ bằng nội dung cần thiết cùng phần tương tác gần nhất. Những thông tin về mục tiêu, phạm vi và workflow tiếp tục được duy trì riêng. Session Store lưu các thành phần này cùng lịch sử hiện có để việc tiếp tục phiên dựa trên dữ liệu đã được bảo toàn.

Khi khôi phục phiên, hệ thống nạp dữ liệu đã lưu và xây dựng lại chỉ dẫn cho mô hình. Việc khôi phục không tiếp tục một tiến trình công cụ đang chạy hoặc tự tái lập đầy đủ môi trường thực thi trước đó. Các lượt xác minh chưa kết thúc được ghi

nhận là gián đoạn. Hệ thống không tự khôi phục trạng thái kích hoạt kỹ năng hoặc các quyền thực thi tạm thời, nên các thao tác tiếp theo vẫn phải được kiểm tra quyền. Khi bắt đầu một lượt xác minh mới, hệ thống sử dụng định danh riêng thay vì dùng lại định danh của lượt đã gián đoạn.

Agent có thể dùng các tham chiếu trong lịch sử và workflow để xem lại dữ liệu đã thu được. Agent chỉ có thể đọc lại khi dữ liệu vẫn còn, tính toàn vẹn được bảo đảm và các điều kiện truy cập tại thời điểm đọc được đáp ứng. Hệ thống không tự nạp toàn bộ nội dung của các kết quả công cụ đã lưu riêng vào ngữ cảnh khi mở lại phiên. Vì vậy, khôi phục phiên cần được hiểu là tiếp tục từ trạng thái thông tin đã lưu, trong khi các điều kiện cho hành động tiếp theo vẫn được đánh giá tại thời điểm thực hiện.

b) Mô-đun quản lý bộ nhớ

Memory Store quản lý những thông tin bền vững có thể tái sử dụng giữa nhiều phiên làm việc. Vai trò này khác với `SessionMemory`, vốn duy trì bản tóm lược để tiếp tục một phiên cụ thể. Memory Store cũng được quản lý riêng với Intelligence Store. Intelligence tổ chức tri thức thành các scenario phục vụ phân tích và kiểm thử, còn bộ nhớ bền vững lưu từng thông tin kỹ thuật, tham chiếu hoặc ghi chú để có thể sử dụng lại khi cần.

Đơn vị dữ liệu cơ bản của kho bộ nhớ là `MemoryFact`. Mỗi `MemoryFact` biểu diễn một thông tin có nội dung, mô tả và phạm vi lưu trữ riêng để có thể quản lý và truy hồi khi cần. Các thuộc tính chính được trình bày trong Bảng 2.11.

Bảng 2.11. Các thuộc tính chính của cấu trúc MemoryFact

Thuộc tính	Ý nghĩa
`name`	Tên định danh của thông tin.
`description`	Mô tả ngắn giúp nhận biết và hỗ trợ truy hồi.
`type`	Phân loại nội dung của thông tin.
`scope`	Xác định phạm vi lưu trữ của bản ghi bộ nhớ.
`text`	Nội dung chi tiết được lưu trữ.
`created_at`	Thời điểm thông tin được tạo.

Các bản ghi `MemoryFact` được quản lý theo phạm vi lưu trữ tương ứng. Bộ nhớ không được tổ chức thành kho riêng cho từng Target, do đó thông tin đã lưu không mặc nhiên phù hợp khi chuyển sang mục tiêu kiểm thử khác. Khi được truy hồi, các bản ghi cần được xem xét theo ngữ cảnh hiện tại và chỉ đóng vai trò hỗ trợ

quá trình phân tích, không thay thế bằng chứng kiểm thử hoặc cơ chế kiểm soát quyền thực thi.

Khi bổ sung thông tin vào bộ nhớ, hệ thống xử lý dữ liệu nhạy cảm và tổ chức nội dung thành các bản ghi có thể truy xuất độc lập. Hệ thống nhận diện và loại khỏi bộ nhớ bền vững những thông tin về phạm vi kiểm thử được cho phép, các quyết định phê duyệt một lần và trạng thái YOLO của phiên làm việc. Cơ chế này giúp hạn chế việc sử dụng lại những quyền và điều kiện chỉ có hiệu lực trong một phiên cho các nhiệm vụ hoặc mục tiêu khác.

Đối với truy hồi, Agent sử dụng yêu cầu hiện tại kết hợp với thông tin mục tiêu để hình thành truy vấn. Memory Store đối chiếu các từ khóa với tên, mô tả, loại và nội dung của `MemoryFact`, sau đó xếp hạng theo mức độ phù hợp có xét độ mới của thông tin. Chỉ một số bản ghi phù hợp được lựa chọn và lượng nội dung bổ sung vào ngữ cảnh được giới hạn. Cách tổ chức này giúp cung cấp thông tin cần thiết cho nhiệm vụ mà không phải chuyển toàn bộ kho bộ nhớ tới mô hình.

Agent sử dụng thông tin truy hồi từ bộ nhớ để tham khảo khi xử lý nhiệm vụ. Nội dung đã cũ hoặc không còn phù hợp với môi trường hiện tại có thể cần được kiểm tra lại. Thông tin truy hồi không thay thế bằng chứng kiểm thử, kết quả workflow hoặc quyết định của người kiểm thử. Các quyền thực thi và phạm vi được cho phép vẫn phải được xác định trong ngữ cảnh phiên hiện hành.

2.5.5. Mô-đun trạng thái và kết quả kiểm thử

Mô-đun trạng thái và kết quả kiểm thử quản lý tiến trình, bằng chứng và các phát hiện hình thành trong quá trình đánh giá. `WorkflowState` theo dõi công việc và kết quả xác minh, Coverage tổng hợp thông tin bao phủ, còn những phát hiện đáp ứng điều kiện được lưu thành báo cáo Finding.

a) Quản lý trạng thái quy trình kiểm thử

`WorkflowState` quản lý thông tin về những đối tượng cần kiểm tra, công việc đang chờ và kết quả đã thu được. Workflow Tool cung cấp các thao tác tra cứu, cập nhật để Agent và Decision Planner theo dõi nhiệm vụ và lựa chọn bước tiếp theo. Trạng thái này được lưu cùng phiên làm việc. Các thành phần dữ liệu chính được trình bày trong Bảng 2.12.

Bảng 2.12. Các thành phần dữ liệu chính của WorkflowState

Thành phần	Vai trò
`attack_surface_inputs`	Quản lý điểm đầu vào của bề mặt ứng dụng và trạng thái xử lý tương ứng.
`candidates`	Quản lý các đối tượng nghi vấn cần được kiểm tra và xác minh.
`attempts`	Theo dõi các lượt xác minh và thông tin định danh gắn với Candidate, mục tiêu và ngữ cảnh phiên.
`evidence`	Quản lý các `EvidenceArtifact` đã đăng ký và tham chiếu tới bản lưu bằng chứng.
`validation_results`	Lưu đánh giá có cấu trúc cùng liên kết tới Candidate, lượt xác minh và nguồn bằng chứng.
`phase_coverage`, `phase_completions`	Theo dõi nội dung đã xử lý trong các pha khảo sát và lưu các bản ghi hoàn thành pha.
`objective`	Duy trì mục tiêu workflow và các mục tiêu kiểm thử cần xử lý trong yêu cầu.
`persisted_findings`	Ghi nhận liên kết giữa Finding đã lưu và kết quả xác minh tương ứng.

Candidate biểu diễn một dấu hiệu nghi vấn hoặc giả thuyết về lỗ hổng cần kiểm chứng. Một Candidate có thể được kiểm tra nhiều lần. Mỗi lần được ghi nhận thành một lượt xác minh (Validation Attempt), có định danh riêng và gắn với mục tiêu, phiên làm việc tương ứng.

Quá trình thực thi công cụ hoặc tiếp nhận lưu lượng HTTP tạo ra dữ liệu quan sát (`Observation`). Những dữ liệu này được quản lý riêng với `WorkflowState`. Bản ghi lưu nội dung, nguồn thu thập, ngữ cảnh yêu cầu và trạng thái thực thi, nhưng không mặc nhiên chứa toàn bộ phản hồi.

Từ các quan sát phù hợp, Agent lập tệp bằng chứng và đăng ký với workflow. Khi đăng ký thành công, hệ thống tạo bản lưu bất biến và quản lý bằng chứng qua `EvidenceArtifact`, có liên kết với Candidate cùng nguồn quan sát. Tệp chưa đăng ký chỉ là tệp cục bộ. Việc đăng ký bằng chứng chưa đồng nghĩa với xác nhận lỗ hổng. Bằng chứng văn bản được xử lý để hạn chế lưu thông tin nhạy cảm.

Trước khi chấp nhận kết quả, KAgent kiểm tra nguồn gốc và tính toàn vẹn của bằng chứng. Hệ thống đối chiếu bằng chứng với Candidate, lượt xác minh, mục tiêu và phiên làm việc để tránh sử dụng nhầm dữ liệu. Các nhận định trong kết quả

phải được gắn với dữ liệu quan sát dùng làm căn cứ. Quan sát khảo sát dùng để hình thành Candidate không tự trở thành bằng chứng kết luận của một lượt xác minh khác.

Sau lượt kiểm tra, Agent đánh giá dữ liệu thu được và ghi nhận kết quả qua Workflow Tool dưới dạng `ValidationResult`. Kết quả `confirmed` thể hiện Agent đánh giá đã xác nhận giả thuyết. Kết quả `not-confirmed` được dùng khi phép kiểm tra có giới hạn đã hoàn tất nhưng chưa xác nhận được. Cả hai kết quả đều đưa Candidate sang trạng thái `validated`, nên trạng thái này phản ánh tiến độ xử lý, không khẳng định có lỗ hổng. Nếu kết quả được ghi nhận là chưa thể kết luận, chẳng hạn do lỗi thực thi, thiếu quyền hoặc thiếu bằng chứng, Candidate chuyển sang `deferred`.

Trong chế độ Whole-target, hệ thống quản lý các điểm đầu vào bằng `AttackSurfaceInput` và liên kết Candidate với đầu vào tương ứng. Tiến độ từng Candidate được theo dõi riêng với tiến độ các pha kiểm thử. Trước một lượt xác minh mới, Reconnaissance và Web Enumeration phải hoàn tất với các tệp kết quả hợp lệ. Candidate cần gắn với đầu vào phù hợp đã được phân tích và có tài liệu bàn giao phân tích hợp lệ. Những đầu vào độc lập còn chờ hoặc bị chặn không tự ngăn Candidate đủ điều kiện tiếp tục xác minh, nhưng vẫn được giữ là công việc chưa hoàn thành.

Trong chế độ Direct hoặc chế độ xác minh Candidate được chỉ định, hệ thống không bắt buộc ghi nhận `AttackSurfaceInput` trước. Các điều kiện chung về lượt xác minh, bằng chứng và quyền thực thi vẫn được áp dụng.

b) Theo dõi mức độ bao phủ kiểm thử

`WorkflowState` theo dõi quá trình xử lý từng Candidate và từng lượt xác minh, còn Coverage tổ chức thông tin kiểm thử theo tổ hợp endpoint, tham số và lớp lỗ hổng. Coverage Store lưu các bản ghi này, trong khi Coverage Tool hỗ trợ Agent tra cứu dữ liệu đã ghi nhận. Các thành phần chính của `CoverageEntry` được trình bày trong Bảng 2.13.

Bảng 2.13. Các thành phần dữ liệu chính của CoverageEntry

Thành phần	Ý nghĩa
`endpoint`	Endpoint của đối tượng được kiểm thử.
`param`	Tham số được xem xét tại endpoint tương ứng.
`vulnClass`	Lớp lỗ hổng được kiểm thử.
`status`	Trạng thái của tổ hợp, gồm `tried`, `passed`, `failed`, `waf-blocked` và `skipped`.

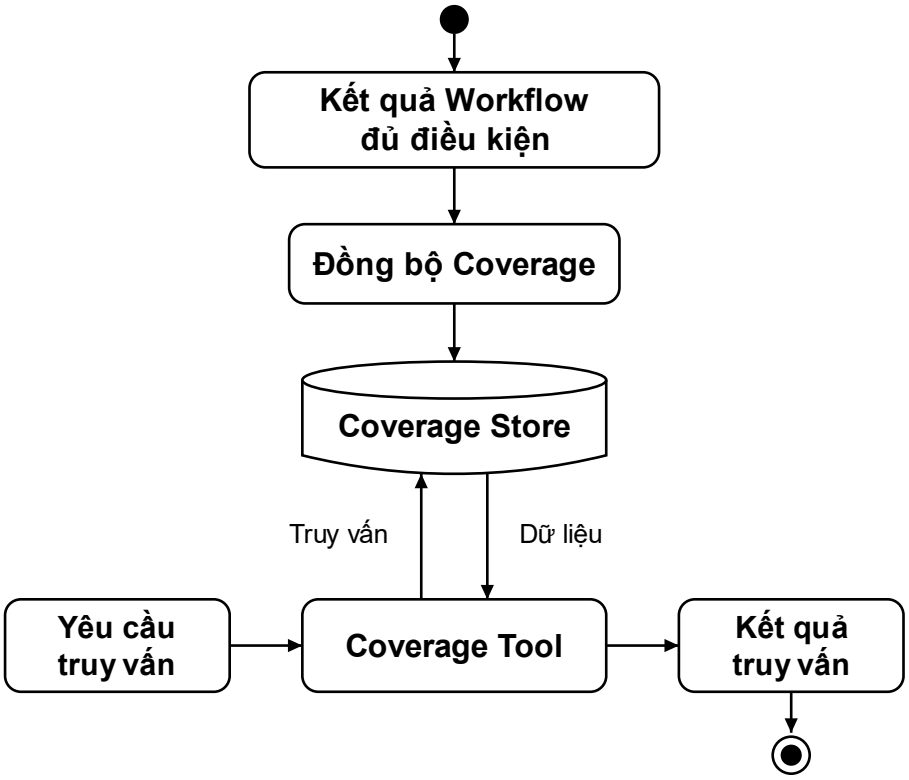
Thành phần	Ý nghĩa
`count`	Số lượt ghi nhận sau khi áp dụng cơ chế nhận diện và loại trừ trùng lặp. Không tương đương số yêu cầu HTTP hoặc số lần đồng bộ.
`context`	Ngữ cảnh phân biệt mục tiêu workflow, origin đích, phương thức, vị trí đầu vào, kiểu nội dung, xác thực và trường hợp kiểm tra khi có.

Cùng một đường dẫn có thể thuộc các phép kiểm tra khác nhau, chẳng hạn khi thay đổi phương thức HTTP hoặc điều kiện xác thực. Vì vậy, mỗi tổ hợp được lưu kèm ngữ cảnh tương ứng để những trường hợp này không bị gộp thành một đối tượng đã được xử lý.

Workflow Tool đồng bộ các kết quả xác minh đủ điều kiện sang Coverage. Agent sử dụng Coverage Tool để truy vấn, thay vì trực tiếp đánh dấu kết quả kiểm thử bằng công cụ này. Khi đồng bộ, kết quả `confirmed` tương ứng với trạng thái `failed`, còn `not-confirmed` tương ứng với `passed`. Failed cho biết phép kiểm tra đã ghi nhận vấn đề bảo mật. Passed chỉ cho biết phép kiểm tra tương ứng chưa xác nhận được giả thuyết, không chứng minh đối tượng an toàn. Những nhãn này không thay thế kết quả xác minh hoặc bằng chứng.

Trước khi lưu Coverage, hệ thống kiểm tra kết quả xác minh mới nhất, bằng chứng và ngữ cảnh của phép kiểm tra. Sau khi lưu thành công, các thông tin này được đối chiếu lại rồi mới ghi nhận đồng bộ hoàn tất. Nếu lưu thất bại hoặc hồ sơ thay đổi, việc đồng bộ vẫn chưa hoàn tất.

Cơ chế đồng bộ và truy vấn được thể hiện trong Hình 2.11.



Hình 2.11. Cơ chế đồng bộ và truy vấn Coverage

Coverage Tool trả về những bản ghi phù hợp với phạm vi và điều kiện truy vấn. Khi dùng `untested`, Agent cung cấp tập endpoint, tham số và lớp lỗ hổng cần đối chiếu. Công cụ xác định những tổ hợp chưa có bản ghi trong ngữ cảnh tương ứng. Kết quả này chỉ phản ánh tập được cung cấp, không tự khảo sát toàn bộ ứng dụng. Việc một tổ hợp đã có bản ghi cũng không đồng nghĩa đã hoàn tất xác minh.

Các lần đồng bộ lại hoặc cập nhật kết quả của cùng một Candidate trong cùng ngữ cảnh được nhận diện để hạn chế đếm trùng. Agent và Planner sử dụng Coverage cùng trạng thái Candidate, kết quả hiện hành và các trở ngại trong `WorkflowState` để lựa chọn công việc tiếp theo.

c) Xác nhận và quản lý kết quả phát hiện

Finding biểu diễn phát hiện đã đáp ứng các điều kiện ghi nhận của hệ thống và được lưu thành báo cáo. Để tạo Finding, Agent gửi yêu cầu đến Confirm Finding Tool. Công cụ kiểm tra hồ sơ xác minh trong workflow, chuẩn bị nội dung và chuyển dữ liệu tới Finding Store để lưu báo cáo.

Confirm Finding Tool sử dụng kết quả xác minh mới nhất của Candidate. Kết quả phải ở trạng thái `confirmed`, có bằng chứng đã đăng ký và còn phù hợp với hồ sơ kiểm thử hiện tại. Công cụ kiểm tra tính toàn vẹn, ngữ cảnh mục tiêu và liên kết

của bằng chứng với lượt xác minh. Khi việc đồng bộ Coverage tương ứng còn đang chờ, yêu cầu chưa được chấp nhận. Các thành phần dữ liệu chính của Finding được trình bày trong Bảng 2.14.

Bảng 2.14. Các thành phần dữ liệu chính của Finding

Thành phần	Vai trò
`title`	Tên của phát hiện.
`severity`	Mức độ nghiêm trọng từ đánh giá được chấp nhận.
`url`, `parameter`, `method`	Vị trí, tham số và phương thức của yêu cầu liên quan.
`candidate_id`, `evidence_refs`	Liên kết Candidate và bằng chứng đã đăng ký.
`observed_impact`, `potential_impact`	Tác động quan sát được và tác động tiềm năng chưa kiểm chứng.
`payload`, `responseExcerpt`, `curl`	Dữ liệu kiểm thử, trích đoạn phản hồi và lệnh tái hiện khi có.
`remediation`	Khuyến nghị khắc phục phù hợp với thông tin đã kiểm chứng.
`cwe`, `owasp`	Phân loại CWE và OWASP khi có cơ sở.

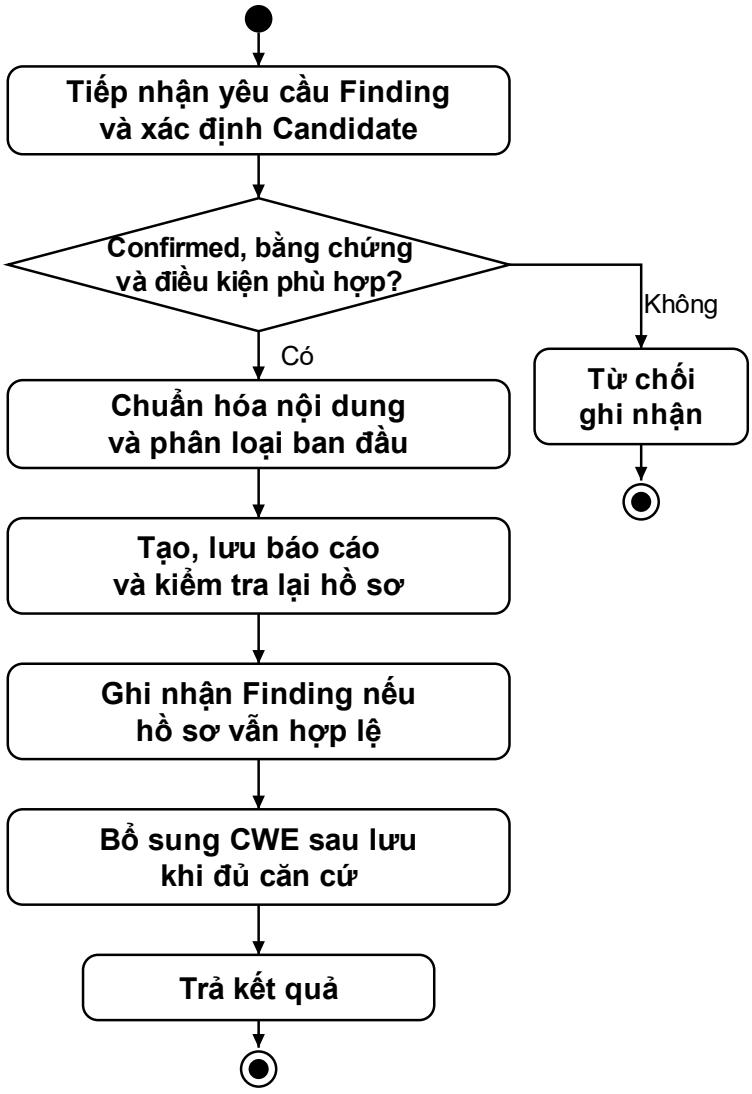
Mức độ nghiêm trọng và tác động quan sát được trong báo cáo lấy từ đánh giá đã được chấp nhận. Các giá trị truyền vào yêu cầu tạo báo cáo không thay thế những thông tin này. Tác động tiềm năng được trình bày riêng và có điều kiện, phù hợp với giới hạn của bằng chứng. Thông tin tái hiện và trích đoạn phản hồi được xử lý để hạn chế lưu dữ liệu nhạy cảm. Khuyến nghị khắc phục cần dựa trên thông tin thực tế đã được kiểm chứng.

Khi xác định được lớp lỗ hổng phù hợp, hệ thống bổ sung mã phân loại tương ứng theo CWE [14] và OWASP. Nếu chưa có ánh xạ trực tiếp, báo cáo có thể được lưu với phân loại chưa xác định. CWE có thể được bổ sung sau khi lưu, khi có đủ căn cứ từ danh mục đã kiểm chứng. Việc bổ sung phân loại không thay thế bằng chứng hoặc tự làm tăng độ chắc chắn của kết luận.

Finding Store lưu báo cáo Markdown dưới thư mục `artifacts/findings` của dự án. Khi lưu, hệ thống tránh ghi đè báo cáo khác chỉ vì trùng tên và hỗ trợ nhận diện báo cáo đã có của cùng Candidate.

Sau khi lưu, công cụ kiểm tra lại kết quả xác minh và bằng chứng. Nếu báo cáo vẫn khớp với kết quả mới nhất và hồ sơ còn hợp lệ, workflow ghi nhận Finding đã được lưu cho kết quả đó. Nếu kết quả hoặc bằng chứng thay đổi trong lúc lưu, tệp báo cáo đã tạo không tự được công nhận là Finding phù hợp với trạng thái hiện tại. Kết quả được ghi nhận hợp lệ sẽ được trả về Agent và có thể hiển thị trên giao diện.

Quy trình xử lý được thể hiện trong Hình 2.12.



Hình 2.12. Quy trình xử lý của Confirm Finding

Confirm Finding Tool kiểm tra điều kiện ghi nhận và lưu báo cáo, không thực hiện một phép kiểm chứng lỗ hổng độc lập. Do đó, cần phân biệt kết quả xác minh `confirmed` với Finding đã được lưu hợp lệ. Độ chính xác của kết luận vẫn phụ thuộc vào quá trình kiểm thử, bằng chứng thu thập được và cách Agent đánh giá dữ liệu.

2.6. Kết luận chương 2

Chương 2 đã trình bày quá trình phân tích và thiết kế hệ thống AI Pentest Agent dựa trên cơ sở lý thuyết đã xây dựng ở Chương 1. Trước hết, chương đã mô tả bài toán đặt ra cho hệ thống, xác định rõ đầu vào, đầu ra và cơ chế xử lý theo hướng tiếp cận tương tự mô hình ReAct, gồm ba pha lập kế hoạch, thực hiện hành động và tiếp nhận kết quả. Trên cơ sở đó, các yêu cầu chức năng và phi chức năng của hệ thống đã được xác định, làm căn cứ cho việc thiết kế kiến trúc và các thành phần ở phần sau.

Bên cạnh đó, chương đã đề xuất kiến trúc tổng quát của KAgent với các thành phần chính gồm Interaction Layer, Agent Core, LLM Client, Skills System, Tool Registry, Tool Execution System và Knowledge & State, đồng thời phân rã chi tiết kiến trúc từng thành phần và mối quan hệ phụ thuộc giữa chúng. Luồng xử lý và cơ chế hoạt động của hệ thống, bao gồm quy trình vận hành tổng quan, cơ chế quản lý ngữ cảnh, trạng thái phiên và cơ chế xử lý, thực thi công cụ, cũng đã được làm rõ. Trên nền tảng kiến trúc đó, năm mô-đun chức năng cốt lõi đã được thiết kế chi tiết, gồm mô-đun điều phối và lập kế hoạch tác vụ, mô-đun tri thức và kỹ năng kiểm thử, mô-đun quản lý và thực thi công cụ, mô-đun quản lý phiên và bộ nhớ, cùng mô-đun trạng thái và kết quả kiểm thử.

Những nội dung đã trình bày trong chương là cơ sở thiết kế quan trọng để tiến hành xây dựng và hiện thực hệ thống. Trên nền tảng đó, Chương 3 sẽ tập trung trình bày quá trình xây dựng, thực nghiệm và đánh giá hệ thống AI Pentest Agent.

CHƯƠNG 3. XÂY DỰNG VÀ ĐÁNH GIÁ HỆ THỐNG AI PENTEST AGENT

Trên cơ sở kiến trúc và các mô-đun chức năng đã được phân tích, thiết kế ở Chương 2, Chương 3 tập trung vào quá trình hiện thực, triển khai và đánh giá hệ thống AI Pentest Agent. Nội dung chương bắt đầu với việc trình bày môi trường phát triển và các công nghệ được sử dụng để xây dựng KAgent. Từ đó, các thành phần chính của hệ thống được hiện thực và tích hợp theo kiến trúc đã đề xuất, hình thành nguyên mẫu có khả năng tiếp nhận yêu cầu kiểm thử, phối hợp với mô hình ngôn ngữ lớn, lựa chọn và thực thi công cụ, quản lý trạng thái và tổng hợp kết quả trong quá trình kiểm thử. Sau khi hệ thống được xây dựng, các kịch bản thực nghiệm được thiết lập trong môi trường kiểm soát nhằm khảo sát và đánh giá khả năng hoạt động của KAgent đối với các tác vụ kiểm thử xâm nhập ứng dụng web. Những kết quả thu được là cơ sở phân tích khả năng vận hành và các giới hạn của hệ thống trong phạm vi đã khảo sát.

3.1. Môi trường và công nghệ sử dụng

KAgent được triển khai trên Linux với Python 3.11 trở lên và sử dụng các gói phụ thuộc được khai báo trong tệp pyproject.toml. Để vận hành, hệ thống cần được cấu hình kết nối tới dịch vụ LLM.

Các thành phần kỹ thuật chính của hệ thống được tổng hợp tại Bảng 3.1.

Bảng 3.1. Các công nghệ và thư viện chính sử dụng trong KAgent

Công nghệ, thư viện	Vai trò trong KAgent
Python	Ngôn ngữ phát triển và môi trường thực thi chính của hệ thống
Textual và Rich	Xây dựng giao diện TUI và trình bày nội dung trên terminal
httpx	Thực hiện giao tiếp HTTP bất đồng bộ với ứng dụng web và các dịch vụ bên ngoài
REST API của LLM	Kết nối với các nhà cung cấp mô hình ngôn ngữ thông qua các adapter của hệ thống
JSON, JSONL, Markdown và YAML	Tổ chức và lưu trữ cấu hình, trạng thái, tri thức, metadata và kết quả của hệ thống

KAgent được phát triển bằng Python, kết hợp cơ chế xử lý bất đồng bộ của asyncio để thực hiện các tác vụ liên quan đến giao tiếp mạng và thực thi công cụ. Hệ thống cung cấp giao diện tương tác trên terminal thông qua Textual và Rich, cho phép người dùng theo dõi trạng thái, thực hiện thao tác và xem kết quả trong quá trình kiểm thử.

Trong quá trình hoạt động, KAgent sử dụng httpx để giao tiếp với ứng dụng web mục tiêu và các dịch vụ LLM. Việc kết nối với các nhà cung cấp mô hình được tổ chức thông qua các adapter, thay vì phụ thuộc trực tiếp vào SDK riêng của từng nhà cung cấp. Cách tổ chức này cho phép hệ thống sử dụng nhiều backend LLM thông qua lớp giao tiếp thống nhất.

Dữ liệu cấu hình, trạng thái, tri thức và kết quả kiểm thử được quản lý dưới dạng tệp, không yêu cầu hệ quản trị cơ sở dữ liệu riêng. Trong đó, JSON và JSONL được sử dụng để lưu trữ dữ liệu có cấu trúc, YAML phục vụ khai báo metadata, còn Markdown được sử dụng cho các hướng dẫn kỹ năng và báo cáo phát hiện.

3.2. Hiện thực và triển khai hệ thống

3.2.1. Tổ chức mã nguồn và khởi tạo hệ thống

Để hiện thực thiết kế đã trình bày ở Chương 2, mã nguồn KAgent được tổ chức theo trách nhiệm của từng thành phần. Các thành phần chính của hệ thống được triển khai trong thư mục `src/` và phân chia thành các nhóm chức năng như trình bày tại Bảng 3.2.

Bảng 3.2. Các nhóm mã nguồn chính của KAgent

Nhóm chức năng	Thư mục chính	Vai trò
Khởi chạy và giao diện	`src/cli/`, `src/ui/`	Tiếp nhận tham số khởi chạy, khởi tạo ứng dụng và cung cấp giao diện tương tác trên terminal
Điều phối và kết nối LLM	`src/agent/`, `src/llm/`	Điều phối quá trình xử lý yêu cầu và giao tiếp với dịch vụ LLM được cấu hình
Kỹ năng và công cụ	`src/skills/`, `src/tools/`	Nạp các kỹ năng kiểm thử, đăng ký và thực thi các công cụ mà Agent có thể sử dụng

Nhóm chức năng	Thư mục chính	Vai trò
Trạng thái và kết quả	`src/session/`, `src/workflow/`, `src/coverage/`, `src/findings/`, `src/memory/`	Quản lý phiên làm việc, tiến trình xử lý, mức độ bao phủ và các dữ liệu được lưu lại
Cấu hình và kiểm soát	`src/config/`, `src/permission/`, `src/target/`	Quản lý cấu hình vận hành, phạm vi mục tiêu kiểm thử và cơ chế cấp quyền khi thực thi công cụ

Khi người dùng khởi chạy chương trình, hệ thống phân tích các tham số dòng lệnh và nạp cấu hình đã lưu. Từ các thông tin này, chương trình khởi tạo client giao tiếp với LLM, nạp các kỹ năng có sẵn, tạo các đối tượng quản lý trạng thái, đồng thời đăng ký các công cụ tích hợp sẵn và công cụ từ những tích hợp được bật. Những thành phần đã được chuẩn bị sau đó được kết nối với Agent trước khi giao diện tương tác trên terminal được mở. Cầu nối Burp được khởi tạo khi có tùy chọn khởi chạy tương ứng.

Mỗi lần khởi chạy, KAgent tạo một phiên làm việc mới nếu người dùng không yêu cầu khôi phục phiên trước đó. Tham số `--target` cho phép chỉ định mục tiêu kiểm thử ban đầu và không phải là điều kiện để tạo phiên. Khi sử dụng `--resume`, hệ thống nạp lại dữ liệu của phiên đã lưu để tiếp tục quá trình làm việc. Khi ứng dụng kết thúc, chương trình dừng các thành phần theo dõi tệp và đóng các kết nối tích hợp đã được mở.

3.2.2. Hiện thực các chức năng chính

Trong KAgent, các chức năng chính được triển khai thành những thành phần phối hợp qua yêu cầu, kết quả công cụ và trạng thái phiên. Agent điều phối quá trình xử lý, còn các thành phần chuyên trách kết nối LLM, nạp kỹ năng, kiểm soát công cụ và lưu dữ liệu.

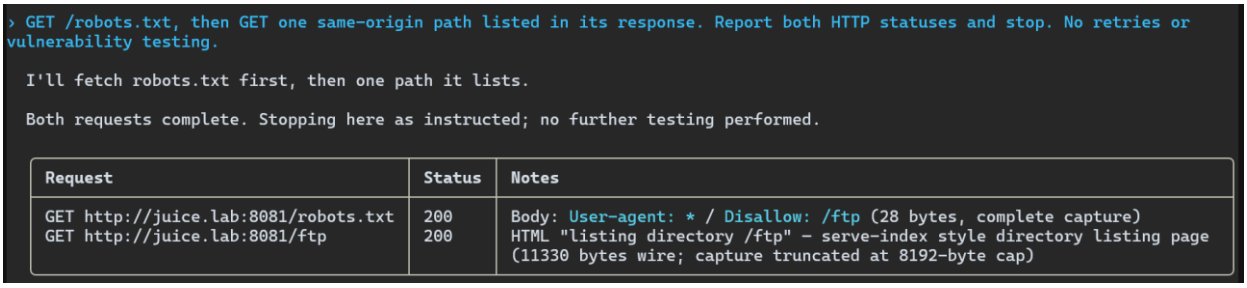
a) Hiện thực vòng lặp điều phối của Agent

Khi tiếp nhận yêu cầu, Agent Core chuẩn bị ngữ cảnh từ lịch sử phiên, thông tin mục tiêu và trạng thái kiểm thử hiện có. Decision Planner dựa trên mục tiêu và trạng thái kiểm thử để đề xuất định hướng hoặc công việc cần tiếp tục. Agent Core bổ sung chỉ dẫn và khai báo các công cụ phù hợp vào yêu cầu gửi tới LLM. LLM sử dụng ngữ cảnh, hướng dẫn của Skill đã nạp và phản hồi công cụ để lựa chọn hành

động cụ thể. Việc chuẩn bị và điều phối được duy trì trong cùng một Agent Core theo kiến trúc Single-Agent.

Phản hồi của LLM có thể chứa nội dung trả lời hoặc các lời gọi công cụ có cấu trúc (Tool Calls). Agent Core tiếp nhận tên công cụ và đối số, kiểm tra các ràng buộc của Skill và workflow rồi chuyển lời gọi tới cơ chế kiểm soát và thực thi tương ứng. Sau khi công cụ kết thúc, kết quả được liên kết với lời gọi bằng định danh và bổ sung vào lịch sử cùng ngữ cảnh làm việc. Thông tin lỗi hoặc từ chối thực thi cũng được ghi nhận. Trước lần gọi LLM tiếp theo, hệ thống cập nhật định hướng và trạng thái liên quan để mô hình tiếp tục phân tích dựa trên kết quả thực tế.

Hình 3.1 minh họa hoạt động gọi công cụ HTTP và tiếp nhận kết quả trong quá trình KAgent khảo sát tài nguyên web.



Hình 3.1. Minh họa vòng lặp LLM và công cụ trong KAgent

Với tương tác thông thường, Agent có thể kết thúc lượt khi LLM trả lời mà không yêu cầu công cụ. Đối với nhiệm vụ có workflow, hệ thống còn kiểm tra công việc và các điều kiện hoàn thành đã ghi nhận. Một phản hồi bằng văn bản chưa đủ để kết thúc mục tiêu khi vẫn còn công việc khả thi. Khi các điều kiện hoàn thành được đáp ứng, hệ thống chuyển sang tổng hợp kết quả. Những trường hợp bị chặn, hết giới hạn số bước hoặc mọi lời gọi công cụ đều bị từ chối được xử lý theo nhánh tương ứng. Tín hiệu hủy cho phép dừng quá trình khi cần. Phần tổng hợp phải phân biệt kết quả đã có với công việc chưa hoàn thành, vì kết thúc lượt tương tác không đồng nghĩa hoàn tất mục tiêu kiểm thử.

b) Hiện thực kết nối và tương tác với LLM

Người dùng lựa chọn nhà cung cấp và mô hình thông qua cấu hình của KAgent. Giao diện cấu hình LLM được minh họa trong Hình 3.2.



Hình 3.2. Giao diện chọn nhà cung cấp và mô hình LLM

Bên trái Hình 3.2 là menu `/provider`, cho phép lựa chọn nhà cung cấp và thiết lập cấu hình kết nối. Bên phải là hộp thoại lựa chọn mô hình của DeepSeek.

Thành phần kết nối LLM chuyển yêu cầu của Agent sang giao thức của dịch vụ được cấu hình. Lớp kết nối sử dụng thông tin về mô hình, địa chỉ dịch vụ và xác thực để chuyển thông điệp hội thoại cùng khai báo công cụ trong Chat Request sang định dạng mà dịch vụ hỗ trợ. Nhờ đó, Agent làm việc với một dạng yêu cầu chung thay vì tự xử lý giao thức của từng dịch vụ.

Khi nhận Chat Response, lớp kết nối chuyển nội dung trả lời và các Tool Calls về cấu trúc thống nhất để Agent xử lý. Mỗi lời gọi giữ tên công cụ, đối số và định danh để liên kết với thông điệp kết quả tương ứng ở lần trao đổi tiếp theo. Khi Streaming được bật và client có hỗ trợ, lớp kết nối tiếp nhận các phần phản hồi, tập hợp nội dung và lời gọi công cụ, đồng thời cung cấp các đoạn văn bản trả lời để cập nhật giao diện. Khi luồng kết thúc, client trả về Chat Response hoàn chỉnh. Với các giao thức đã được hỗ trợ, việc thay đổi nhà cung cấp LLM không yêu cầu thay đổi logic điều phối chính của Agent.

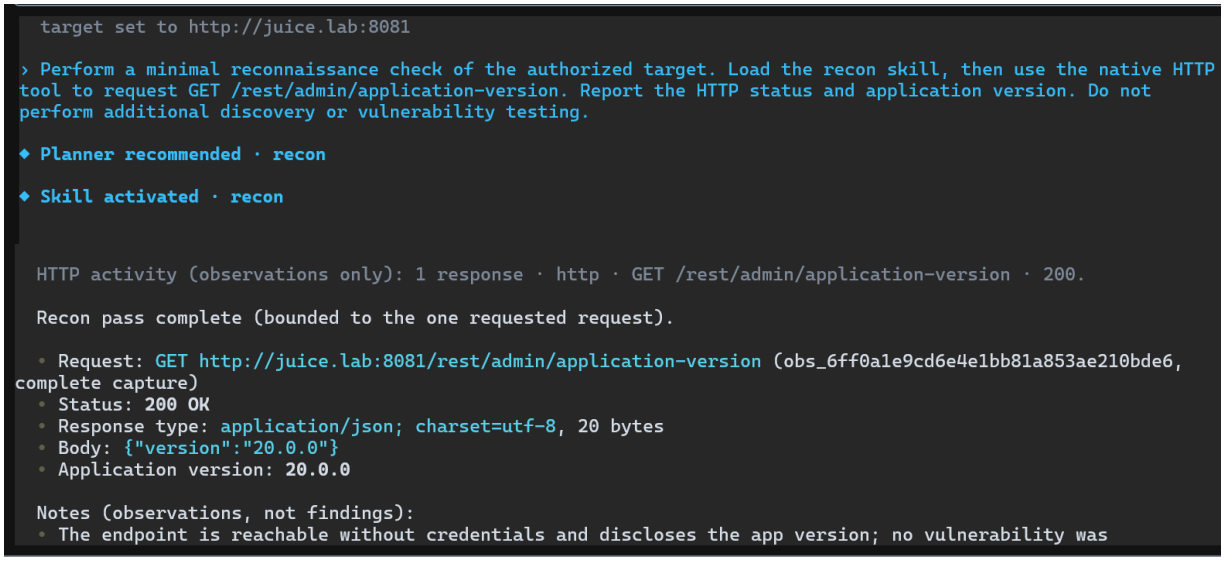
Lớp kết nối xử lý các lỗi xảy ra khi gửi yêu cầu và nhận phản hồi từ dịch vụ LLM. Với lỗi tạm thời, hệ thống mặc định gửi lại yêu cầu tối đa hai lần và tăng thời gian chờ giữa các lần thử. Trong chế độ nhận phản hồi theo luồng (Streaming), việc thử lại chỉ áp dụng ở bước mở luồng, trước khi xử lý nội dung trả về. Nếu vẫn không thể tiếp tục, KAgent thông báo lỗi và ghi nhận công việc chưa hoàn tất. Các thông báo được lọc để hạn chế làm lộ thông tin xác thực.

c) Hiện thực cơ chế lựa chọn kỹ năng và lập kế hoạch

Skill Registry quản lý thông tin mô tả các kỹ năng từ tệp `SKILL.md`, bao gồm chức năng, giai đoạn sử dụng và giới hạn công cụ. Dựa trên mục tiêu, trạng thái kiểm thử và các kỹ năng khả dụng, Decision Planner đề xuất kỹ năng hoặc công việc cần thực hiện tiếp theo. Các đề xuất này được đưa vào ngữ cảnh để LLM tham khảo khi

lựa chọn hành động cụ thể. Agent Core và runtime kiểm tra và thực thi các lời gọi công cụ tương ứng.

Khi LLM yêu cầu nạp một kỹ năng, công cụ `load_skill` kiểm tra kỹ năng có tồn tại, đang được bật và cho phép mô hình tự nạp hay không. Chỉ sau khi nạp thành công, hướng dẫn của Skill mới được bổ sung vào ngữ cảnh làm việc của LLM. Quá trình đề xuất, nạp Skill `recon` và sử dụng công cụ HTTP để kiểm tra phiên bản ứng dụng được minh họa trong Hình 3.3.



Hình 3.3. Nạp kỹ năng và thực thi công cụ trên KAgent

Trong ví dụ ở Hình 3.3, KAgent tiếp nhận yêu cầu kiểm tra phiên bản ứng dụng theo Skill và endpoint do người dùng chỉ định, kích hoạt Skill `recon` và thực hiện yêu cầu HTTP. Phản hồi có mã trạng thái 200 và cho biết phiên bản ứng dụng là 20.0.0. Ví dụ minh họa cách định hướng kỹ năng được kết hợp với hành động của LLM và kết quả công cụ trong một lượt xử lý.

Đối với Candidate cần xác minh, Planner đề xuất Skill chuyên biệt khi có duy nhất một kỹ năng khả dụng được ánh xạ tới lớp lỗ hổng tương ứng. Nếu ánh xạ mơ hồ hoặc có ánh xạ nhưng không có kỹ năng khả dụng, kế hoạch nêu lý do chưa thể tiếp tục xác minh. Nhánh xác minh tổng quát có giới hạn chỉ được xem xét khi không có ánh xạ Skill chuyên biệt, Skill Registry không ghi nhận lỗi nạp metadata và các điều kiện thực hiện được đáp ứng.

d) Hiện thực cơ chế gọi và kiểm soát công cụ

Lời gọi công cụ do LLM đề xuất được Agent chuyển tới Tool Registry sau khi kiểm tra ràng buộc của kỹ năng và workflow. Mỗi công cụ có tên định danh và mô tả tham số để hệ thống tra cứu đúng chức năng. Registry kiểm tra đối số theo các

yêu cầu của từng công cụ trước khi chuyển sang bước thực thi. Việc có lời gọi đúng cấu trúc chưa đồng nghĩa thao tác đã được cho phép.

Khi hành động cần phê duyệt, hệ thống chuẩn bị thông tin để người vận hành xem xét qua hộp thoại quyền ở Hình 3.7, Mục 3.2.3. Quyết định cho phép có hiệu lực trong phạm vi tương ứng. Chế độ YOLO ON tự động phê duyệt các hành động được chính sách hỗ trợ, nhưng không thay thế các kiểm tra về quyền hiện hành, phạm vi và giới hạn thực thi. Những hành động không đủ điều kiện tự động phê duyệt được xử lý theo cơ chế kiểm soát tương ứng.

Các kiểm soát cụ thể được áp dụng theo loại công cụ. Công cụ HTTP tích hợp kiểm tra origin đích, địa chỉ riêng và hạn mức yêu cầu theo các cơ chế kiểm soát khi thực thi. Shell chạy trực tiếp dưới quyền tài khoản vận hành, với giới hạn thời gian và đầu ra. Công cụ này không kế thừa cơ chế kiểm soát mạng riêng của công cụ HTTP.

Sau khi thao tác kết thúc, kết quả hoặc lỗi được trả về Agent. Dữ liệu đưa vào ngữ cảnh được giới hạn dung lượng và xử lý thông tin nhạy cảm theo chính sách của nguồn.

e) Hiện thực quản lý trạng thái và ngữ cảnh phiên

KAgent duy trì lịch sử hội thoại để tiếp tục trao đổi qua nhiều lượt. Khi xử lý yêu cầu mới, Agent Core kết hợp lịch sử với thông tin mục tiêu, dữ liệu workflow và các chỉ dẫn phù hợp để xây dựng ngữ cảnh gửi tới LLM. `SessionMemory` giữ phần thông tin tóm lược sau thu gọn, còn `WorkflowState` lưu tiến độ kiểm thử bằng các bản ghi có cấu trúc. Nhờ được quản lý riêng, trạng thái kiểm thử không phụ thuộc hoàn toàn vào nội dung hội thoại.

Memory và Intelligence bền vững cung cấp thông tin kỹ thuật có thể tái sử dụng khi phù hợp với nhiệm vụ, khác với `SessionMemory` phục vụ việc tiếp tục một phiên. Nội dung truy hồi được đưa vào như dữ liệu tham khảo, không thay thế trạng thái workflow hoặc cấp quyền thực thi.

Session Store lưu lịch sử, mục tiêu, `SessionMemory`, `WorkflowState` và thông tin phạm vi của phiên. Trước khi ghi xuống tệp, hệ thống tạo bản sao nhất quán của các dữ liệu này để sử dụng khi khôi phục phiên. Kết quả công cụ và bằng chứng có thể được lưu riêng. Dữ liệu phiên giữ những tham chiếu cần thiết thay vì chứa toàn bộ nội dung của mọi nguồn.

Đối với kết quả công cụ dài từ những nguồn được hỗ trợ, cơ chế giảm nội dung ngữ cảnh giữ bản xem trước và tham chiếu tới phần lưu riêng. Agent có thể

đọc lại dữ liệu khi đáp ứng các điều kiện truy cập và bản lưu vẫn khớp với dự án, phiên cùng mã định danh ngữ cảnh mục tiêu (generation) hiện hành. Cách lưu này giúp hạn chế lượng nội dung gửi tới LLM mà vẫn giữ khả năng xem lại kết quả.

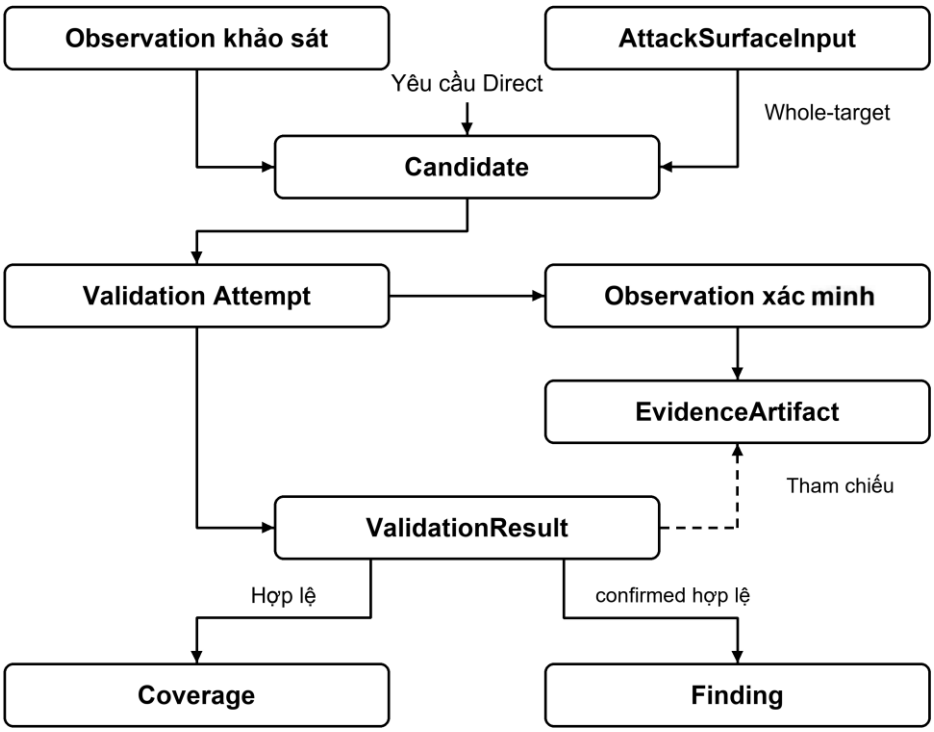
Ở đầu một lượt tương tác mới, khi đáp ứng các điều kiện thu gọn lịch sử, hệ thống tạo bản tóm lược và giữ lại phần tương tác gần nhất. Agent so sánh kích thước yêu cầu dự kiến trước và sau thu gọn. Nếu mức giảm chưa đạt ngưỡng quy định, lịch sử cũ được giữ lại. Khi kết quả được chấp nhận, `SessionMemory` và lịch sử được cập nhật, sau đó lưu để tiếp tục xử lý. Các bản ghi workflow vẫn được duy trì riêng, nên một câu tóm tắt về phát hiện không thay thế kết quả xác minh đã ghi nhận.

Khi khôi phục phiên, hệ thống nạp lại các dữ liệu đã lưu và xây dựng lại chỉ dẫn cho mô hình. Các lượt xác minh đang hoạt động được chuyển sang trạng thái gián đoạn. Tiến trình công cụ không được khởi động lại từ vị trí đã dừng. Trạng thái kích hoạt Skill và bộ nhớ đệm quyền tạm thời cũng không tự khôi phục. Skill cần được nạp hoặc kích hoạt lại khi sử dụng, còn hành động tiếp theo phải đáp ứng quyền, phạm vi và giới hạn hiện hành. Lịch sử và các tham chiếu đã lưu giúp tiếp tục phân tích, nhưng không bảo đảm mọi nguồn dữ liệu hoặc mẫu yêu cầu phát lại vẫn còn khả dụng.

f) Hiện thực quản lý tiến trình và kết quả kiểm thử

Workflow Tool tiếp nhận các yêu cầu cập nhật tiến độ và ghi nhận thay đổi trên `WorkflowState`. Candidate biểu diễn một giả thuyết cần kiểm tra, chưa phải phát hiện đã xác nhận. Khi bắt đầu xác minh hợp lệ, hệ thống tạo bản ghi Validation Attempt gắn với Candidate và ngữ cảnh công việc hiện tại. Những quan sát được thu thập trong lượt đó giữ thông tin nguồn và liên kết tương ứng để Agent lập hồ sơ kết quả. Các thành phần điều phối sử dụng trạng thái đã ghi nhận để xác định công việc cần tiếp tục.

Hình 3.4 thể hiện các liên kết dữ liệu phục vụ xác minh và cập nhật kết quả kiểm thử.



Hình 3.4. Liên kết dữ liệu xác minh và cập nhật kết quả trong KAgent

KAgent phân biệt dữ liệu khảo sát phục vụ hình thành Candidate với quan sát dùng để xác minh lỗ hổng. Từ các `Observation` phù hợp, Agent lập tệp phân tích và đăng ký bằng chứng qua Workflow Tool. Tệp vừa ghi chưa tự trở thành `EvidenceArtifact` đã đăng ký. Hệ thống kiểm tra và tạo bản lưu bất biến, ghi nhận liên kết Candidate cùng các nguồn quan sát được khai báo. Đăng ký thành công mới xác lập bản lưu bằng chứng, chưa xác nhận lỗ hổng.

Agent gửi `ValidationResult` kèm các tham chiếu tới bằng chứng và nguồn quan sát. Runtime kiểm tra hồ sơ theo điều kiện về Candidate, Validation Attempt, nguồn gốc, tính toàn vẹn và ngữ cảnh tương ứng trước khi chấp nhận kết quả. Trong sơ đồ, đường nét đứt biểu thị tham chiếu từ `ValidationResult` tới `EvidenceArtifact`. Các mũi tên biểu diễn quan hệ dữ liệu, không hàm ý mọi bước đều tự động diễn ra hoặc mọi kết quả công cụ đều đủ điều kiện làm bằng chứng.

Với kết quả đủ điều kiện, Workflow Tool đồng bộ tổ hợp endpoint, tham số và lớp lỗ hổng vào Coverage trong ngữ cảnh kiểm thử tương ứng. Kết quả `confirmed` được ánh xạ thành `failed`, còn `not-confirmed` thành `passed`. Các trạng thái chưa thể kết luận không được đồng bộ theo hai nhãn này. Passed chỉ phản

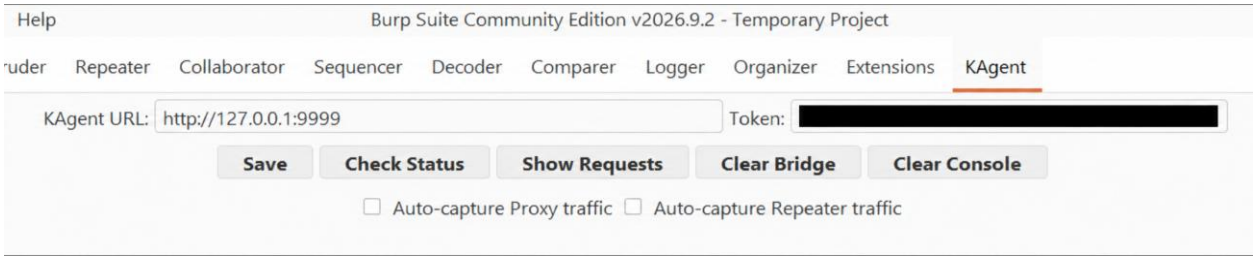
ánh phép kiểm tra đã thực hiện chưa xác nhận được giả thuyết, không chứng minh đối tượng an toàn.

Agent chuyển kết quả `confirmed` đáp ứng điều kiện ghi nhận phát hiện tới công cụ `confirm_finding` để tạo báo cáo. Công cụ kiểm tra kết quả và bằng chứng trước khi lưu vào Finding Store. Sau khi lưu thành công, hệ thống kiểm tra lại tính nhất quán rồi mới ghi nhận Finding cho kết quả tương ứng trong workflow. Nếu đồng bộ Coverage hoặc lưu Finding chưa hoàn tất, công việc còn chờ tiếp tục được theo dõi.

Cơ chế trên giúp KAgent quản lý kết quả xác minh và phát hiện theo từng bước, đồng thời duy trì liên kết giữa Candidate, bằng chứng và Finding. Các điều kiện kiểm tra được áp dụng trước khi cập nhật trạng thái hoặc ghi nhận phát hiện nhằm hạn chế việc sử dụng những hồ sơ không hợp lệ.

g) Hiện thực thành phần mở rộng tích hợp Burp Suite

Thành phần mở rộng Burp Suite được hiện thực bằng Python trên Jython và giao tiếp với cầu nối HTTP cục bộ của KAgent. Qua API của Burp, thành phần này chuyển các giao dịch được lựa chọn hoặc lưu lượng từ Proxy, Repeater theo cấu hình. Cầu nối tiếp nhận và ghi nhận dữ liệu vào kho capture có giới hạn trong bộ nhớ. Người kiểm thử lựa chọn bản capture trên KAgent, sau đó Agent đọc chi tiết qua công cụ truy xuất để phân tích hoặc thực hiện yêu cầu kiểm thử. Giao diện cấu hình được trình bày trong Hình 3.5.



Hình 3.5. Giao diện thành phần mở rộng KAgent trong Burp Suite

Các giao dịch đã tiếp nhận cung cấp cấu trúc yêu cầu và ngữ cảnh cho bước phân tích. Việc chuyển hoặc lựa chọn dữ liệu từ Burp không cấp quyền phát lại request hoặc thực hiện kiểm thử. Nếu sử dụng mẫu yêu cầu cho thao tác tiếp theo, công cụ HTTP vẫn phải kiểm tra nguồn, phạm vi, ngữ cảnh xác thực và điều kiện phát lại. Cơ chế kiểm soát công cụ tại phần d tiếp tục được áp dụng.

Dữ liệu capture từ Burp Suite được lưu tạm thời trong bộ nhớ với dung lượng giới hạn. Khi dữ liệu không còn khả dụng, người kiểm thử cần cung cấp bản capture mới để tiếp tục phân tích. Cơ chế tích hợp này giúp KAgent sử dụng thông tin từ

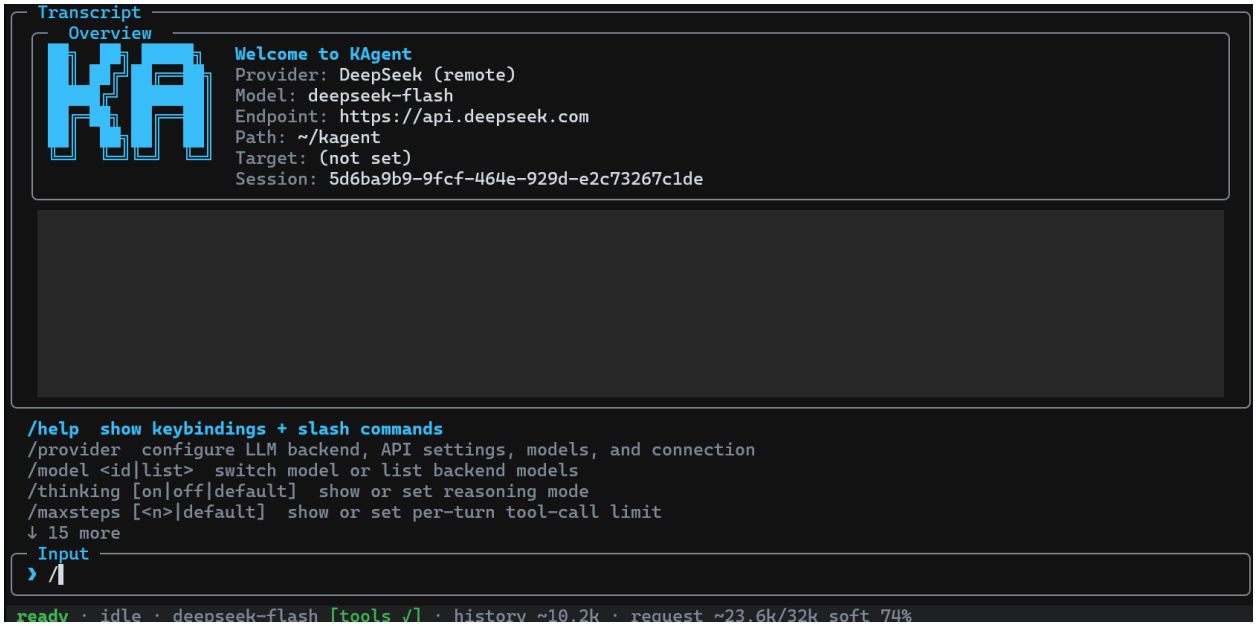
Burp Suite trong quá trình kiểm thử, đồng thời duy trì các điều kiện kiểm soát công cụ của hệ thống.

3.2.3. Giao diện và tương tác với hệ thống

KAgent sử dụng giao diện văn bản trên terminal để người dùng gửi yêu cầu, theo dõi quá trình xử lý và đưa ra quyết định khi cần.

a) Giao diện chính và theo dõi hoạt động

Hình 3.6 minh họa giao diện chính của KAgent.



Hình 3.6. Giao diện chính của KAgent

Phần tổng quan cung cấp thông tin về nhà cung cấp LLM, mô hình đang sử dụng và phiên làm việc hiện tại. Khu vực hội thoại hiển thị yêu cầu của người dùng, phản hồi của Agent, kết quả thực thi công cụ và các thông báo liên quan. Người dùng có thể nhập yêu cầu hoặc lệnh điều khiển tại ô nhập phía dưới. Thanh trạng thái cung cấp thêm thông tin về tình trạng hoạt động của hệ thống và ngữ cảnh phiên làm việc.

b) Tương tác và điều khiển phiên làm việc

Trong cùng một phiên, người dùng có thể xem xét kết quả của lượt xử lý trước và đưa ra chỉ dẫn tiếp theo, chẳng hạn chọn một đầu vào đã phát hiện để tiếp tục kiểm tra.

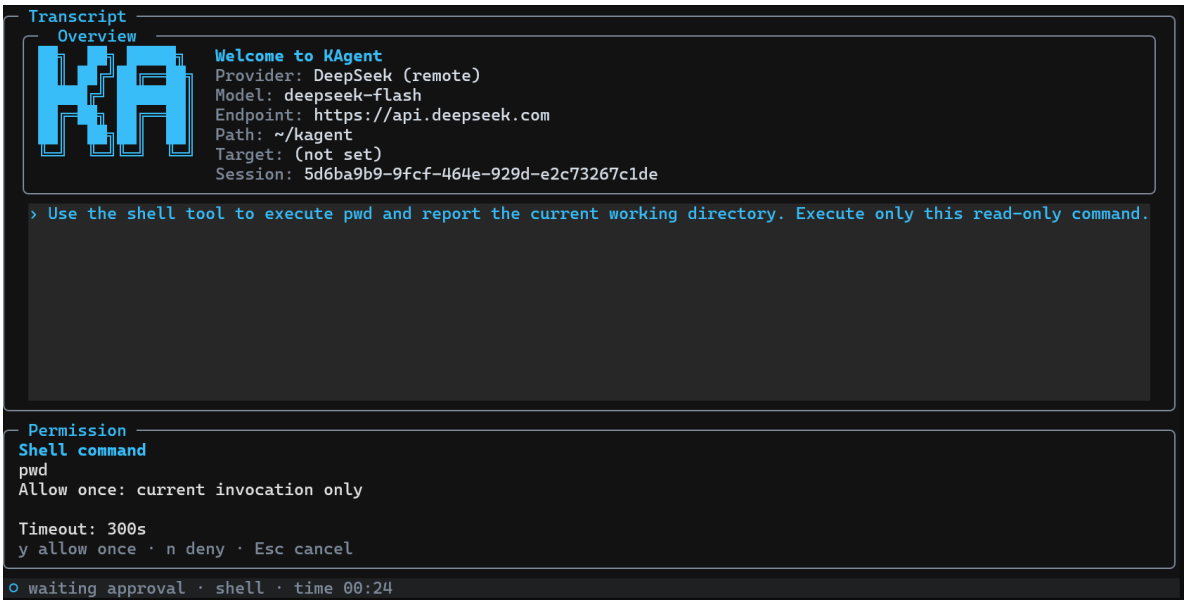
KAgent sử dụng lịch sử hội thoại và trạng thái kiểm thử để duy trì ngữ cảnh giữa các lượt. Trong quá trình xử lý, Decision Planner cung cấp định hướng dựa trên trạng thái công việc, còn LLM sử dụng ngữ cảnh, hướng dẫn của Skill khi được nạp và phản hồi công cụ để lựa chọn hành động cụ thể. Kết quả thực thi được đưa trở lại ngữ cảnh để LLM tiếp tục phân tích và xác định bước xử lý tiếp theo. Ví dụ về việc

điều chỉnh phép thử theo phản hồi của ứng dụng được trình bày tại Hình 3.12, Mục 3.3.1.

Ngoài yêu cầu bằng ngôn ngữ tự nhiên, người dùng có thể nhập lệnh điều khiển trong ô nhập. Chẳng hạn, `/provider` hỗ trợ cấu hình dịch vụ LLM, `/target` quản lý mục tiêu kiểm thử và `/skills` cho phép xem hoặc thay đổi trạng thái các kỹ năng. Lệnh `/yolo` được dùng để thay đổi chế độ phê duyệt. Khi cần thêm thông tin, hệ thống có thể hiển thị câu hỏi để người dùng lựa chọn hoặc nhập câu trả lời.

c) Phê duyệt và kiểm soát quyền thực thi

Hình 3.7 minh họa hộp thoại xác nhận quyền thực thi công cụ.



Hình 3.7. Hộp thoại xác nhận quyền thực thi công cụ

Khi cần xác nhận, giao diện hiển thị thông tin về hành động cùng các lựa chọn xử lý. Ví dụ trong hình là yêu cầu thực thi lệnh `pwd`, với các lựa chọn cho phép một lần, từ chối hoặc hủy yêu cầu.

Đề xuất hành động của LLM được chuyển qua cơ chế kiểm soát thực thi trước khi công cụ thực hiện. Ở chế độ YOLO OFF, những hành động thuộc diện cần phê duyệt được đưa tới người dùng xem xét trước khi thực thi. Các quyền đã được cấp phù hợp, nếu có, vẫn được áp dụng theo phạm vi và điều kiện của chúng.

Ở chế độ YOLO ON, hệ thống tự động phê duyệt các hành động được chính sách hỗ trợ. Việc tự động phê duyệt không thay thế các kiểm tra về quyền, phạm vi và giới hạn tài nguyên được áp dụng theo từng loại công cụ. Những hành động không đủ điều kiện tự động phê duyệt vẫn phải tuân theo quyết định kiểm soát tương ứng của hệ thống. Cơ chế này được trình bày trong phần hiện thực cơ chế gọi và kiểm soát công cụ tại Mục 3.2.2.

3.2.4. Cấu hình và triển khai hệ thống

KAgent được xây dựng dưới dạng ứng dụng Python và cung cấp giao diện tương tác trên terminal. Từ thư mục mã nguồn và môi trường Python đã chuẩn bị, ứng dụng có thể được cài đặt và khởi chạy bằng các lệnh sau:

```
python -m pip install .
kagent
```

Hệ thống mặc định đọc cấu hình từ `~/.kagent/config.json` khi khởi động. Một số thiết lập cũng có thể được cung cấp qua tham số dòng lệnh hoặc biến môi trường. Mục tiêu, phạm vi và trạng thái phiên được quản lý theo ngữ cảnh làm việc. Các nhóm thiết lập chính được trình bày trong Bảng 3.3.

Bảng 3.3. Các nhóm thiết lập phục vụ khởi chạy và vận hành KAgent

Nhóm thiết lập	Mô tả
Kết nối LLM	Nhà cung cấp, mô hình, địa chỉ dịch vụ và khóa API khi dịch vụ yêu cầu.
Công cụ và Skill	Các công cụ được đăng ký, thư mục chứa Skill và trạng thái bật, tắt của Skill.
Mục tiêu và phiên làm việc	Thông tin mục tiêu và trạng thái của phiên mới hoặc phiên được khôi phục.
Giới hạn xử lý	Số bước tối đa của vòng lặp Agent trong một lượt xử lý.
Kiểm soát thực thi	Phạm vi kiểm thử, quyền sử dụng công cụ và chế độ phê duyệt.

Khi khởi động, KAgent sử dụng cấu hình hiện có để khởi tạo client giao tiếp với dịch vụ LLM. Một số nhà cung cấp yêu cầu khóa API để xác thực. Sau khi giao diện TUI được mở, người dùng có thể sử dụng `/provider` để quản lý nhà cung cấp, lựa chọn mô hình và kiểm tra kết nối.

Trong quá trình chuẩn bị hệ thống, KAgent phát hiện các Skill từ những thư mục đã cấu hình và đăng ký các công cụ tích hợp. Hệ thống tạo phiên làm việc mới hoặc khôi phục dữ liệu của phiên trước đó theo lựa chọn của người dùng.

Các yêu cầu sử dụng công cụ được kiểm soát theo chính sách và ngữ cảnh tương ứng. Chế độ YOLO có thể tự chấp thuận một số yêu cầu phê duyệt đủ điều kiện, nhưng không bỏ qua mọi kiểm tra và giới hạn thực thi. Việc khôi phục phiên không tự khôi phục các quyền thực thi tạm thời.

Quy trình cài đặt, cấu hình, khởi chạy và sử dụng hệ thống được trình bày chi tiết trong Phụ lục.

3.3. Thực nghiệm và đánh giá kết quả

Nhằm đánh giá khả năng hoạt động của KAgent, đồ án xây dựng hai kịch bản thực nghiệm trên các ứng dụng web trong môi trường kiểm thử được phép. Kịch bản thứ nhất tập trung đánh giá khả năng tự điều phối quy trình kiểm thử qua nhiều giai đoạn, từ khảo sát mục tiêu, khám phá và phân tích đầu vào đến xác minh SQL Injection và ghi nhận kết quả. Kịch bản thứ hai tập trung đánh giá khả năng phối hợp với người kiểm thử thông qua Burp Suite, tiếp nhận yêu cầu HTTP đã lựa chọn để phân tích và xác minh XSS. Kết quả thực nghiệm được xem xét dựa trên diễn biến quá trình kiểm thử, hoạt động sử dụng công cụ, bằng chứng thu thập được và các phát hiện được ghi nhận.

3.3.1. Thực nghiệm kiểm thử xâm nhập ứng dụng web

a) Mục tiêu thực nghiệm

Thực nghiệm này đánh giá khả năng tự chủ của KAgent khi bắt đầu từ địa chỉ một ứng dụng web. Người dùng giao mục tiêu khảo sát ứng dụng, khám phá và phân tích các đầu vào, xác minh SQL Injection và lưu kết quả kèm bằng chứng trong phạm vi được cấp phép. Agent phải sử dụng thông tin thu thập được để lựa chọn vị trí và phương pháp kiểm tra, thay vì nhận sẵn endpoint, tham số hoặc payload.

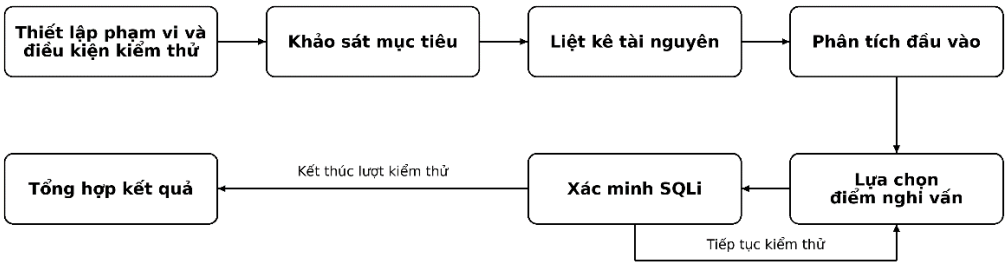
b) Cấu hình thực nghiệm

Đối tượng thực nghiệm là OWASP Juice Shop [15] tại `http://juice.lab:8081`. KAgent sử dụng mô hình deepseek-flash [16] của DeepSeek và không được cung cấp sẵn tài khoản đăng nhập ứng dụng. Kiểm thử được giới hạn ở SQL Injection.

Giới hạn xử lý được đặt ở 200 bước nhằm kiểm soát thời gian và tài nguyên sử dụng, đồng thời tránh để Agent tiếp tục thử nghiệm kéo dài khi chưa tìm được hướng xử lý phù hợp. Chế độ YOLO được bật để tự động chấp thuận các thao tác thuộc chính sách cho phép, giúp Agent thực hiện liên tiếp những công việc đủ điều kiện. Các ràng buộc về mục tiêu, quyền thực thi và tài nguyên vẫn được duy trì. Người dùng tiếp tục tham gia khi cần đưa ra những quyết định cần sự can thiệp của người kiểm thử.

c) Quy trình thực nghiệm

Quy trình thực nghiệm bắt đầu từ việc thiết lập phạm vi và điều kiện kiểm thử. Sau đó, KAgent khảo sát ứng dụng, liệt kê tài nguyên, phân tích các đầu vào, xác minh những điểm nghi vấn và tổng hợp kết quả. Quy trình tổng quát được minh họa tại Hình 3.8.

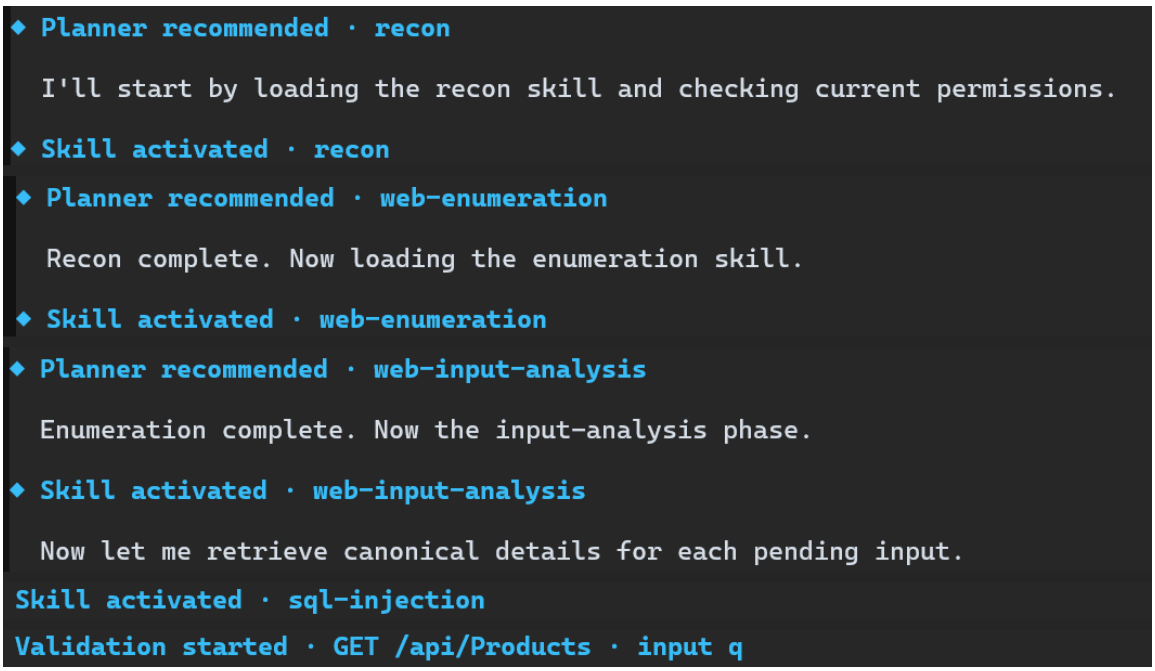


Hình 3.8. Quy trình kiểm thử của KAgent

1. **Thiết lập phạm vi và nhiệm vụ:** Người kiểm thử chuẩn bị mục tiêu, khởi tạo phiên KAgent, thiết lập phạm vi và quyền thực thi. KAgent nhận nhiệm vụ kiểm thử SQL Injection mà không được cung cấp sẵn endpoint, tham số hoặc payload.
2. **Khảo sát và liệt kê tài nguyên:** KAgent kiểm tra khả năng tiếp cận ứng dụng, thu thập thông tin và sử dụng các nguồn tìm được để xác định endpoint cùng tham số đầu vào.
3. **Phân tích và lựa chọn điểm kiểm tra:** Agent phân tích các đầu vào và phản hồi để lựa chọn điểm nghi vấn, ghi nhận thành Candidate. Người kiểm thử tham gia khi cần quyết định về quyền thực thi hoặc phạm vi.
4. **Xác minh SQL Injection:** KAgent sử dụng Skill và công cụ phù hợp để xây dựng phép thử, so sánh phản hồi và đánh giá bằng chứng. Kết quả quan sát được dùng để điều chỉnh phép thử hoặc bổ sung dữ liệu khi cần.
5. **Tổng hợp và ghi nhận kết quả:** Workflow lưu các đầu vào, Candidate, bằng chứng và kết quả xác minh. Những kết quả `confirmed` đủ điều kiện được lưu thành Finding. Các trường hợp `not-confirmed`, bị chặn, thiếu bằng chứng hoặc chưa hoàn tất được ghi nhận riêng.

d) Kết quả thực nghiệm

KAgent thực hiện kiểm thử qua bốn giai đoạn: khảo sát mục tiêu, liệt kê tài nguyên, phân tích đầu vào và xác minh SQL Injection với các Skill tương ứng `recon`, `web-enumeration`, `web-input-analysis` và `sql-injection`. Công cụ `http` thu nhận phản hồi để phân tích, còn Workflow ghi nhận đầu vào, Candidate, tham chiếu bằng chứng và kết quả kiểm thử. Hình 3.9 minh họa các Skill được nạp trong phiên.



Hình 3.9. KAgent kích hoạt kỹ năng theo từng giai đoạn kiểm thử

Kết quả khám phá và phân tích đầu vào

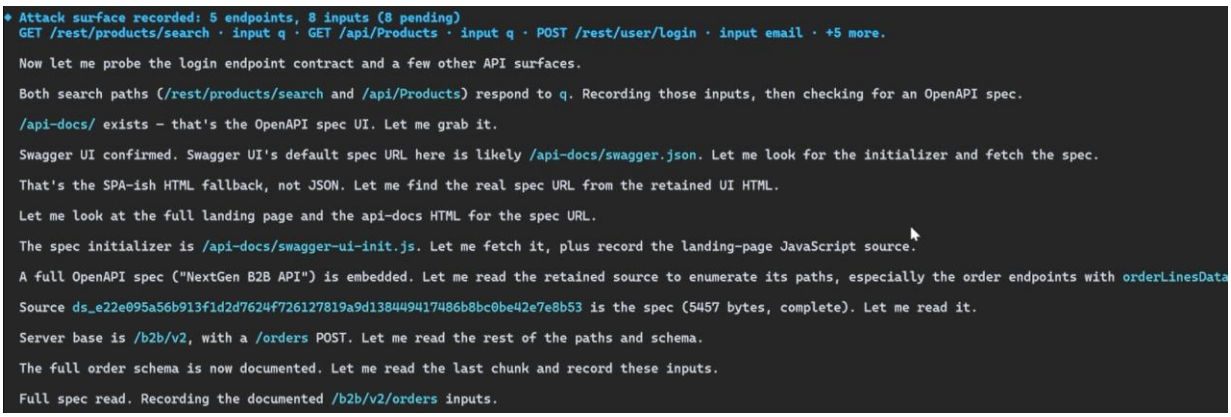
Ứng dụng OWASP Juice Shop tại `http://juice.lab:8081` trả HTTP 200. Trang chủ và dấu hiệu điều hướng cho thấy đây là ứng dụng một trang. Nội dung `robots.txt` chứa đường dẫn `/ftp` trong mục Disallow, cung cấp thêm một tài nguyên để khảo sát.

Chức năng tìm kiếm và các API liên quan được nhận diện qua phản hồi HTTP. Yêu cầu `GET /rest/products/search?q=apple` trả JSON chứa ba sản phẩm phù hợp khi chưa đăng nhập, còn `GET /rest/user/whoami` trả `{"user":{}}`, cho thấy yêu cầu hiện tại chưa gắn với người dùng đã xác thực.

Các phản hồi trang chủ, tệp JavaScript và kết quả thăm dò cung cấp nguồn xác định API. Agent chọn mức thu nhận 16 KiB cho tệp JavaScript chính nên mới phân tích được một phần nội dung tệp.

Đường dẫn dự đoán `/api-docs/swagger.json` trả HTML thay vì JSON. Đặc tả OpenAPI được tìm thấy trong `/api-docs/swagger-ui-init.js`, cung cấp thông tin về `POST /b2b/v2/orders` và các trường của yêu cầu đặt hàng.

Kết hợp đặc tả OpenAPI với các phản hồi trực tiếp, KAgent ghi nhận tám đầu vào thuộc năm endpoint trong Workflow. Hình 3.10 minh họa kết quả khám phá tài nguyên và ghi nhận đầu vào.

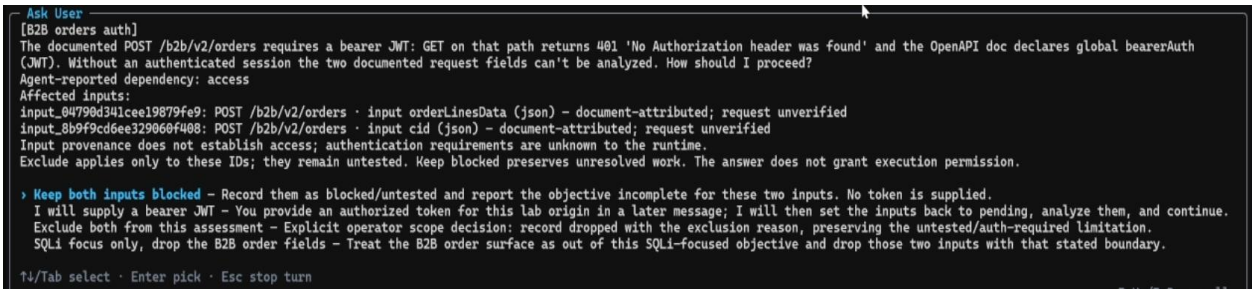


Hình 3.10. KAgent khám phá tài nguyên và ghi nhận đầu vào kiểm thử

Phân tích đầu vào cho thấy `q` ảnh hưởng đến kết quả tìm kiếm tại `/rest/products/search` và `/api/Products`: `q=apple` trả sản phẩm phù hợp, còn giá trị đánh dấu không trùng tên sản phẩm trả tập rỗng. Tham số `criteria` tại `/rest/products/search` chưa cho thấy tác dụng lọc trong phép thử vì phản hồi vẫn là danh sách sản phẩm đầy đủ như khi không gửi tham số.

Yêu cầu `POST /rest/user/login` với `email` và mật khẩu không hợp lệ trả HTTP 401 cùng thông báo `Invalid email or password`. Trường `email` được chọn để kiểm tra giả thuyết SQL Injection, cùng với tham số `q` tại hai API tìm kiếm. Sự thay đổi kết quả tìm kiếm và phản hồi đăng nhập thất bại là cơ sở lựa chọn điểm kiểm tra, chưa phải bằng chứng xác nhận lỗ hổng.

Hai trường `cid` và `orderLinesData` trong JSON của `POST /b2b/v2/orders` được mô tả trong OpenAPI với yêu cầu bearer token. Yêu cầu GET đến cùng đường dẫn trả HTTP 401 và thông báo thiếu thông tin xác thực. Phản hồi GET không cho biết cách xử lý POST, do POST chưa được gửi. Hình 3.11 thể hiện yêu cầu của KAgent để người kiểm thử quyết định phương án xử lý hai đầu vào này.



Hình 3.11. KAgent yêu cầu xác định phương án xử lý các đầu vào B2B

Người kiểm thử quyết định loại cả hai trường B2B khỏi phạm vi xác minh. Chúng được ghi nhận là chưa kiểm thử, không phải kết quả âm tính. Trong tám đầu vào, sáu đầu vào đã được phân tích và hai đầu vào bị loại khỏi phạm vi. KAgent tạo

ba Candidate SQL Injection tại hai tham số `q` và trường `email`, đồng thời lưu lý do chưa lựa chọn `criteria`, `password` và `comment` để xác minh.

Kết quả xác minh SQL Injection

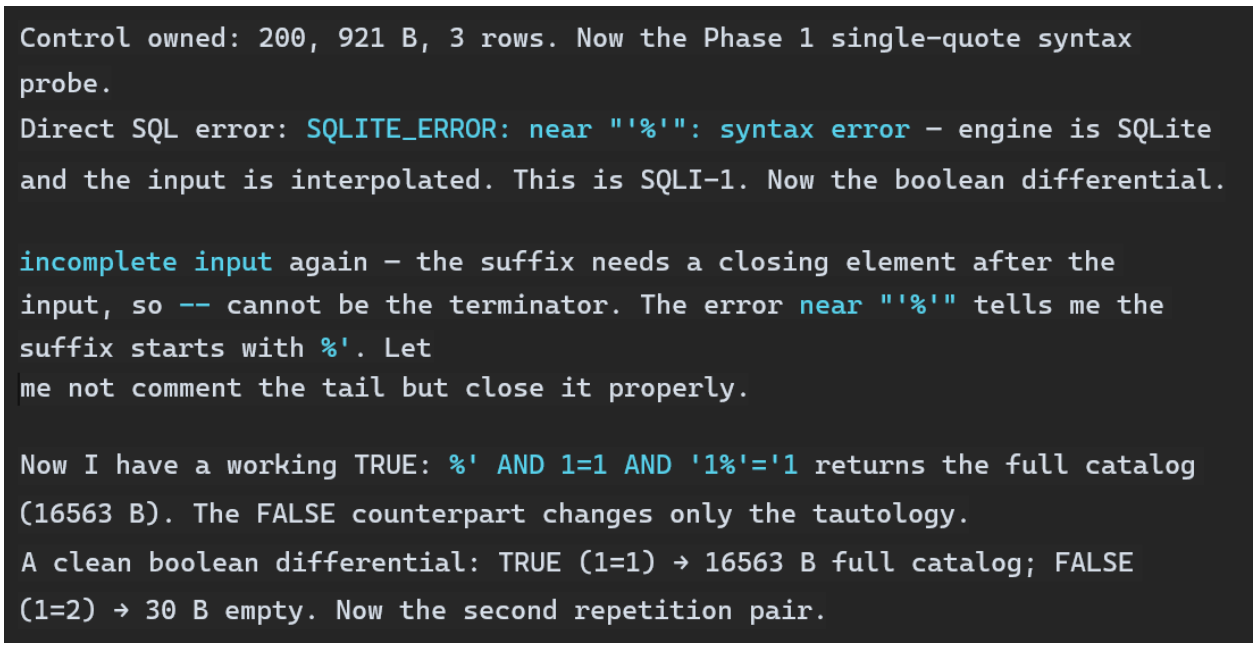
Tại `GET /api/Products`, giá trị `apple` trả ba sản phẩm và ký tự đại diện `%` trả danh sách đầy đủ. Ngược lại, `apple'` và các giá trị chứa dấu nháy đơn kèm biểu thức SQL đã thử đều trả HTTP 200 với tập rỗng 30 byte. Các phản hồi này không cho thấy lỗi cơ sở dữ liệu hoặc sự khác biệt hỗ trợ giả thuyết SQL Injection.

Phản hồi đối chứng với `%` được thu lại do bản ghi trước bị cắt. KAgent ghi nhận Candidate tại `/api/Products` là `not-confirmed`: các phép thử đã thực hiện chưa xác nhận được SQL Injection. Bằng chứng cũng chưa đủ để xác định cơ chế truy vấn bên trong hoặc kết luận endpoint an toàn.

Tại `POST /rest/user/login`, trong khi mật khẩu sai được giữ nguyên, giá trị `email ' OR 1=1--` cho phản hồi HTTP 200 dài 799 byte và token xác thực. Đổi riêng điều kiện thành `' OR 1=2--` cho phản hồi HTTP 401 dài 26 byte với thông báo `Invalid email or password`. Việc lặp lại hai giá trị cho kết quả tương ứng ở từng nhánh.

Bốn yêu cầu riêng biệt có bốn phản hồi đầy đủ, tạo thành hai cặp đối chứng TRUE/FALSE độc lập. Sự khác biệt nhất quán khi thay điều kiện đúng và sai là cơ sở để KAgent ghi nhận `confirmed` đối với khả năng vượt qua kiểm tra đăng nhập bằng SQL Injection. Thông tin trong phản hồi và token xác định tài khoản `admin@juice-sh.op` có vai trò admin. Token chưa được dùng để truy cập API khác, vì vậy các chức năng quản trị chưa được kiểm chứng.

Tại `/rest/products/search`, `q=apple'` gây HTTP 500 với thông báo lỗi SQLite trong phản hồi dài 994 byte. Một số biểu thức ban đầu tiếp tục gây lỗi. Dựa trên phản hồi, KAgent điều chỉnh cách kết thúc giá trị đầu vào để thu được phép thử đối chứng TRUE/FALSE, như Hình 3.12 minh họa.



Hình 3.12. Điều chỉnh phép thử SQLi theo phản hồi của ứng dụng

Giá trị `q=%' AND 1=1 AND '1%'='1` trả HTTP 200 và danh sách sản phẩm dài 16.563 byte. Khi chỉ đổi `1=1` thành `1=2`, phản hồi vẫn là HTTP 200 nhưng có tập rỗng 30 byte. Cùng với lỗi SQLite ở phép thử dấu nháy đơn, sự khác biệt này là cơ sở để KAgent ghi nhận `confirmed`. Tuy nhiên, một phản hồi nhánh đúng đầy đủ còn thiếu để hoàn thiện cặp đối chứng độc lập thứ hai.

Cấu trúc truy vấn SQL nội bộ chỉ được suy luận từ phản hồi và chưa được kiểm tra qua mã nguồn. Các phép thử cũng chưa kiểm tra việc đọc dữ liệu ngoài danh mục sản phẩm hoặc thay đổi dữ liệu trong cơ sở dữ liệu.

Tổng hợp kết quả thực nghiệm

Bảng 3.4 so sánh kết quả xác minh và Finding được lưu tại ba Candidate SQL Injection.

Bảng 3.4. Kết quả xác minh SQL Injection tại ba Candidate

Điểm kiểm tra	Quan sát chính	Kết quả xác minh	Finding
`GET /api/Products` tham số `q`	Các giá trị chứa dấu nháy đơn và biểu thức SQL đã thử đều trả về tập rỗng. Chưa có bằng chứng cho thấy biểu thức SQL trong đầu vào được thực thi.	`not-confirmed`	Không tạo

Điểm kiểm tra	Quan sát chính	Kết quả xác minh	Finding
`POST /rest/user/login` trường `email`	Điều kiện đúng trả phản hồi đăng nhập thành công kèm token, điều kiện sai trả thông báo đăng nhập thất bại. Kết quả lặp lại qua hai cặp đối chứng độc lập.	`confirmed`	Đã lưu, mức critical
`GET /rest/products/search` tham số `q`	Ứng dụng trả lỗi SQLite khi thêm dấu nháy đơn. Điều kiện đúng trả danh sách sản phẩm, điều kiện sai trả tập rỗng.	`confirmed`	Đã lưu, mức high

Kết quả gồm hai Candidate `confirmed` và một Candidate `not-confirmed`. Từ hai Candidate đã xác nhận, KAgent lưu hai Finding SQL Injection kèm liên kết bằng chứng, với mức critical và high do Agent đánh giá. Hình 3.13 minh họa kết quả lưu Finding.

```
◆ Finding saved · SQL injection in login email field allows authentication bypass as the admin account (critical)

◆ Finding saved · SQL injection in product search q parameter (SQLite LIKE predicate) (high)
```

Hình 3.13. Lưu hai Finding SQL Injection sau xác minh

Bảng 3.5 ghi nhận Skill chính, công cụ tiêu biểu, số bước và lượt gọi công cụ theo từng giai đoạn. Một bước tương ứng với một chu kỳ của vòng lặp Agent và có thể phát sinh nhiều lượt gọi công cụ. Số liệu được phân theo thời điểm nạp Skill chính, bao gồm cả thao tác thực hiện lại.

Bảng 3.5. Sử dụng kỹ năng và công cụ trong thực nghiệm

Giai đoạn	Skill chính	Công cụ tiêu biểu	Số bước	Lượt gọi công cụ
Khảo sát	`recon`	`permissions_status`, `http`, `service_discovery`	6	9
Liệt kê tài nguyên	`web-enumeration`	`http`, `read_discovery_source`, `content_discovery`, `workflow`	30	42
Phân tích đầu vào	`web-input-analysis`	`http`, `workflow`, `ask_user`	17	27

Giai đoạn	Skill chính	Công cụ tiêu biểu	Số bước	Lượt gọi công cụ
Xác minh SQLi	`sql-injection`	`http`, `read_payloads`, `workflow`, `confirm_finding`	57	58

Bảng 3.6 tổng hợp các kết quả được ghi nhận qua những giai đoạn thực nghiệm.

Bảng 3.6. Tổng hợp kết quả qua các giai đoạn thực nghiệm

Giai đoạn	Kết quả
Liệt kê đầu vào	Ghi nhận 8 đầu vào.
Phân tích đầu vào	Phân tích 6 đầu vào, loại 2 đầu vào B2B theo quyết định của người kiểm thử.
Lựa chọn điểm xác minh	Tạo 3 Candidate SQL Injection.
Xác minh	Ghi nhận 2 kết quả `confirmed` và 1 kết quả `not-confirmed`.
Lưu phát hiện	Lưu 2 Finding SQL Injection.

e) Nhận xét và đánh giá

Thực nghiệm cho thấy KAgent có khả năng tự tổ chức quá trình kiểm thử từ địa chỉ mục tiêu ban đầu, thực hiện qua các giai đoạn khảo sát, liệt kê tài nguyên, phân tích đầu vào và xác minh SQL Injection. Trong quá trình này, Planner hỗ trợ định hướng công việc, LLM phân tích phản hồi và lựa chọn phép thử, còn các Skill và công cụ hỗ trợ thực hiện từng nhiệm vụ. Agent cũng thể hiện khả năng điều chỉnh phép thử và bổ sung dữ liệu dựa trên kết quả quan sát. Tuy nhiên, chất lượng bằng chứng giữa các kết quả xác minh chưa đồng đều. Chức năng đăng nhập có hai cặp đối chứng TRUE/FALSE độc lập, trong khi Product Search còn thiếu một phản hồi đầy đủ để hoàn thiện cặp đối chứng thứ hai. Kết quả `not-confirmed` tại `/api/Products` không đồng nghĩa với việc endpoint an toàn. Ngoài ra, phạm vi khảo sát còn chịu ảnh hưởng của giới hạn thu thập dữ liệu và quyết định của người kiểm thử. Những kết quả này cho thấy khả năng hỗ trợ kiểm thử qua nhiều giai đoạn của KAgent, đồng thời phản ánh các giới hạn về độ bao phủ, chất lượng xác minh và mức độ tự chủ.

3.3.2. Thực nghiệm phối hợp KAgent với Burp Suite

a) Mục tiêu thực nghiệm

Kịch bản này đánh giá khả năng phối hợp giữa Burp Suite và KAgent trong việc kiểm tra XSS đối với một yêu cầu HTTP do người dùng lựa chọn. Burp Suite cung cấp yêu cầu đã thu thập làm dữ liệu đầu vào. KAgent sử dụng Skill tương ứng để phân tích phản hồi của ứng dụng, xây dựng giả thuyết và điều phối các công cụ kiểm chứng. Trọng tâm của thực nghiệm là quá trình xử lý từ yêu cầu đầu vào đến kết luận xác minh, kèm theo bằng chứng làm cơ sở cho kết luận.

b) Cấu hình thực nghiệm

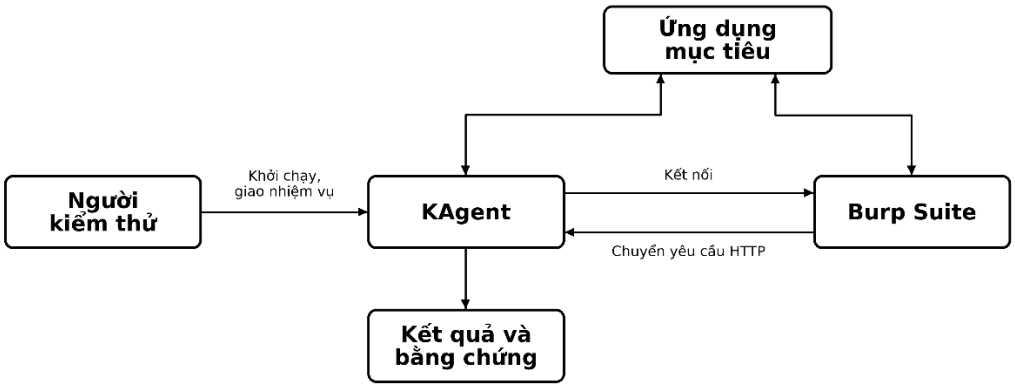
Đối tượng thực nghiệm là chức năng tìm kiếm bài viết trong một bài lab thuộc PortSwigger Web Security Academy [17]. Giá trị tìm kiếm được đưa vào nội dung HTML và một chuỗi JavaScript. Trong chuỗi này, ứng dụng thêm dấu gạch chéo ngược trước dấu nháy đơn hoặc dấu gạch chéo ngược có trong đầu vào để các ký tự đó được xử lý như dữ liệu của chuỗi. Môi trường thực nghiệm gồm Burp Suite và KAgent sử dụng mô hình deepseek-flash của DeepSeek. Giới hạn xử lý được đặt ở 100 bước nhằm kiểm soát thời gian và tài nguyên sử dụng. Phạm vi kiểm tra được giới hạn trong bài lab này.

Trong quá trình thực nghiệm, người kiểm thử bật chế độ YOLO để KAgent tự động chấp thuận các thao tác thuộc chính sách cho phép. Các điều kiện về phạm vi, quyền thực thi và giới hạn tài nguyên vẫn được duy trì.

Yêu cầu HTTP được chọn là `GET /?search=test`, với tham số `search` là đầu vào cần kiểm tra. Dựa trên dữ liệu do Burp Suite cung cấp, KAgent lựa chọn phép thử và xác minh khả năng khai thác XSS tại tham số này. Kịch bản tập trung đánh giá khả năng phân tích và kiểm chứng một đầu vào đã xác định, không đánh giá khả năng khám phá toàn bộ bề mặt tấn công của ứng dụng.

c) Quy trình thực nghiệm

Quy trình thực nghiệm được thực hiện thông qua sự phối hợp giữa người kiểm thử, Burp Suite và KAgent. Burp Suite hỗ trợ thu thập và chuyển yêu cầu HTTP, còn KAgent tiếp nhận dữ liệu để thực hiện kiểm thử và ghi nhận kết quả. Quy trình tổng quát được minh họa tại Hình 3.14.

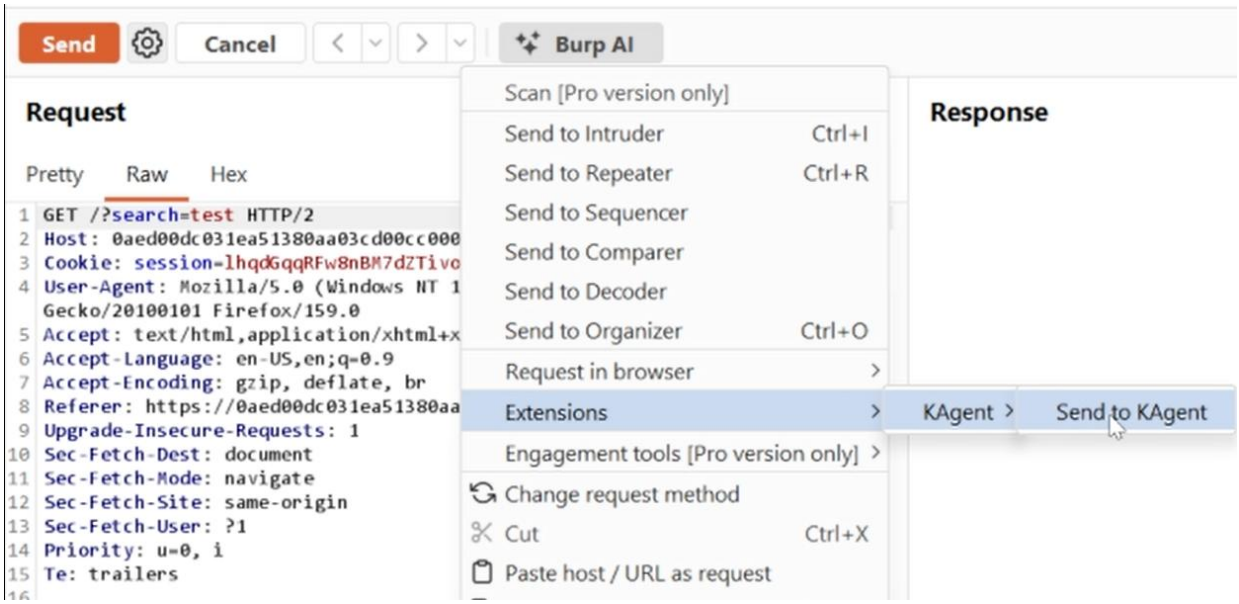


Hình 3.14. Quy trình phối hợp Burp Suite và KAgent

1. **Chuẩn bị và thiết lập phạm vi:** Người kiểm thử khởi tạo KAgent và xác định phạm vi được phép kiểm thử trên bài lab XSS.
2. **Thu thập và chuyển yêu cầu HTTP:** Người kiểm thử sử dụng Burp Suite để thu thập yêu cầu tìm kiếm và chuyển sang KAgent bằng Send to KAgent. Việc chuyển yêu cầu không tự cấp quyền thực hiện mọi phép thử. Các hành động tiếp theo vẫn tuân thủ phạm vi và cơ chế kiểm soát quyền của KAgent.
3. **Lựa chọn đầu vào và phân tích ngữ cảnh:** Người kiểm thử chọn yêu cầu và chỉ định tham số `search`. KAgent sử dụng Skill XSS để phân tích vị trí phản xạ và cách ứng dụng xử lý dữ liệu đầu vào.
4. **Xây dựng và xác minh phép thử:** KAgent lựa chọn phép thử phù hợp, gửi yêu cầu HTTP, phân tích phản hồi và đánh giá bằng chứng XSS.
5. **Ghi nhận kết quả:** KAgent lưu kết quả xác minh và bằng chứng trong Workflow, đồng thời lưu Finding khi đáp ứng điều kiện ghi nhận.

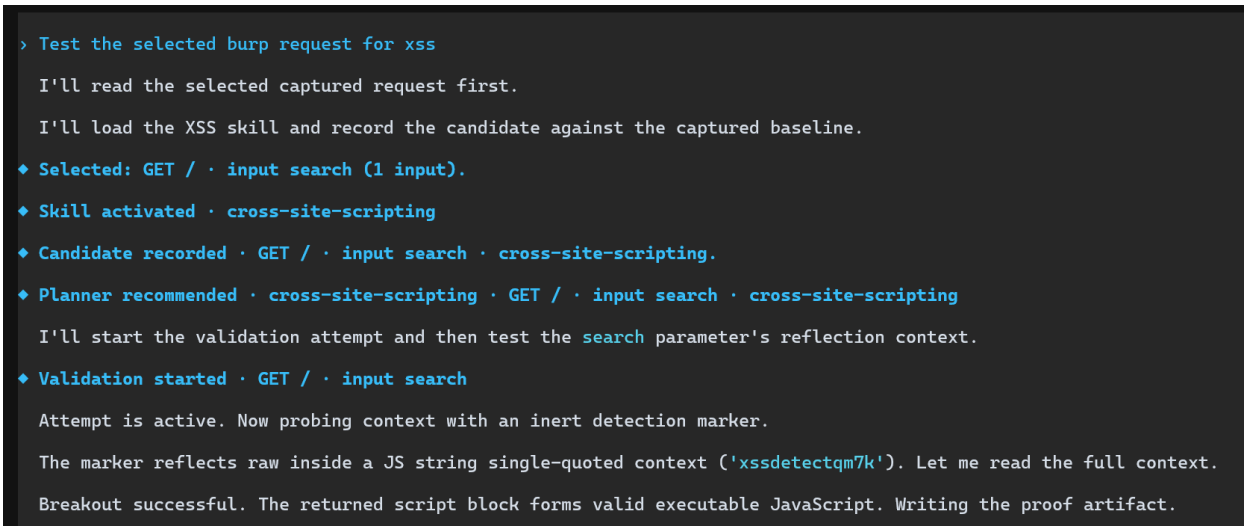
d) Kết quả thực nghiệm

KAgent tiếp nhận yêu cầu `GET /?search=test` từ Burp Repeater qua Send to KAgent, như Hình 3.15. Tham số `search` được lựa chọn để kiểm tra XSS.



Hình 3.15. Chuyển yêu cầu tìm kiếm từ Burp Repeater tới KAgent

KAgent nạp Skill XSS và ghi nhận một Candidate tại tham số `search`. Hình 3.16 minh họa yêu cầu được lựa chọn và quá trình xác minh.



Hình 3.16. Nạp Skill XSS và triển khai xác minh tham số search

Kết quả phân tích và xác minh XSS

Giá trị đánh dấu `xssdetectqm7k` xuất hiện trong phần tử `h1` và trong chuỗi JavaScript gán cho biến `searchTerms`. Hai vị trí này cung cấp cơ sở phân tích cách xử lý đầu vào theo từng ngữ cảnh phản xạ.

Với đầu vào chứa `</script><script>document.title=731942</script>`, các ký tự tạo thẻ được mã hóa tại `h1` nhưng vẫn xuất hiện trong phản hồi ở khối script. Chuỗi `</script>` có thể kết thúc khối script ban đầu. Phần đầu vào theo sau chứa thẻ `<script>` có thể tạo một khối script mới với lệnh thay đổi tiêu đề trang.

Khi bổ sung dấu nháy đơn trước chuỗi đóng thẻ, phản hồi được đọc lại qua `read_http_observation` cho thấy dấu nháy đơn được escape thành `\'`, trong khi `</script>` vẫn giữ nguyên. Việc escape dấu nháy đơn vì vậy không ngăn được cách kết thúc khối script ban đầu. Cơ sở xác minh là cấu trúc HTML và JavaScript trong phản hồi HTTP. Việc thực thi JavaScript trên trình duyệt chưa được quan sát trực tiếp.

Tổng hợp kết quả thực nghiệm

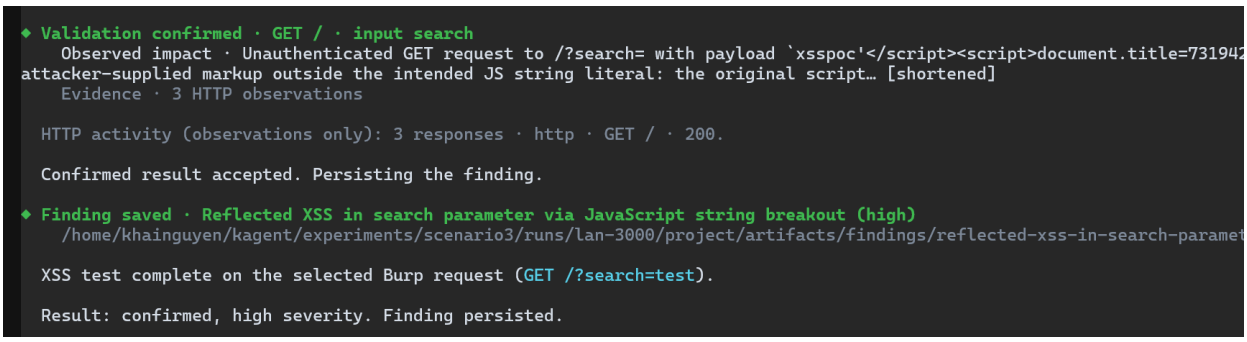
Bảng 3.7 tổng hợp các phép thử, phản hồi quan sát được và nhận định của KAgent tại tham số `search`.

Bảng 3.7. Các phép thử và kết quả phân tích tại tham số search

Phép thử	Giá trị tham số `search`	Kết quả quan sát và nhận định
Xác định vị trí xuất hiện của đầu vào	`xssdetectqm7k`	Dữ liệu xuất hiện trong phần tử `h1` và chuỗi JavaScript của biến `searchTerms`. Agent xác định hai vị trí cần kiểm tra.
Kiểm tra khả năng thoát khỏi script	`xsspoc</script><script>document.title=731942</script>`	Các ký tự tạo thẻ được mã hóa tại `h1` nhưng được giữ nguyên trong khối script. Agent nhận định đầu vào có thể kết thúc khối script ban đầu và tạo khối script mới.
Kiểm tra cách xử lý dấu nháy đơn	`xsspoc'</script><script>document.title=731942</script>`	Dấu nháy đơn được escape thành `\'`, còn chuỗi `</script>` vẫn được giữ nguyên. Agent nhận định việc escape dấu nháy đơn chưa ngăn được cách chèn mã này.

Ba phản hồi HTTP được lưu và liên kết làm bằng chứng cho quá trình phân tích và kết luận xác minh. Lượt thực nghiệm sử dụng 17 lần gọi LLM, 17 lượt gọi công cụ và ba yêu cầu HTTP mới.

Từ một Candidate XSS, KAgent ghi nhận một `ValidationResult confirmed`, cập nhật Coverage và lưu một Finding XSS mức high, thuộc CWE-79 và nhóm OWASP A03:2021 Injection. Hình 3.17 thể hiện kết quả xác minh, ba phản hồi HTTP làm bằng chứng và thông báo lưu Finding.



Hình 3.17. Kết quả xác minh và thông báo lưu Finding XSS

e) Nhận xét và đánh giá

Thực nghiệm cho thấy KAgent có thể phối hợp với Burp Suite để kiểm tra yêu cầu HTTP do người kiểm thử lựa chọn. Từ dữ liệu được chuyển sang, KAgent sử dụng Skill XSS để phân tích phản hồi, lựa chọn phép thử và đánh giá bằng chứng. Agent phân biệt được cách ứng dụng xử lý cùng một đầu vào tại phần tử `h1` và trong khối script, từ đó xác định hướng kiểm tra phù hợp với ngữ cảnh phản xạ. Các phản hồi HTTP được lưu và liên kết với kết quả xác minh, giúp người kiểm thử đối chiếu cơ sở kết luận. Tuy nhiên, kết quả xác minh mới dựa trên cấu trúc HTML và JavaScript trong phản hồi, chưa có quan sát trực tiếp việc thực thi mã trên trình duyệt. Bên cạnh đó, kịch bản chỉ tập trung vào một đầu vào được lựa chọn, chưa đánh giá khả năng tự khám phá toàn bộ bề mặt tấn công của ứng dụng.

3.4. Hạn chế và hướng phát triển

KAgent là nguyên mẫu Single-Agent kết hợp LLM, Skill, công cụ và cơ chế quản lý trạng thái. Phần này phân tích những hạn chế còn lại dựa trên thiết kế, phạm vi hiện thực và kết quả thực nghiệm, từ đó đề xuất hướng cải tiến.

3.4.1. Hạn chế của hệ thống

Khả năng điều phối và quản lý nhiệm vụ: KAgent duy trì tiến độ dựa trên quy tắc lựa chọn Skill, trạng thái quy trình, mức ưu tiên và bằng chứng của Candidate. Tuy vậy, việc lựa chọn phép thử và diễn giải thông tin mới vẫn phụ thuộc vào suy luận của LLM. Cách xếp hạng công việc chưa phản ánh đầy đủ độ tin cậy của giả thuyết, giá trị thông tin dự kiến và chi phí thực hiện. Vì thế, một thứ tự xử lý hợp lệ theo quy trình chưa bảo đảm các điểm kiểm tra quan trọng được ưu tiên phù hợp. Khi chuyển sang nhiệm vụ mới trong cùng phiên, mục tiêu và trạng thái cũ có thể ảnh hưởng đến việc xử lý yêu cầu hiện tại. Khả năng phân biệt nhiệm vụ và cập nhật trạng thái cần được đánh giá thêm.

Phạm vi kiểm thử và chất lượng Skill: KAgent hiện có hướng dẫn xác minh chuyên biệt cho SQL Injection và Cross-Site Scripting. Các nhóm lỗ hổng khác cần được xử lý bằng quy trình tổng quát hoặc Skill bổ sung. Hiệu quả kiểm thử phụ thuộc vào tính chính xác, mức độ đầy đủ và điều kiện áp dụng của hướng dẫn. Các kiểm tra về cấu trúc và điều kiện thực thi chưa đủ để đánh giá chất lượng lựa chọn phép thử và phân tích phản hồi. KAgent có thể gửi yêu cầu HTTP và tiếp nhận lưu lượng từ Burp Suite, nhưng chưa có công cụ điều khiển trình duyệt để tương tác với giao diện phía máy khách.

Độ tin cậy của kết quả kiểm thử: KAgent đã có các kiểm tra về nguồn gốc, liên kết với lượt xác minh, tính toàn vẹn và độ đầy đủ của dữ liệu bằng chứng. Những kiểm tra này hỗ trợ ghi nhận và sử dụng bằng chứng đúng nguồn, nhưng không tự bảo đảm kết luận về lỗ hổng là chính xác. Chất lượng phép đối chứng và cách diễn giải ý nghĩa bảo mật của phản hồi vẫn phụ thuộc vào phương pháp kiểm thử và suy luận của LLM. Hallucination có thể dẫn đến kết luận vượt quá bằng chứng. Công cụ ghi nhận Finding sử dụng kết quả xác minh đã được chấp nhận cùng bằng chứng liên quan, không tự thực hiện một lượt kiểm chứng độc lập.

Quản lý ngữ cảnh và khôi phục phiên: KAgent có cơ chế thu gọn lịch sử, truy hồi Memory và Intelligence, đồng thời duy trì trạng thái quy trình riêng. Tuy nhiên, bản tóm lược và dữ liệu thu nhận trong giới hạn dung lượng có thể không giữ đủ chi tiết cần cho những bước phân tích sau. Dữ liệu hai phiên thực nghiệm cho thấy Intelligence ghi nhận thông tin về lỗi SQLite và sự khác biệt trong xử lý đầu vào ở ngữ cảnh HTML, JavaScript. Tuy nhiên, tính chính xác của tri thức được trích xuất và hiệu quả tái sử dụng chưa được đánh giá độc lập. Thông tin tái sử dụng vẫn phải được kiểm tra với trạng thái hiện tại của mục tiêu. Khi khôi phục phiên, hệ thống chỉ nạp lại dữ liệu đã lưu. Việc nạp lại dữ liệu không tái lập môi trường ứng dụng, tiến trình công cụ hoặc quyền tạm thời. Các điều kiện tiếp tục kiểm thử vì vậy phải được kiểm tra lại.

Chi phí vận hành và hiệu năng: Khi sử dụng Cloud API, các phiên kiểm thử phát sinh chi phí token và chịu ảnh hưởng của kết nối mạng, giới hạn dịch vụ và chính sách nhà cung cấp. Nhiệm vụ nhiều bước có thể làm tăng tổng chi phí. Local LLM giúp chủ động hơn trong quản lý dữ liệu và giảm phụ thuộc vào dịch vụ bên ngoài. Tuy nhiên, phương án này vẫn có chi phí phần cứng, điện năng và vận hành, đồng thời cần được đánh giá về thời gian đáp ứng và khả năng sử dụng công cụ. KAgent đã ghi nhận số token sử dụng và thời gian thực hiện yêu cầu tới LLM, nhưng mức độ đầy đủ của số liệu còn phụ thuộc vào phản hồi của nhà cung cấp. Các chỉ số

về số bước và lượt gọi công cụ trong thực nghiệm chưa thay thế phép đo đầy đủ về token, chi phí dịch vụ và tài nguyên hạ tầng khi quy mô kiểm thử tăng lên. Tổng chi phí cần được đánh giá theo mô hình, kích thước ngữ cảnh, quy mô nhiệm vụ và phương thức triển khai.

An toàn bảo mật của hệ thống: KAgent đã có cơ chế kiểm soát quyền, phạm vi và xử lý thông tin nhạy cảm. Khả năng cô lập môi trường thực thi và bảo vệ dữ liệu vẫn còn giới hạn. Khi sử dụng Cloud API, nội dung phục vụ phân tích có thể được gửi tới nhà cung cấp LLM. Vì vậy, việc chia sẻ thông tin nội bộ, dữ liệu xác thực và bằng chứng phải được kiểm soát. Các rủi ro trong OWASP Top 10 for LLM Applications cần được đánh giá thêm. Phạm vi kiểm chứng hiện tại chưa đủ để kết luận hệ thống an toàn trong mọi điều kiện, nhất là khi xử lý dữ liệu nhạy cảm hoặc thao tác có ảnh hưởng lớn.

Xét chung hai kịch bản, phạm vi đánh giá còn giới hạn ở OWASP Juice Shop và bài lab XSS đã lựa chọn trên Web Security Academy. Một lượt thực nghiệm trong từng điều kiện cụ thể chưa đủ để khái quát tính ổn định, khả năng tái lập hoặc hiệu quả trên các ứng dụng khác. Các kết quả vì vậy phản ánh những năng lực và hạn chế đã quan sát, cần được kiểm tra thêm qua các lượt thực nghiệm lặp có kiểm soát.

3.4.2. Định hướng phát triển hệ thống

Cải thiện điều phối và quản lý nhiệm vụ: Trên nền tảng Single-Agent hiện có, hướng cải tiến là đưa độ tin cậy của giả thuyết, giá trị thông tin dự kiến và chi phí thực hiện vào cơ chế lựa chọn phép thử. Quá trình lựa chọn cũng phải xét đến các quan sát đã thu được và bằng chứng còn thiếu. Cơ chế cập nhật mục tiêu và chuyển nhiệm vụ cần được hoàn thiện để trạng thái workflow phù hợp với yêu cầu đang xử lý. Hiệu quả cải tiến cần được đánh giá qua mức độ hoàn thành nhiệm vụ và chất lượng xác minh. Hoạt động kiểm thử vẫn phải tuân thủ phạm vi và quyền hạn được cấp.

Mở rộng Skill và công cụ kiểm thử: Các Skill mới và hiện có đều cần được rà soát để bảo đảm hướng dẫn chính xác, nêu rõ điều kiện áp dụng, yêu cầu đầu ra và điều kiện dừng. Mỗi Skill xác minh cần được đánh giá trên ba nhóm trường hợp: có lỗ hổng, không có lỗ hổng và chưa đủ điều kiện kết luận. Quá trình đánh giá cần xem xét chất lượng phép đối chứng, bằng chứng và đối chiếu hiệu quả của hướng dẫn khi sử dụng các LLM khác nhau. Những công cụ bổ sung, nhất là tự động hóa trình duyệt, cần cung cấp dữ liệu quan sát có thể liên kết với workflow.

Một hướng mở rộng là nghiên cứu tích hợp công cụ bên ngoài thông qua Model Context Protocol (MCP) để tăng khả năng kết nối và sử dụng công cụ của KAgent. Việc triển khai cần kiểm soát nguồn công cụ, tham số, quyền thực thi và phạm vi kiểm thử. Đối với công cụ chạy dưới dạng tiến trình, cần có cơ chế cô lập phù hợp, không mặc định cho phép thực thi trực tiếp trên máy vận hành.

Nâng cao độ tin cậy của kết quả: Các tiêu chí đánh giá cần được hoàn thiện theo từng lớp lỗ hổng, kế thừa các kiểm tra về nguồn gốc và tính toàn vẹn của bằng chứng. Quá trình xác minh phải làm rõ mối liên hệ giữa yêu cầu gửi đi, phản hồi, phép đối chứng và tác động được kết luận. Nếu bằng chứng chưa đủ, kết quả cần được giữ ở trạng thái phù hợp để kiểm tra bổ sung. Khi đánh giá hệ thống, Finding cần được thẩm định độc lập để xác định độ chính xác của kết luận.

Cải thiện quản lý ngữ cảnh và khôi phục phiên: Thông tin trong ngữ cảnh cần được lựa chọn theo yêu cầu của nhiệm vụ và giữ tham chiếu tới nguồn. Cơ chế thu gọn cũng cần được đánh giá về mức độ mất thông tin. Khi tái sử dụng Memory và Intelligence, hệ thống cần kiểm tra sự phù hợp của thông tin với công nghệ, trạng thái mục tiêu và điều kiện áp dụng hiện tại. Việc tiếp tục phiên cũng cần có bước kiểm tra những điều kiện có thể đã thay đổi. Thông tin truy hồi phục vụ phân tích, còn quyền thực thi và phạm vi kiểm thử phải được xác định từ trạng thái hiện hành.

Tối ưu tài nguyên và chi phí LLM: Số token sử dụng, thời gian xử lý và chi phí cần được lưu và tổng hợp theo từng phiên. Khi thống kê, cần phân biệt dữ liệu do nhà cung cấp trả về với ước tính ngữ cảnh của KAgent. Việc so sánh Cloud API và Local LLM nên xét chất lượng thực hiện nhiệm vụ, khả năng sử dụng công cụ, độ trễ và tổng chi phí triển khai. Tổng chi phí bao gồm dịch vụ, phần cứng, điện năng và vận hành. Từ số liệu so sánh, có thể nghiên cứu cơ chế kiểm soát ngân sách và lựa chọn cấu hình phù hợp với điều kiện triển khai.

Tăng cường an toàn bảo mật: Công tác rà soát mã nguồn và kiểm thử tích hợp cần được mở rộng để bao quát các trường hợp biên, lỗi và hủy tác vụ giữa các thành phần. Hướng nghiên cứu tiếp theo là cô lập môi trường thực thi công cụ và giới hạn quyền của tài khoản vận hành, kết hợp với các cơ chế kiểm soát hiện có. Quá trình đánh giá an toàn cần xem xét cách bảo vệ khóa API, dữ liệu phiên, bằng chứng và nội dung trao đổi với LLM. Các kịch bản Prompt Injection, tiết lộ thông tin và thao túng công cụ cũng cần được đưa vào đánh giá.

Định hướng Multi-Agent: Các nhiệm vụ khảo sát, phân tích và thẩm định bằng chứng có thể được phân chia cho nhiều tác tử. Phương án này cần được so sánh với Single-Agent về chất lượng kết quả, chi phí và khả năng phối hợp trạng thái.

Multi-Agent vẫn là hướng nghiên cứu, chưa phải chức năng đã hiện thực. Trước khi mở rộng kiến trúc, cần ưu tiên hoàn thiện chất lượng Skill, độ tin cậy của xác minh và an toàn hệ thống.

Các cải tiến cần được đánh giá qua thực nghiệm lặp với điều kiện được xác định rõ. Chất lượng xác minh, mức độ hoàn thành nhiệm vụ, độ bao phủ và tài nguyên tiêu thụ cần được theo dõi riêng. Mọi hoạt động phải tiếp tục tuân thủ nguyên tắc kiểm thử có ủy quyền, giới hạn phạm vi và khả năng truy vết.

3.5. Kết luận chương 3

Chương 3 đã trình bày quá trình xây dựng, triển khai và đánh giá hệ thống KAgent dựa trên kiến trúc và các cơ chế đã thiết kế ở Chương 2. Các nội dung hiện thực cho thấy sự kết hợp giữa mô hình ngôn ngữ lớn, hệ thống kỹ năng, công cụ kiểm thử và cơ chế quản lý trạng thái, bằng chứng nhằm hỗ trợ thực hiện các tác vụ kiểm thử xâm nhập ứng dụng web trong phạm vi được kiểm soát.

Thông qua hai kịch bản thực nghiệm, đồ án đã khảo sát khả năng hoạt động của KAgent trong những điều kiện đầu vào khác nhau. Kịch bản kiểm thử SQL Injection từ địa chỉ mục tiêu ban đầu thể hiện quá trình khám phá ứng dụng, phân tích đầu vào, lựa chọn và xác minh các đối tượng nghi vấn. Kịch bản tích hợp Burp Suite cho thấy khả năng tiếp nhận yêu cầu HTTP, phân tích và xác minh XSS tại đầu vào được lựa chọn. Kết quả thực nghiệm ghi nhận khả năng phối hợp giữa suy luận của LLM và hoạt động thực thi công cụ, điều chỉnh phép thử dựa trên phản hồi, đồng thời lưu trữ kết quả cùng bằng chứng liên quan.

Bên cạnh những kết quả đạt được, quá trình đánh giá cũng cho thấy KAgent vẫn còn hạn chế về khả năng điều phối, mức độ hoàn tất nhiệm vụ, độ tin cậy của kết luận và phạm vi kiểm thử. Những hạn chế này là cơ sở để đề xuất các hướng cải tiến về lựa chọn phép thử, xác minh bằng chứng, mở rộng khả năng quan sát và đánh giá hiệu quả hệ thống. Kết quả Chương 3 cho thấy tính khả thi của việc ứng dụng AI tạo sinh để hỗ trợ một phần quy trình kiểm thử xâm nhập, đồng thời xác định những vấn đề cần tiếp tục hoàn thiện để nâng cao khả năng hoạt động của KAgent.

KẾT LUẬN

Đề tài "Nghiên cứu ứng dụng AI tạo sinh trong kiểm thử xâm nhập" được thực hiện nhằm nghiên cứu khả năng ứng dụng mô hình ngôn ngữ lớn (LLM) trong việc hỗ trợ các hoạt động kiểm thử xâm nhập ứng dụng web. Trên cơ sở tìm hiểu các phương pháp kiểm thử xâm nhập, mô hình ngôn ngữ lớn và kiến trúc AI Agent, đồ án đã lựa chọn hướng tiếp cận Single-Agent để thiết kế và xây dựng hệ thống KAgent, kết hợp khả năng phân tích của LLM với các công cụ kiểm thử trong một quy trình xử lý nhiều bước.

Trong quá trình thực hiện, đồ án đã phân tích yêu cầu, thiết kế kiến trúc và hiện thực các thành phần chính của KAgent. Hệ thống cho phép tiếp nhận nhiệm vụ, lựa chọn kỹ năng và công cụ phù hợp, thực hiện các thao tác kiểm thử và xử lý kết quả trả về để xác định bước tiếp theo. Bên cạnh đó, các cơ chế quản lý trạng thái, lưu trữ bằng chứng và ghi nhận phát hiện được xây dựng nhằm hỗ trợ theo dõi tiến trình và truy vết kết quả kiểm thử. Các hoạt động thực thi được đặt trong phạm vi và quyền hạn do người kiểm thử thiết lập, qua đó duy trì vai trò giám sát của con người trong quá trình sử dụng hệ thống.

Để đánh giá khả năng hoạt động của KAgent, đồ án đã tiến hành hai kịch bản thực nghiệm đối với các lỗ hổng SQL Injection và Cross-Site Scripting. Kịch bản thứ nhất khảo sát khả năng tự động khám phá các điểm đầu vào, lựa chọn và thực hiện phép thử trên ứng dụng web mục tiêu. Kịch bản thứ hai đánh giá khả năng phối hợp với Burp Suite để tiếp nhận yêu cầu HTTP và hỗ trợ xác minh lỗ hổng XSS. Kết quả thực nghiệm cho thấy KAgent có thể thực hiện các bước kiểm thử liên tiếp, sử dụng phản hồi từ công cụ để điều chỉnh hướng kiểm tra và ghi nhận các phát hiện kèm theo bằng chứng. Tuy nhiên, một số kết quả vẫn cần được kiểm chứng bổ sung, đồng thời khả năng hoàn thành toàn bộ nhiệm vụ chưa được bảo đảm trong mọi trường hợp khảo sát.

Nhìn chung, đồ án đã hoàn thành việc xây dựng nguyên mẫu KAgent và bước đầu đánh giá khả năng ứng dụng AI tạo sinh trong hỗ trợ kiểm thử xâm nhập ứng dụng web. Kết quả đạt được cho thấy tính khả thi của việc kết hợp LLM với cơ chế điều phối, quản lý trạng thái và thực thi công cụ. Tuy vậy, phạm vi kiểm thử của hệ thống còn phụ thuộc vào các kỹ năng và công cụ được hỗ trợ. Việc lựa chọn phép thử, đánh giá bằng chứng và duy trì tiến trình kiểm thử cũng cần tiếp tục được cải thiện để nâng cao độ tin cậy của kết quả.

Trong thời gian tới, hệ thống có thể được phát triển theo hướng hoàn thiện cơ chế điều phối, mở rộng khả năng quan sát và kiểm thử, đồng thời tăng cường các điều kiện xác minh trước khi ghi nhận phát hiện. Việc thực nghiệm trên nhiều mục tiêu và điều kiện khác nhau cũng là cần thiết để đánh giá đầy đủ hơn khả năng hoạt động, tính ổn định và những giới hạn của hệ thống. Ngoài ra, có thể nghiên cứu bổ sung các cơ chế khai thác tri thức bảo mật theo ngữ cảnh và mở rộng cách tổ chức tác vụ khi yêu cầu kiểm thử trở nên phức tạp hơn. Những hướng phát triển này cần đi kèm với việc kiểm soát phạm vi thực thi và bảo đảm khả năng truy vết kết quả.

Thông qua quá trình nghiên cứu, thiết kế và thực nghiệm, đồ án đã hình thành một nguyên mẫu AI Pentest Agent làm cơ sở cho việc tiếp tục nghiên cứu ứng dụng AI tạo sinh trong kiểm thử xâm nhập. KAgent chưa thay thế vai trò của kiểm thử viên, nhưng có thể hỗ trợ tổ chức và thực hiện một số hoạt động kiểm thử, qua đó tạo tiền đề cho việc phát triển các hệ thống hỗ trợ kiểm thử có khả năng tự động hóa cao hơn.

TÀI LIỆU THAM KHẢO

[1] NIST, “Technical Guide to Information Security Testing and Assessment,” SP 800-115, 2008. https://csrc.nist.gov/pubs/sp/800/115/final

[2] PTES Team, “The Penetration Testing Execution Standard.” http://www.pentest-standard.org/

[3] OWASP Foundation, “OWASP Web Security Testing Guide, Version 4.2.” https://owasp.org/www-project-web-security-testing-guide/

[4] OWASP Foundation, “OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks,” 2021. https://owasp.org/Top10/2021/

[5] A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017. https://arxiv.org/abs/1706.03762

[6] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” NeurIPS, 2022. https://arxiv.org/abs/2201.11903

[7] T. Kojima et al., “Large Language Models are Zero-Shot Reasoners,” NeurIPS, 2022. https://arxiv.org/abs/2205.11916

[8] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” ICLR, 2023. https://arxiv.org/abs/2210.03629

[9] L. Wang et al., “A Survey on Large Language Model based Autonomous Agents,” arXiv:2308.11432, 2023. https://arxiv.org/abs/2308.11432

[10] A. B. Hassouna, H. Chaari, and I. Belhaj, “LLM-Agent-UMF: LLM-based Agent Unified Modeling Framework for Seamless Integration of Multi Active/Passive Core-Agents,” arXiv:2409.11393v1, 2024. https://arxiv.org/html/2409.11393v1

[11] G. Deng et al., “PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing,” 33rd USENIX Security Symposium, 2024. https://www.usenix.org/conference/usenixsecurity24/presentation/deng

[12] X. Shen et al., “PentestAgent: Incorporating LLM Agents to Automated Penetration Testing,” 20th ACM AsiaCCS, 2025. https://doi.org/10.1145/3708821.3733882

[13] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020. https://arxiv.org/abs/2005.11401

[14] MITRE Corporation, “Common Weakness Enumeration (CWE).” https://cwe.mitre.org/

[15] OWASP Foundation, “OWASP Juice Shop.” https://owasp-juice.shop/

[16] DeepSeek, “Models & Pricing,” DeepSeek API Docs. https://api-docs.deepseek.com/quick_start/pricing/

[17] PortSwigger, “Lab: Reflected XSS into a JavaScript string with single quote and backslash escaped,” Web Security Academy. https://portswigger.net/web-security/cross-site-scripting/contexts/lab-javascript-string-single-quote-backslash-escaped

[18] N. V. Khải, “KAgent - AI-assisted Penetration Testing Agent,” GitHub repository. https://github.com/ynvk404/KAgent

[19] PortSwigger, “Installing extensions manually,” Burp Suite Documentation. https://portswigger.net/burp/documentation/desktop/extend-burp/extensions/installing/manual-install

PHỤ LỤC

Chuẩn bị và cài đặt

KAgent yêu cầu Python 3.11 trở lên. Trên Ubuntu, cần có mạng, Git, môi trường ảo Python và quyền sudo. Chạy các lệnh sau để tải và cài đặt KAgent [18]:

```
sudo apt update
sudo apt install git python3 python3-venv ca-certificates
mkdir -p ~/kagent-demo
cd ~/kagent-demo
git clone https://github.com/ynvk404/KAgent.git kagent
cd kagent
python3 -m venv .venv
source .venv/bin/activate
python -m pip install .
python -m pip check
kagent --version
```

Cài đặt thành công khi `pip check` không báo xung đột và KAgent hiển thị phiên bản. Với terminal mới, vào `~/kagent-demo/kagent` và chạy `source .venv/bin/activate` để kích hoạt lại môi trường.

Cấu hình LLM

KAgent lưu cấu hình tại `~/.kagent/config.json`. Tạo thư mục nếu chưa có và mở tệp:

```
umask 077
mkdir -p ~/.kagent
nano ~/.kagent/config.json
```

Với tệp mới, nhập cấu hình deepseek-flash [16] và thay `API_KEY_CUA_BAN` bằng khóa DeepSeek:

```
{
  "backend": "deepseek", "model": "deepseek-flash",
  "base_url": "", "active_custom_provider_id": null,
  "api_keys": {"deepseek": "API_KEY_CUA_BAN"}
}
```

Nếu tệp đã có cấu hình, chỉ cập nhật các trường cần thiết và giữ lại những trường khác, gồm khóa của các nhà cung cấp khác trong `api_keys`. Trong nano, nhấn Ctrl+O, Enter để lưu và Ctrl+X để thoát.

Giới hạn quyền đọc tệp rồi khởi chạy KAgent:

```
chmod 600 ~/.kagent/config.json
kagent
```

Trong TUI, nhập `/provider`, chọn Show current config để xem cấu hình và Test connection để kiểm tra kết nối. Kết quả mong đợi là Connection OK. Tệp chứa API key dạng văn bản thuần, nên không chia sẻ hoặc đưa vào kho mã nguồn.

Khởi chạy và sử dụng cơ bản

Trong terminal đã kích hoạt môi trường ảo, chạy ví dụ sau với giới hạn 100 bước. Thay địa chỉ bằng ứng dụng đang hoạt động và được phép kiểm thử:

```
kagent --target http://127.0.0.1:3000 --max-steps 100
```

Trong TUI, nhập `/skills`, chọn Skill và nhấn Enter. Nhập yêu cầu, theo dõi kết quả và xem xét phạm vi, hành động trước khi cấp quyền. Dùng `/exit` để thoát. Giới hạn bước không phải là giới hạn số yêu cầu HTTP.

Kết nối Burp Suite

Cài Burp Suite và tải Jython Standalone 2.7.x. Trong Burp, chọn tệp JAR tại Settings → Extensions → Python Environment [19]. Chọn Extensions → Installed → Add, chọn Python và tệp `integrations/burp/kagent_burp.py` [18]. Nếu nạp thành công, tab KAgent sẽ xuất hiện.

Trong TUI KAgent, bật cầu nối và xem thông tin kết nối:

```
/burp 8888
/burp credentials
```

Nhập địa chỉ cầu nối và token vào tab KAgent trong Burp, rồi nhấn Save. Không chia sẻ token. Chọn yêu cầu HTTP thuộc mục tiêu được phép, chọn Send to KAgent, sau đó trở lại TUI để liệt kê và chọn yêu cầu theo mã hiển thị:

```
/burp list
/burp use <id>
```

Kiểm tra yêu cầu và phạm vi trước khi yêu cầu Agent phân tích. Việc chọn yêu cầu không tự cấp quyền thực thi.